



Decentralized Crypto World Bank

A Blockchain-Based Banking Platform
with AI-Enhanced Security and Assistance

by

Md. Bokhtiar Rahman Juboraz

20301138

Md. Mahir Ahnaf Ahmed

20301083

A pre-thesis 1 report submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science

Department of Computer Science and Engineering
BRAC University
February 2026

Declaration

It is hereby declared that

1. The report submitted is our own original work while completing degree at BRAC University.
2. The report does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The report does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md. Bokhtiar Rahman Juboraz

20301138

Md. Mahir Ahnaf Ahmed

20301083

Approval

The pre-thesis 1 report of final year project titled “Decentralized Crypto World Bank: A Blockchain-Based Banking Platform with AI-Enhanced Security and Assistance” submitted by

1. Md. Bokhtiar Rahman Juboraz (20301138)
2. Md. Mahir Ahnaf Ahmed (20301083)

Of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on February 20, 2026.

Examining Committee:

Supervisor:
(Member)

Mr. Annajat Alim Rasel
Senior Lecturer
Department of Computer Science and Engineering
BRAC University

Project Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
BRAC University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi
Chairperson and Associate Professor
Department of Computer Science and Engineering
BRAC University

Ethics Statement

This project is designed to be functional on public blockchain testnets, with no intention for monetary profit, nor to create institutional leverage over decentralized financing. No personal banking data, or private financial data, or codes of existing projects, are used in the development of the prototype platform and training AI/ML data. The full project prototype build is to be demonstrated in the final defence of the project; the platform is expected to be ready for testing and troubleshooting by pre-thesis 2. All the development workflow can be traced by the official GitHub repository for the project, and the planning, coding, testing, and usage of different dependencies for this project is considered open-source with MIT licensing, and will be open for contribution after the submission of the final build.

For the training of the AI models, AI-assisted financial decision-making, risk assessment, privacy features, on-chain and off-chain data processing and data storing, we acknowledge the ethical considerations while making decisions for the development of the project.

Abstract

Modern Institutional Banking is established in layers to obtain better control of global financial flows to create intentional restrictions for equal access to financing capabilities and centralized system. To address the complex layered or tier-based architecture that the global market follows for better governance and balance in distribution of financing, and to demonstrate that a decentralized crypto currency system can be established as a more viable solution to global financing, more fair and equal distribution with advanced security system, *Crypto World Bank* project is built with the intention to show a connecting bridge between Decentralized banking system for transparency in traditional institutional layer system for adaptability of Blockchain technology for the global market. The project reveals that it is possible to build a programmable lending platform with better transparency in global financial flow with better security implementation and how Artificial Intelligence can enhance the experience for better adaptability with the growing demand of AI assistance and security. The integration of a combined stack of blockchain technology into the layered-based banking system with AI enhanced security and assistance has not been recorded or attempted before and the purpose of the project is to show that it is possible to make this combined complex system sustainable and functional while making it adaptable with the growing global financial development demands.

Keywords: Blockchain, Decentralized Finance, Deposit Mobilization, Explainable AI, Financial Sustainability, Group Lending, Hierarchical Lending, Institutional DeFi, Multi-Entity Co-Lending, Oracle-Mediated Risk Analytics, Reserve Management, Smart Contracts

Dedication

Dedicated to our families for their unwavering support throughout our academic journey.

Acknowledgment

We would like to express our sincere gratitude to our panel members for their guidance and support throughout this project. We also thank our supervisor, Mr. Annajiat Alim Rasel, Senior Lecturer at the Department of Computer Science and Engineering, BRAC University, for his guidance and support. Finally, we thank the Department of Computer Science and Engineering at BRAC University for providing us with the resources and academic environment to pursue this work.

Contents

Declaration	2
Approval	3
Ethics Statement	5
Abstract	6
Dedication	7
Acknowledgment	8
List of Tables	iv
List of Figures	vi
List of Formulas	viii
List of Abbreviations	ix
1 Introduction	1
1.1 Background	2
1.2 Rationale and Motivation	3
1.3 Problem Statement	3
1.4 Objectives	4
1.5 Proposed Solution	5
1.6 Methodology in Brief	7
1.7 Scopes and Challenges	8
1.8 Thesis Organization	8
2 Literature Review	10
2.1 Preliminaries	10
2.1.1 Review Methodology (PRISMA-Style Frame)	10
2.2 Review of Existing Research	10
2.2.1 Decentralized Lending and DeFi Protocol Design	10
2.2.2 Machine Learning, Explainable AI, and Extended AI Security	10
2.2.3 Hierarchical Distribution, Cross-Tier Flows, and Group Lending	10
2.2.4 Smart Contract Security and Layer 2 Scalability	11
2.2.5 Monetary Policy, Tokenization, and Institutional Landscape	11
2.2.6 Graph ML, Federated Learning, Regulation, and Identity Primitives	11
2.2.7 Oracle-Mediated ML and Optional LLM Interfaces	11
2.3 Summary of Key Findings	14
3 Specifications, Impact, Ethics, and Project Management	15
3.1 Final Specifications and Requirements	15

3.1.1	Minimum Viable Thesis (MVT) Checklist	16
3.1.2	List of Features	17
3.2	Societal Impact	19
3.3	Environmental Impact	19
3.4	Ethical Issues	20
3.5	Project Management Plan	20
3.5.1	Implementation Phase Plan and Deliverables	20
3.5.2	SDLC Stage Mapping	26
3.6	Risk Management	27
3.7	Economic Analysis	27
3.7.1	Prototype Cost Model	27
3.7.2	Gas Cost and Break-Even	28
3.7.3	Net Present Value and Return on Investment	28
4	Design Alternatives	29
4.1	Design Process or Methodology Overview	29
4.1.1	Process Model Selection	30
4.1.2	Implementation Feasibility and Demonstration Scope	30
4.2	Preliminary Design or Design (Model) Specification	31
4.2.1	System Design Overview	31
4.2.2	High-Level Architecture	31
4.2.3	Blockchain Platform Selection	37
4.2.4	Data Model and Database Design	39
4.2.5	On-Chain and Off-Chain Data Partitioning	47
4.2.6	Operating Context	49
4.2.7	User Taxonomy and Onboarding Flows	50
4.2.8	Banking Modules	52
4.2.9	Multi-Entity and Cross-Tier Capital Operations	55
4.2.10	System Modeling	58
4.2.11	Group Solidarity Lending	72
4.2.12	Governance Framework	73
4.2.13	Five-Layer Defense-in-Depth Security Architecture	76
4.2.14	Threat Model and Security Controls	77
4.2.15	Design Decisions and Alternatives	77
4.2.16	Planned AI/ML Support and Risk-Score Wiring	81
4.2.17	Conversational Agent (Phase III–IV)	82
4.2.18	Real-Time Dashboard Pipeline and Runtime Monitoring	89
5	Evaluation, Analysis, and Final Design	91
5.1	Performance Evaluation	91
5.2	Analysis of Design Solutions	92
5.2.1	Requirements Traceability Matrix	92
5.2.2	Final Design Adjustments	92
5.3	Statistical Analysis	93
5.3.1	Datasets and Benchmark Results	93
5.4	Comparisons and Relationships	94
5.5	Discussion	94
6	Conclusion	95

6.1	Conclusion	95
6.2	Future Work	95
A	Database Schema Reference	97
A.1	Full Relational ERD	97
A.2	Indexing Strategy	99
A.3	Functional Dependencies	100
A.4	Materialized Views and Agent Query Optimisation	101
A.5	Representative SQL Queries	101
B	Agent Harness Reference	104
B.1	Per-User Context and Prompt Assembly	104
B.2	Session Lineage, Compression, and Toolset Scoping	104
B.3	Lifecycle Hooks and Optional Capabilities	105
C	Smart Contract Capabilities	106
	Appendix D: WorldBankReserve Contract Interface	107
	Appendix E: Internal Industry Research Notes	108
	Appendix F: UI Prototype Wireframes (Figma)	109
	References	113

List of Tables

2.1	Literature review summary (Part 1 of 2)	12
2.2	Literature review summary (Part 2 of 2)	13
2.3	Protocol comparison on eleven architectural dimensions	14
3.1	Functional requirements (representative)	15
3.2	Non-functional requirements	16
3.3	MVT deliverable checklist	16
3.4	Complete feature inventory	17
3.5	Phase I task register	21
3.6	Phase II task register	22
3.7	Phase III task register	23
3.8	Phase IV task register	24
3.9	Four-phase roadmap	25
3.10	SDLC stage mapping	26
3.11	Technology Stack: Additional Alternative Evaluations	26
3.12	Design reference versus demonstration path	27
3.13	Economic feasibility — zero-cost prototype	27
3.14	Gas cost and break-even assumptions	28
3.15	Illustrative three-year NPV and ROI (pilot scale)	28
4.1	Capability priority summary	29
4.2	Process model comparison	30
4.3	Four-layer architecture technical detail	31
4.4	Smart contract inventory	36
4.5	Blockchain platform selection criteria and justification	37
4.6	Blockchain Platform Selection: Comparison of EVM-Compatible Networks	37
4.7	Blockchain platform selection (continued)	38
4.8	Blockchain Platform Selection (Continued): Operational and Deployment Factors	38
4.9	Chainlink upgrade paths versus MVT substitutes	39
4.10	Core database entities	41
4.11	Deferred and extension entities	42
4.12	Audit and logging table taxonomy	43
4.13	EER constructs applied	44
4.14	EER Constructs Applied: Specialization, Hierarchy, and Constraints	44
4.15	Relational integrity constraints	46
4.16	Relational Integrity Constraints: Referential and Domain Rules	46
4.17	Data partitioning between on-chain and off-chain storage	47
4.18	On-Chain vs. Off-Chain Data Partitioning: State, Analytics, and Privacy Boundaries	47
4.19	Operational user taxonomy	50
4.20	Formal actor taxonomy	51
4.21	Actor permission matrix (tabular summary)	51
4.22	Credit Passport tier schedule	54

4.23	FK anchors for extended and multi-entity entities	58
4.24	FK Anchors for Extended and Multi-Entity Entities	58
4.25	Governance framework summary	74
4.26	Five-layer defense-in-depth model	77
4.27	Threat model (summary)	77
4.28	Design decisions and alternatives considered	78
4.29	Design Decisions: Evaluated Alternatives and Selection Rationale	78
4.30	Design alternative evaluation criteria	79
4.31	Selected technology justifications	80
4.32	Alternative technology justifications	81
4.33	MCP tool server (MVT subset)	84
4.34	Technology stack summary	89
4.35	Local LLM Assistant Integration: Components, Configuration, and Proto- type Stance	89
5.1	Implementation evidence	91
5.2	Design selection vs. CO6 factors	92
5.3	Requirements traceability (excerpt)	92
5.4	BCCC fraud-detection benchmark	93
5.5	Selected vs. rejected design choices	94
5.6	Research limitations (summary)	94
A.1	Indexing strategy	99
A.2	Indexing Strategy: B-tree Indexes for Time-Window and High-Frequency Queries	99
A.3	Representative functional dependencies	100
A.4	Functional Dependencies in the Core Lending and Identity Schema	100
B.1	MCP toolset scoping	104
C.1	Smart contract capabilities summary	106
C.2	Internal industry research corpus	108

List of Figures

1.1	Crypto World Bank system overview	2
1.2	Cross-tier lending flow directions	6
1.3	Multi-entity co-funding mechanisms	7
3.1	Planned development and verification toolchain	25
4.1	Four-layer application architecture	32
4.2	Component diagram with architecture-pattern labels	33
4.3	Level-0 data-flow diagram (core lending)	33
4.4	Level-1 data-flow diagram (core lending)	34
4.5	System component architecture	35
4.6	Layered blockchain and application stack	37
4.7	Oracle architecture	39
4.8	Core system entity graph	40
4.9	Extended ERD: banking products	40
4.10	On-chain versus off-chain data partitioning	48
4.11	Financial data authority and lifecycle	49
4.12	Actor permission matrix (primary actions)	52
4.13	Kinked utilization-based borrowing rate	53
4.14	LiquidationEngine tier models and lifecycle	53
4.15	SavingsVault and FixedDeposit closed loop	54
4.16	On-chain Credit Passport SBT lifecycle	54
4.17	Multi-entity and cross-tier capital operations	55
4.18	Use-case diagram (nine-actor taxonomy)	60
4.19	USDC loan lifecycle activity flow	61
4.20	Retail onboarding and identity activity flow	62
4.21	Client banking session activity flow	63
4.22	Loan approval sequence	65
4.23	Installment and income verification	66
4.24	Hierarchical banking and data sync	67
4.25	Chat and AI agent sequences	68
4.26	UML class diagram (core lending domain)	70
4.27	Four-tier hierarchical capital flow	72
4.28	Solidarity group lending lifecycle	73
4.29	Governance dual-path	74
4.30	SAR and AML compliance workflow	75
4.31	Five-layer defense-in-depth security architecture	76
4.32	ML oracle commit-reveal gate	82
4.33	AI banking agent request path	87
4.34	AI agent data flow	88
5.1	ML evaluation and SHAP	93
A.1	Full relational ERD (51 entities)	98

C.1	Retail loan application prototype (Figma)	109
C.2	Authority Brief approver dashboard prototype (Figma)	109
C.3	Client onboarding funnel prototype (Figma)	110
C.4	Reserve transparency dashboard prototype (Figma)	110
C.5	Conversational banking agent prototype (Figma)	111
C.6	Group solidarity lending prototype (Figma)	111
C.7	Compliance and AML alert review prototype (Figma)	112
C.8	Mobile navigation and responsive layout prototype (Figma)	112

List of Formulas

Utilization (U): $U = \frac{L}{L+B}$

Kinked rate ($U \leq U^*$): $r_b(U) = r_0 + \frac{U}{U^*} \cdot r_1$

Kinked rate ($U > U^*$): $r_b(U) = r_0 + r_1 + \frac{U-U^*}{1-U^*} \cdot r_2$

Collateral Ratio: $CR = \frac{C}{D}$

Loan-to-Value: $LTV = \frac{\text{Loan Amount}}{\text{Collateral Value}}$

APR (simple): $APR = r_{\text{period}} \times N$

Compound interest: $A = P \left(1 + \frac{r}{n}\right)^{nt}$

EMI: $EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$

Health Factor: $HF = \frac{C \times LT}{D}$

Group Health Factor: $HF_{\text{group}} = \frac{\text{total_group_collateral} \times LT}{\text{total_group_outstanding_debt}}$

Institutional Health Factor: $HF_{\text{inst}} = \frac{\text{on_chain_reserve} - \text{min_reserve}}{\text{outstanding_borrowing}}$

Reserve Ratio: $RR = \frac{\text{Reserves}}{\text{Total Deposits}}$

Platform Solvency Ratio: $PSR = \frac{\text{TotalReserveBalance}}{\text{TotalOutstandingLoans}}$

Credit Velocity: $CV = \frac{\text{Capital Recycled per Period}}{\text{Total Reserve}}$

Interbank rate cap: $r_{\text{IB}}(U) \leq r_{\text{downward}}(U) - \delta$

Upward deposit rate cap: $r_{\text{up}} < r_{\text{down}} - \delta$

FX conversion: $\text{Amount}_B = \text{Amount}_A \times \frac{P_A}{P_B}$

Treasury swap: $\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread})$

Oracle commit-reveal: $h = \text{keccak256}(s \parallel \text{nonce})$

Group loan share: $\text{Share}_i = \frac{\text{LoanTotal}}{N_{\text{members}}}$

On-chain credit score: $CS = \sum_i w_i \cdot \text{feature}_i$

Isolation Forest score: $s(x, n) = 2^{-E(h(x))/c(n)}$

Random Forest fraud probability: $P(\text{fraud} \mid x) = \frac{1}{T} \sum_{t=1}^T f_t(x)$

Net interest split: $\text{NetInterest} = \text{InterestCollected} - \text{DepositorYield} - \text{InsuranceAllocation}$

SHAP (Shapley value): $\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$

List of Abbreviations

AA: Account Abstraction (ERC-4337)

ALM: Asset-Liability Management

AI: Artificial Intelligence

AI/ML: Artificial Intelligence / Machine Learning

AML: Anti-Money Laundering

APR: Annual Percentage Rate

API: Application Programming Interface

BRAC: Bangladesh Rural Advancement Committee

CAR: Capital Adequacy Ratio (Basel III; see PSR for this platform)

CBDC: Central Bank Digital Currency

CCIP: Cross-Chain Interoperability Protocol (Chainlink)

CCS: ACM Computing Classification System

CR: Collateral Ratio

CVL: Certora Verification Language

DeFi: Decentralized Finance

DID: Decentralized Identifier (W3C)

DON: Decentralised Oracle Network (Chainlink)

EIP: Ethereum Improvement Proposal

EIP-7702: Ethereum Improvement Proposal 7702 — Session Key standard enabling scoped, time-bound wallet authorisations for an autonomous agent

EMI: Equated Monthly Installment

ERC: Ethereum Request for Comments (token / interface standard)

EVM: Ethereum Virtual Machine

FL: Federated Learning

FATF: Financial Action Task Force

FX: Foreign Exchange

GNN: Graph Neural Network

HF: Health Factor

KYC: Know Your Customer

LLM: Large Language Model

L2: Layer 2

LTV: Loan-to-Value Ratio

MFI: Microfinance Institution

MiCA: Markets in Crypto-Assets Regulation (EU)

ML: Machine Learning

MCP: Model Context Protocol — tool-calling interface used by the AI agent to interact with CWB banking operations

NIM: Net Interest Margin

PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses

PoS: Proof of Stake

PoR: Proof of Reserve (Chainlink on-chain reserve verification)

PSR: Platform Solvency Ratio

QRC: QR code

RBAC: Role-Based Access Control

RLS: Row-Level Security (PostgreSQL policy feature enforcing append-only access on audit tables)

RR: Reserve Ratio

SAR: Suspicious Activity Report

SBT: Soulbound Token (non-transferable ERC standard)

SHAP: SHapley Additive exPlanations

SOFR: Secured Overnight Financing Rate

SSE: Server-Sent Events

SSI: Self-Sovereign Identity

SoK: Systematization of Knowledge

SWC: Smart Contract Weakness Classification

TVL: Total Value Locked

U: Utilization

UUPS: Universal Upgradeable Proxy Standard (ERC-1822)

VC: Verifiable Credential (W3C)

XAI: Explainable Artificial Intelligence

ZKP: Zero-Knowledge Proof

zkAML: Zero-Knowledge Anti-Money-Laundering proof

zkEVM: Zero-Knowledge Ethereum Virtual Machine — a ZK rollup that inherits Ethereum L1 security via cryptographic validity proofs

zk-SNARK: Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

Chapter 1

Introduction

The *Crypto World Bank* is a research-based project for exploring how blockchain technology can be shaped to fit into institutional financial systems with all the added benefits that come with a decentralized financing system. Based on our findings that had similar motivation to our study, existing DeFi protocols show isolated functionalities, such as Aave for lending, Uniswap for exchange, and MakerDAO for stablecoins. This project specifies a tier-based, hierarchically governed platform that includes a mixture of institutional financing with DeFi, accommodating deposit mobilization, credit allocation, interbank liquidity, installment repayment, FX facilitation, solidarity group lending, reserve enforcement, syndicated co-lending, and risk-based governance covering all four tiers that systematically mirrors real-world institutional financing. At the top, Tier 1 represents the World Bank reserve; the core banking hierarchy is specified in Solidity and being improved until final deployment. Another significant feature—the multi-entity contract suite—is fully specified in Chapter 4, which is scheduled for practical development in Phases II–III.

The system is designed for three categories of participant: a programmable World Bank reserve as Tier 1, national and local development banks serving in Tiers 2–3, and retail clients in Tier 4. Retail clients get access to savings, group loans, and installment lending features. This hierarchical structure in this project was motivated by unbanked and MSME financing gaps [9, 13]. The project is not built with any motivation to deploy only in regions with access to advanced technologies; rather, proper distribution of access to finance and fairness is the leading cause of this attempt to find suitable ground to adapt DeFi among the general population. The whole project is open source and available in our GitHub repository. The added benefits of blockchain: it provides state visibility, tamper-evident audit trails, and programmable rule enforcement among institutions having competition, as demonstrated by the renowned World Bank FundsChain programme [25] and JPMorgan Kinexys [26]. From the study of institutions, open ledger infrastructure and institutional hierarchy are not mutually exclusive.

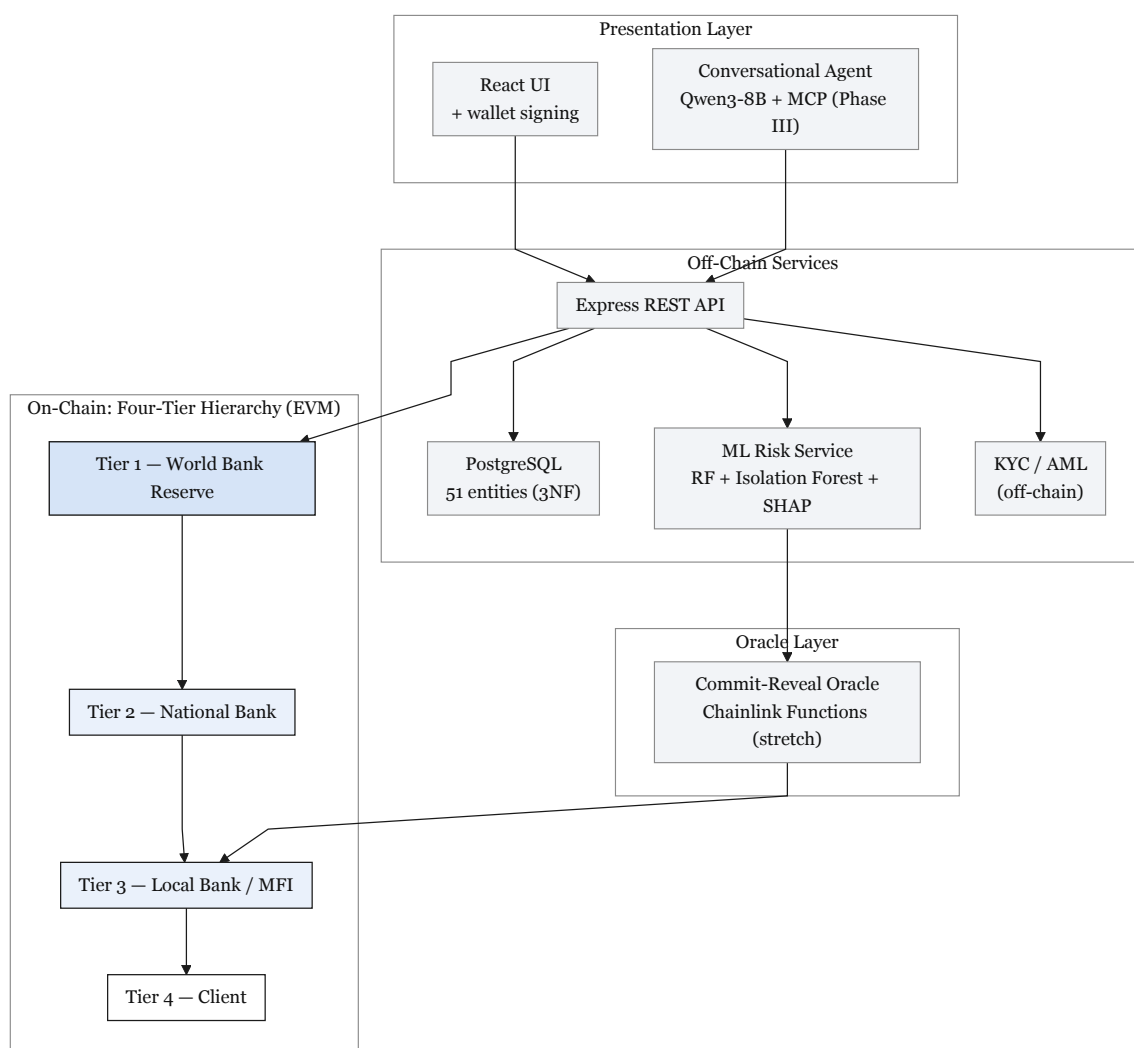


Figure 1.1: Crypto World Bank system overview

1.1 Background

Local lenders, or our target users, for this aspect of lending through layered institutions—supranational bodies that is provided by development finance channels have structural costs. Idle capital is usually locked by correspondent banking in pre-funded nostro/vostro accounts [15], cross-border transactions that take around two to five days cost roughly \$45 per transaction [101], charges for remittance corridors average 6.49%, which is above the UN SDG target of 3% [17]. Due to unequal access to traditional banking systems, 1.4 billion adults remain unbanked [9] and MSMEs face a \$4.5 trillion annual financing gap [13].

In decentralized financing, lending, collateral, and interest can be automated by enabling rules on-chain transparently. However, institutions like Aave v3 that holds \$26.3 billion TVL [18] and the broader DeFi lending market exceeds \$55 billion [8], use flat pool design that ignore institutional hierarchy, inter-tier exchange, and accessing clients via local bank lend path which is more generalized for normal people with less knowledge about DeFi.

Contribution of smart-contract and blockchain. Smart-contracts can develop lending rules as immutable, atomic, with auditable on-chain logic. This technology replaces the

bilateral ledger reconciliation with a system of shared record, when a local bank initiates a loan, the World Bank’s reserve ratio updates on the same transaction status [25]. DeFi extends this feature to financial services without licensed custodians. Here in *The Crypto World Bank* adds three dimensional feature that is missing from the existing institutional protocols: institutional hierarchy (four-tier capital flows), multi-entity operations (solidarity groups, syndicated loans, tranching pools), and a connection to the evolving development finance framework identified as absent in the literature [1].

Optional conversational interface. Every user interacts with this system primarily through the React UI and wallet credential signing. For development purposes and demonstration, a locally hosted agent (Qwen3-8B + MCP, 17 banking tools) provides an alternate path to get the essential tasks done, including all the customer services that require system interaction information guidance and assistance in acquiring a loan or exchange finances with the system. In the final build it is planned to be replaced with secured API-based services.

Technical foundations. Consensus. The chain is an append-only ledger: nodes agree on history, and no one can rewrite past blocks without controlling most of the network [50].

EVM execution [104]. Loan logic runs as deterministic bytecode on Ethereum-compatible chains. Storage writes are the main cost driver (Section 3.7.2).

State machines. Each loan moves through clear stages—PENDING → APPROVED → ACTIVE → REPAYING → COMPLETED/DEFAULTED—with only authorised roles allowed to advance it.

Oracles [45, 96]. Contracts cannot read off-chain data on their own. ML risk scores reach the chain via a commit-reveal oracle (Section 4.2.3); Chainlink Feeds, Automation, and Proof of Reserve handle prices, due-date checks, and reserve attestation. Live ML wiring starts after pre-thesis 2.

Polygon zkEVM. We deploy on Cardona testnet (Section 4.2.2) for low fees and L1-backed validity proofs.

1.2 Rationale and Motivation

Real development finance depends on layered institutions, yet settlement friction, opaque reserves, and correspondent-banking idle capital leave both intermediaries and unbanked clients underserved [17, 90]. Flat DeFi protocols automate lending at scale but ignore institutional hierarchy, cross-tier flows, and the Local-Bank-to-client path that microfinance and development banks use in practice [1]. This study is motivated by the opportunity to encode that hierarchy on a programmable ledger—with enforced reserves, role-based governance, and auditable capital movement—while supporting (not replacing) human credit judgment through lightweight analytics and an optional conversational interface. The thesis specifies a reusable architecture; it does not claim a particular national deployment target.

1.3 Problem Statement

Three primary forces of motivation for this study: development-finance has restricting transparency and settlement; the absence of multi-tiered banking functions in DeFi (de-

posits, solidarity and lending); the keen interest in combining blockchain auditability with ML computational analytics. There are six structural inefficiencies noticeable in development finance routes for individual borrowers from national and local banks:

1. **Lack of transparency:** there is very little chance of the lower tier users to verify capital management, reserve levels or lending decisions are made by the higher tiers, very less transparency on how money flows through the different levels of systems. Only the top tiers get the information and records of reserves and audited at most quarterly.
2. **Sluggish settlements and idle capital:** cross-border transactions are quite expensive, average \$45 per transaction [101], and time consuming taking two to five days [23].
3. **Inconsistent risk evaluation:** fraud and credit assessment are completely relied on human judgement, borrowers are generally re-evaluated at each tier or institutions based on information available at that tier, no unified credit history.
4. **Access and trust barriers:** the cost of inter-institutional trust is quite expensive for legal and audit process. It creates congestions in developing countries where investors earn returns roughly 3% below compared to developed markets [103].
5. **No programmable savings:** depositors in developing economies have no real-time access to visibility into how savings are processed and deployed.
6. **No on-chain group credit:** without individual collateral, borrowers cannot access programmable on-chain mutual group lending with shared responsibility. Real-world organizations like BRAC exist but does not function like DeFi.

DeFi enabled automatic lending transparency at scale such as Aave v3 with \$26.3 billion TVL, \$17.7 billion borrows [18], Compound v3 (\$1.4 billion) [19], MakerDAO with \$10.5 billion stablecoin issuance [20]. Despite large market coverage they exhibit four limitations for institutional development finance:

- **Flat lending models.** Aave/Compound uses one protocol for many asset pools (USDC, ETH, etc.). There is no cross-tier allocation, no institutional hierarchy, no difference in role-based governance. Here our CWB models institutional relationships that go deep into what kind of services to provide to what kind of roles/institutions to increase liquidity in overall lending and exchange situations.
- **No AI risk analytics.** DeFi relies on collateral ratio, utilization curves. There is no behavioral fraud detection or explainable credit support system.
- **No tiered governance.** DeFi protocol treats tokens as a measurement of voting, not based on what is the role of the client or user or institution in the bigger to smaller picture as it lacks relation to a deeper level like institutions.
- **Single-level institutional DeFi.** Maple & Goldfinch [21] with \$2.6–3.8 billion TVL and \$680 million originations in 18+ countries respectively serve institutions, but none models a full multi-tier banking system with interbank flows, lack hierarchical capital flows.

1.4 Objectives

The objectives of this project are:

1. Design and specify all features and constraints of a four-tier lending architecture on an EVM-compatible blockchain and complete a full testnet deployment (World Bank

→ National Bank → Local Bank → Client). The whole development cycle is planned for Phases I–IV.

2. Design not only pool lending, but specify an extended banking product suite on the hierarchical system (deposits, savings, and solidarity group lending).
3. Investigate a lightweight off-chain analytics layer using ML, training and testing the models in practical testnet for fraud detection, anomaly detection, explainable review, and an optional MCP-based conversational interactive agent for ease of use.
4. Specify programmable on-chain lending with borrowing limits based on financial behaviour analysis, installments, and role-based access control and information via smart contracts.
5. Plan and execute testnet deployment using Polygon zkEVM Cardona, Ethereum Sepolia, and evaluate hierarchical controls in the final thesis phase.
6. Document governance for future improvements, security analysis and feature improvements, role-based regulatory considerations, and a controlled rollout toward sandbox piloting.

1.5 Proposed Solution

The Crypto World Bank will implement a four-tier on-chain lending model: Tier 1 World Bank reserve custodian → Tier 2 National Banks → Tier 3 Local Banks processing retail loans → Tier 4 clients building on-chain credit history (Figure 1.1), with six standard banking functions enforced on-chain—deposit mobilization, credit allocation, payment and settlement, risk intermediation, liquidity management, and ancillary services—rather than discretionary audit alone. Banks also borrow from peers, park surplus with parent tiers, and syndicate large loans; the platform replicates four-tier downward flow, same-tier pools, upward deposits, and multi-bank co-funding on-chain (Figure 1.2).

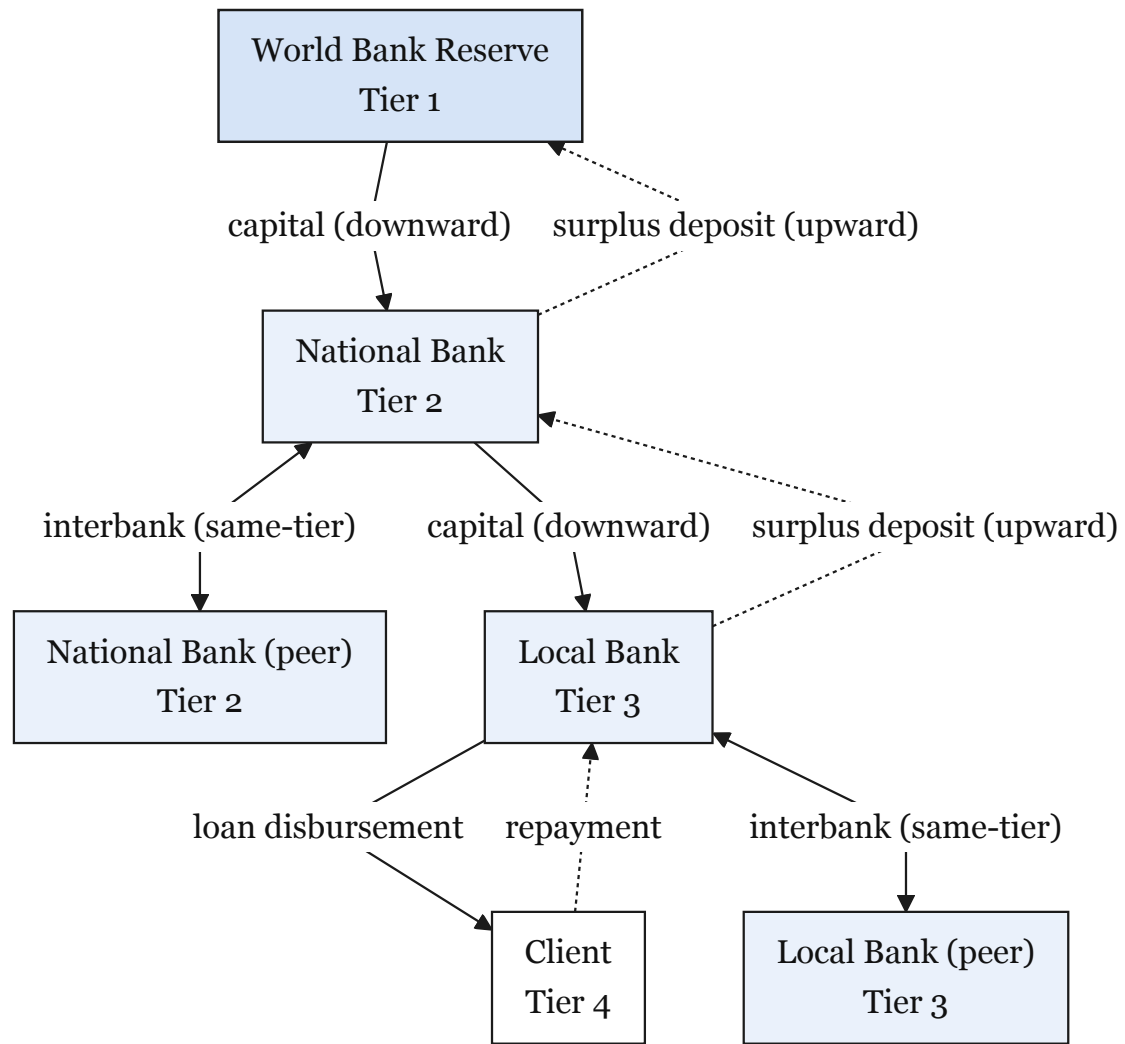


Figure 1.2: Cross-tier lending flow directions

Can a group of entities lend or fund together? Syndicated loans, solidarity groups, tranching pools, and upward surplus deposits (Figure 1.3; Section 4.2.9) coexist with same-tier peer liquidity pools (Section 4.2.9); retail clients enter through Local Banks via KYC, Credit Passport, and borrowing limits, with syndication reserved for large loans.

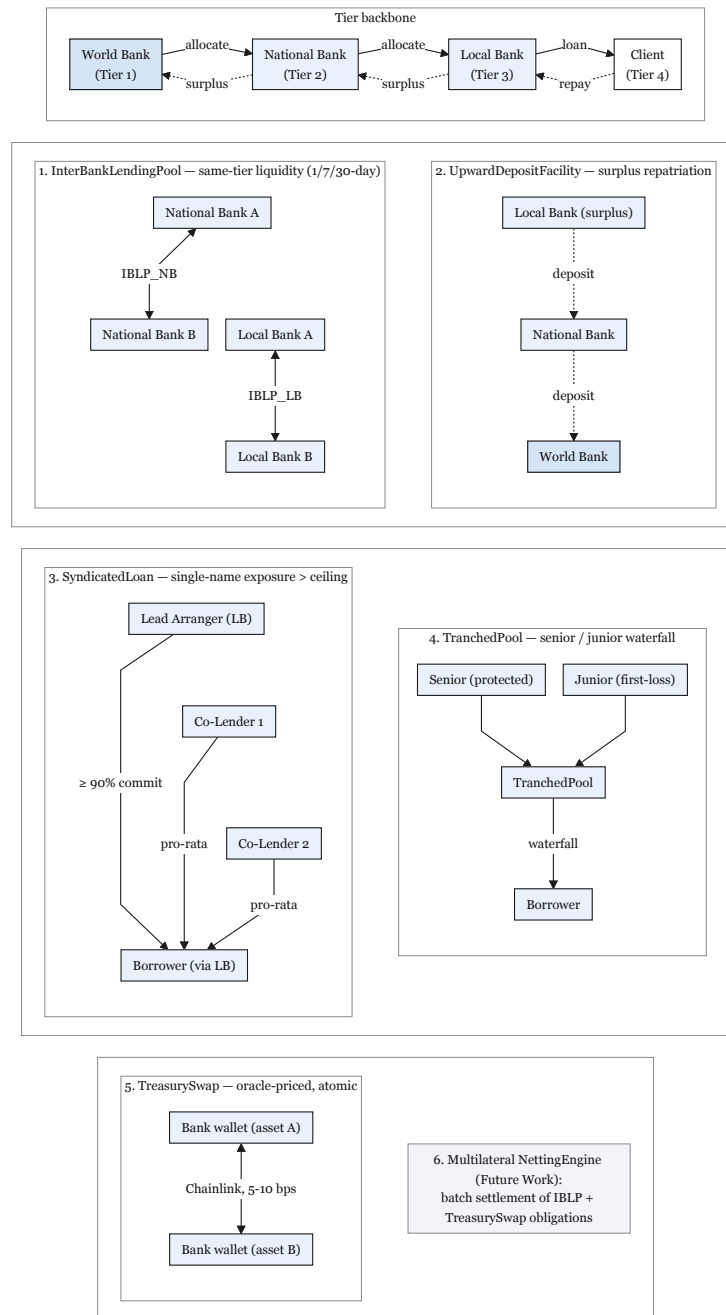


Figure 1.3: Multi-entity co-funding mechanisms: syndicated loan, solidarity group, tranchedException, and upward deposit

1.6 Methodology in Brief

There are four Agile phases to this methodology for this project. This report is designed with the architecture specification. The development phase will start from Phase II after pre-thesis 1. In this report there is architecture, contracts, database, and ML pipeline. The Phase II development phase will demonstrate the loan cycle. Phase III will start the implementation of ML scoring and the Chainlink oracle (Section 4.2.3). Phase IV will show the evaluation, metrics, and the overall feasibility of the project in the real world. Stack: Solidity, React, Express/FastAPI, scikit-learn on Polygon zkEVM Cardona. There is also an AI chat agent that will be built as an optional feature that will run alongside the project to

help the clients navigate better with the system.

1.7 Scopes and Challenges

In scope now. In our development phases, we have calculated that we can implement a limited amount of our full vision: the four-tier lending architecture, loan lifecycle workflow, borrowing limits for the clients and the banks, the design of Oracle fraud analytics, and the training and the live demonstration of the AI/ML workflow. It can be shown to a limited amount on the prototype builds only.

Out of scope for now. Mainnet deployment, live retail payments in the real world, production KYC/AML at the banking level and client interaction, stress testing the whole project, and providing an MVT checklist before the product is ready is not quite possible yet.

Main risks. There are a couple of risks that we have noticed for our project, starting with the fraud labels may not be completely accurate based on the limited amount of data we have for our system architecture. We need more synthetic data that will be collected over time. If regulations have changed—mitigated by testnet-only builds—then we might have to change the architectural elements. The gas toll on micro-loans may not be sustainable. The ML layer with SHAP may keep borrowers in the loop in the system since it is not 100% accurate and accuracy of the system depends on the training process. Rest of the economics and trade-offs are discussed in Section 3.7.

Economic sustainability. On Cardona, 10–15 txs on a typical loan cost under \$0.15—small next to interest earned. Mainnet micro-loans would need tighter gas work.

Institutional capture. Another huge issue that needs to be addressed is that Tier 1 and the National Banks, which have huge reserves and influence over the rates, reserves, and who gets credit, might influence the overall system and cost-to-performance ratio. It might temper the trust and relationship with the retail customers and smaller banks.

Stablecoin-first lending. Retail clients are not to be carrying high-priced positions like ETH, since the prices of Ethereum might fluctuate. MiCA and the U.S. GENIUS Act support the idea that stablecoins should be more generalized for authorized cryptocurrency. There are many settlement projects like *mBridge* and *Agora* [61, 62], moving the value of cryptocurrencies across borders, but it is not a replacement for our lending system. The architectural system for stablecoin pools and L2 deployment (Section 5.2.2) gives the liquidity move fluidly from the top tier to the bottom, and the surplus to go up from the bottom.

1.8 Thesis Organization

- **Ch. 1 — Introduction:** problem, objectives, approach (CO1).
- **Ch. 2 — Literature:** PRISMA review and protocol comparison (CO9).
- **Ch. 3 — Specifications and impact:** requirements, societal/environmental impact, ethics, project management, economics (CO2–4, CO10–12).
- **Ch. 4 — Design alternatives:** design process, preliminary architecture, technology choices (CO5).
- **Ch. 5 — Evaluation and analysis:** performance evaluation, design analysis, discussion (CO6–7).

- **Ch. 6 – Conclusion:** summary, contributions, and future work; appendices hold contracts and deployment notes.

Chapter 2

Literature Review

2.1 Preliminaries

Important terms—XAI, PoS, CBDC, solidarity/group lending, DeFi, smart contracts, over-collateralization, ALM, flash loans, ZKP, and Health Factor—have been introduced in Glossary appendix. This section has been expanded in appendix D that will be presented in the final versions of the complete project file.

2.1.1 Review Methodology (PRISMA-Style Frame)

We did our research to find the appropriate papers and highly qualified papers that goes without project from ACM, IEEE, Elsevier, MDPI, arXiv, and BIS/World Bank (2017–2026) with PRISMA-style screening [52]. Primary topics for this search: DeFi lending, fraud ML, tiered finance, and smart-contract security.

2.2 Review of Existing Research

We explain our literature review, and our findings from the papers and the work of the authors, in the subsections shown below.

2.2.1 Decentralized Lending and DeFi Protocol Design

For a big picture, Werner et al. [1] show that DeFi lenders use flat pools with no bank-style tiers; we have found a gap that can be filled by introducing our project. According to Bastankhah et al. [2], adaptive borrow rates work better than Aave-style curves for financial flows, which we introduced in our kinked-rate module. Dao et al. [39] show how on-chain limits can get off-chain credit scores that in general are applied at our Local Bank level.

2.2.2 Machine Learning, Explainable AI, and Extended AI Security

For fraud detection, Palaiokrassas et al. [3] engineer behavioral features that do a really good job in DeFi transaction fraud detection, not just hashes. We plan to use Random Forest in Phase III that will further strengthen the fraud detection level with the mixture of hash functions that is built into the DeFi protocol. Moreover, for the situations when the fraud detection level is scarce, we will use Liu et al. [6] Isolation Forest. We also studied a very in-depth comparison of SHAP versus LIME by Adom et al. [4], which shows a significant advantage by using SHAP for loan detection; approvers in bank and authority level will see SHAP screening results before signing.

2.2.3 Hierarchical Distribution, Cross-Tier Flows, and Group Lending

Tan [5] demonstrated how CBDC flows in bank-to-bank and bank-to-client financial transactions. We extended this idea in our project by showing tier-based reserve checks. For layered reserve contracts, *Project Pine* [107] has shown a prototype that is similar to our `InterBankLendingPool` and the syndicated-style loan flow. We also integrated the style of

microcredit admin cost with smart contracts based on the study of Asaduzzaman et al. [36]. The structure of GroupLendingPool function is the on-chain version, without claiming that we are trying to replace BRAC-style social microfinancing enforcement.

2.2.4 Smart Contract Security and Layer 2 Scalability

To improve the security system of the project, we are following Atzei et al. [7], which lists the main Ethereum contract attacks. Since the tier-based system is very complex, many transactions on Cardona might be triggered. Gas prices can be cut to 30–50% by transaction batching [102], with further DeFi gas optimisation [37].

2.2.5 Monetary Policy, Tokenization, and Institutional Landscape

Very large institutions like World Bank FundsChain [25] and similar institutions show that development finance is already moving on-chain for auditability; we are further improving this by encoding rules, not just managing disbursement logs.

2.2.6 Graph ML, Federated Learning, Regulation, and Identity Primitives

We noticed the usability and rise of stablecoin and went further deep into it and try to explore how stablecoin pools for retail clients can be strengthened by using MiCA and the U.S. GENIUS Act [58, 60]. For our project, soulbound tokens will be used that fit into our Credit Passport idea, which was extended by the idea of Buterin et al. [94]. This introduces a secured identity on-chain, a portable reputation, without the fear of exposing personal data.

2.2.7 Oracle-Mediated ML and Optional LLM Interfaces

Lastly, for an optional feature to our project, future studies can be made and features can be added to the system. Caldarelli [96] describes how ML scores still have some issues that go beyond price oracles; we focus on this part and extend our commit-reveal or Chainlink path targets. We collected some reputable databases, and more to be added such as the BCCC fraud dataset [99] that goes with our system. This section will be continued and extended after pre-thesis 2.

Table 2.1: Literature review summary (Part 1 of 2)

Author & Focus	Methodology	What they found & how we use it
Palaiokrassas et al. (2024) [3] · DeFi fraud	XGBoost, NN on 54M+ txs	We plan to use behavioural features in Phase II for Random Forest, as it lifted F1 to 0.76–0.85
Adom et al. (2025) [4] · XAI lending	LIME / SHAP comparison	Approvers get a SHAP brief before signing, as it gives a clearer result than LIME on loan data
Tan (2023) [5] · CBDC hierarchy	Developing-nation model	CBDC backs the four-tier layout and works as a bank-to-user tier
Liu et al. (2008) [6] · Anomaly detection	Isolation Forest	Flags odd wallets without many labels; backup when fraud data is thin
Atzei et al. (2017) [7] · SC security	SoK of Ethereum attacks	Lists re-entrancy and access-control risks; guides OpenZeppelin guards
Werner et al. (2022) [1] · DeFi SoK	Survey of 12+ protocol types	Main gap CWB targets is DeFi lending having no bank hierarchy
Bastankhah et al. (2024) [2] · Adaptive lending	Dual fast/slow control	Dynamic rates support our kinked-rate modules as it beats static curves
Hartmann & Hasan (2021) [33] · P2P lending	Smart-contract origination	Gas is cut in on-chain P2P, which is a minor input for loan-facing clients
Hasan et al. (2022) [34] · Federated fraud	FL on encrypted features	Lack of raw-data sharing features by banks leaves future cross-bank options
Wang et al. (2020) [105] · SC vulnerabilities	ML on bytecode	Common bytecode bugs spotting by ML helps provide extra security over manual audit
Liao et al. (2019) [106] · Audit automation	Classification + fuzzing	Workflow idea for Phase V is inspired from ML prioritizing fuzzing speed

Table 2.2: Literature review summary (Part 2 of 2)

Author & Focus	Methodology	What they found & how we use it
BIS et al. (2020) [35] · CBDC design	Retail distribution analysis	Standard CBDC two-tier is extended to four-tier with on-chain reserves
Kiff et al. (2020) [80] · CBDC models	IMF architecture survey	Matching National/Local roles; retail CBDC flow implemented through supervised banks
Bindseil (2020) [79] · CBDC rates	ECB tiered remuneration	Upward deposit spread idea inspired from tiered rates that protect bank intermediation
Dong et al. (2023) [78] · Supply-chain finance	Three-tier game model	Capital can move down a chain, parallel to World Bank → client allocation
Project Pine (2025) [107] · Hierarchical SCs	NY Fed / BIS prototype	IBLP and syndicated loans used on-chain layered reserve contracts prototype
CPMI/FSB [15, 23] · Cross-border settlement	Nostro/vostro cost analysis	Tiered on-chain flow is better than legacy settlement as it is slow and costly
World Bank (2025) [25] · Dev. finance blockchain	13-project pilot	Programmable lending rules added with dev-banks system
Chen et al. (2024) [31] · Multi-chain lending	Cross-chain design	Scaling by splitting loads throughout chain is later option, not for Phase I
Yang & Cui (2023) [32] · Protocol evaluation	Quantitative risk metrics	Gives utilisation and liquidation benchmarks for health checks tier
Asaduzzaman et al. (2020) [36] · Smart micro-credit	Smart-contract MFI	Model for GroupLendingPool is smart contracts, which cuts the admin costs in Bangladesh
Wang et al. (2021) [102] · Tx batching	iBatch middleware	Per call saved 15–59% in gas price by which we deploy on Cardona L2
Kim & Kim (2024) [37] · Gas optimisation	DeFi parameter tuning	~18% gas cut on Ethereum pools helps our L2 deployment
Yuan (2024) [38] · SHAP credit default	RF, GBM + SHAP	Deploying on Cardona L2 as batching saved 30–50% gas; SHAP is also used for our decline path

Feature	Aave	Comp.	Maker	Maple	Gfinch	CWB
Institutional hierarchy	✗	✗	✗	✗	✗	●
Cross-tier capital flow	✗	✗	✗	✗	✗	⌚
Same-tier interbank lending	✗	✗	✗	✗	✗	⌚
Syndicated / co-lending	✗	✗	✗	Part.	Part.	⌚
Upward capital repatriation	✗	✗	✗	✗	✗	⌚
Solidarity group lending	✗	✗	✗	✗	Part.	●
AI/ML fraud detection	✗	✗	✗	Man.	Man.	●
SHAP explainability	✗	✗	✗	✗	✗	●
ZKP KYC compliance	✗	✗	✗	✗	✗	●
Kinked interest rate curve	✓	✓	Gov.	✓	✗	⌚
Role-based access control	Part.	Part.	Gov.	✓	Part.	●
Institutional / L2 focus	✗	✗	✗	✓	Part.	●
TVL / Status (2026)	\$26B	\$1.4B	\$10B	\$2.6B	\$680M	Planning

Table 2.3: Protocol comparison on eleven architectural dimensions

2.3 Summary of Key Findings

Four points stand out after reviewing and Table 2.3:

- **Flat DeFi vs. tiers.** Large pool lenders [1, 8] have no hierarchical bank currently, and it opens a door to explore how a hierarchical system would behave with DeFi. CBDC and Project Pine work [5, 107] prove that nobody combines a four reserve-enforced tier with group lending and cross-tier financial flows and an ML oracle in a singular architecture.
- **ML and security.** ML and security layers combined with behavioural fraud features and SHAP [3, 4, 6] further extend the usability, feasibility, and adaptability of our project. Based on these literature reviews, we further improve our Phase III pipeline; contract attack catalogues [7] and L2 gas batching [102, 37] justify OpenZeppelin guards to be added to our project.
- **Groups and institutions.** We explored smart-contract microcredit in Bangladesh [36] and World Bank on-chain pilots [25] that gave us the idea and improved the implementation of GroupLendingPool function and institutional lending system.
- **Regulation.** For regulation improvement and securities, MiCA and GENIUS [58, 60] are accepted by stablecoin-first pools. Soulbound credit tokens [94] fit the Credit Passport without putting KYC on-chain.

Chapter 3

Specifications, Impact, Ethics, and Project Management

3.1 Final Specifications and Requirements

The functional and non-functional requirements are shown in Table 3.1 and Table 3.2; based on them, the software engineering with in-depth architecture and system analysis is specified in Chapter 4. The implementation plan has been specified in Section 3.5.1. The MVT checklist has been specified in Table 3.3, and the complete features list is in Table 3.4.

Table 3.1: Functional requirements (representative subset). Full feature inventory in Table 3.4.

ID	Requirement	Description
FR-01	Four-tier hierarchy	World, National, and Local Bank contracts with role-gated downward capital allocation and client onboarding at Local Bank tier.
FR-02	Loan lifecycle	Request → ML score commit-reveal → approver decision → disbursement → installment schedule → repayment.
FR-03	Reserve enforcement	Each tier retains a minimum reserve ratio before allocating capital downward; on-chain revert on breach.
FR-04	Borrowing limits	Rolling six-month and one-year limits enforced on-chain; off-chain projection in BORROWING_LIMIT.
FR-05	Risk scoring	Off-chain RF + IF + stacking; SHAP explainability; immutable LOAN_RISK_ASSESSMENT audit row per inference.
FR-06	Group lending	GroupLendingPool with mutual liability, consent, and per-member share disbursement (Section 4.2.11).
FR-07	Event synchronisation	Blockchain event listener projects on-chain state into PostgreSQL for dashboards and agent context.
FR-08	Dual client interface	React dashboard and MCP conversational agent call the same Express.js API and contracts.

Table 3.2: Non-functional requirements.

ID	Category	Constraint / target
NFR-01	Performance	ML inference ≤ 50 ms on laptop hardware; API p95 ≤ 500 ms for read endpoints at MVT scale.
NFR-02	Security	RBAC on contracts and API; ReentrancyGuard; human approver gate before disbursement; agent write-path confirmation.
NFR-03	Scalability	Modular contract deployment; PostgreSQL indexing (Appendix A); L2/testnet gas target sub-cent per retail lifecycle step.
NFR-04	Maintainability	TypeScript/React/Express monorepo at repository root; OpenZeppelin primitives; documented phase gates G1–G4.
NFR-05	Usability	Mobile-first React UI; five-stage onboarding funnel; Authority Brief for approvers (Section 4.2.10).
NFR-06	Compliance (design)	Tiered KYC ladder; GDPR data minimisation (hash-only on-chain); EU AI Act Art. 86 explainability artifacts.
NFR-07	Reliability	Pause mechanism on contracts; zero-cost demo substitutes with documented upgrade path (Section 3.6).
NFR-08	Interoperability	EVM-standard contracts; WalletConnect; ERC-20/ERC-4626 patterns; MCP tool schema for agent integration.

3.1.1 Minimum Viable Thesis (MVT) Checklist

Table 3.3: MVT deliverable checklist.

#	Deliverable	Priority	Phase
1	Four-tier bank contracts on testnet	Must	I
2	Loan lifecycle E2E (request, approve, disburse, repay)	Must	II
3	On-chain reserve and borrowing limits	Must	II
4	ML risk scoring (RF + IF + stacking)	Must	III
5	SHAP explainability and Authority Brief UI	Must	III
6	Risk assessment audit row per inference	Must	III
7	Commit–reveal score gate before approval	Must	III
8	BCCC fraud-detection benchmark	Must	III
9	300-client simulation or gas-cost table	Must	IV
10	README with reproduction instructions	Must	IV
11	MCP agent (3–5 tools + confirmation gate)	Must	III
12	LLM evaluation protocol	Should	IV
13	Contract audit reports (Slither + Mythril)	Must	IV
14	Foundry fuzz and invariant tests	Must	IV
15	Safe multisig for World Bank admin	Must	I
16	Tenderly monitoring alerts	Should	IV

3.1.2 List of Features

Table 3.4 lists the complete list of features of the full project. The Must features will be completed during the development phases, Should and Stretch features are extensions of the main Must features, and Specified features are documented but not yet implemented, Future Work features are to show the potential improvements of the project and some of these features might be deliverable by the end of the project as special features.

Table 3.4: Complete feature inventory by category, priority, and implementation phase.

#	Feature	Priority	Phase
<i>Core platform and hierarchy</i>			
1	Four-tier banking hierarchy (World → National → Local → Client)	Must	I
2	Role-based access control (RBAC) on contracts and API	Must	I
3	World Bank Reserve – global capital custody	Must	I
4	National Bank – jurisdiction-level allocation	Must	I
5	Local Bank – retail operations hub	Must	I
6	Downward capital allocation (allocateCapital)	Must	I
7	Hierarchical bank registration (National → Local)	Must	II
8	Reserve-ratio enforcement before downward allocation	Must	II
9	MockUSDC / testnet stablecoin for lending	Must	I
10	2-of-3 Safe multisig for WorldBankAdmin	Must	I
11	Contract pause / unpause (emergency controls)	Must	I
12	Single-chain deployment (Sepolia / Amoy)	Must	I–IV
<i>Retail lending lifecycle</i>			
13	Loan request submission (on-chain + UI)	Must	II
14	Approver review workflow (approve / reject / request info)	Must	II
15	Loan disbursement to client wallet	Must	II
16	Installment schedule generation	Must	II
17	Installment payment processing	Must	II
18	On-chain loan state machine (PENDING_SCORE → ACTIVE)	Must	II
19	Borrowing limits (six-month / one-year rolling caps)	Must	II
20	Collateral-based lending (LTV)	Must	II
21	Credit-based lending (no collateral, SBT-gated)	Should	II
22	Kinked interest rate model (utilization-based)	Should	II
23	Credit tier schedule (Bronze → Diamond)	Should	II
24	Overdue installment checks (cron / Chainlink Automation)	Should	II
25	Loan history and transaction tracking UI	Should	II
26	Income verification document upload and review	Should	II
<i>Deposits and savings products</i>			
27	SavingsVault – variable-yield savings (ERC-4626)	Should	II
28	FixedDeposit – term deposits (30/90/180/365 days)	Specified	II+
29	Current / checking account (atomic transfers)	Specified	II+
30	Deposit-to-lending pool integration	Should	II
31	Interest split (depositor / InsuranceFund / protocol)	Should	II
32	Withdrawal gated by reserve ratio	Must	II
33	InsuranceFund (5% interest capture, default coverage)	Should	II
<i>Group and solidarity lending</i>			
34	Group formation (3–20 members)	Must (C2)	II
35	Multi-signature group consent	Must (C2)	II
36	Per-member disbursement share	Must (C2)	II
37	Shared collateral / mutual liability	Must (C2)	II
38	Over-indebtedness controls (DTI, cooling-off, max two group loans)	Should	II
39	Group credit history on SBT	Should	II
<i>Multi-entity and cross-tier operations</i>			
40	InterBankLendingPool – same-tier peer lending (1/7/30 day)	Should	II
41	UpwardDepositFacility – surplus repatriation to parent tier	Should	II
42	SyndicatedLoan – multi-bank co-funding	Stretch	III

Table 3.4 (continued on next page)

Table 3.4 (continued)

#	Feature	Priority	Phase
43	TranchedPool – senior/junior tranches	Stretch	III
44	TreasurySwap – cross-tier treasury FX	Should	III
45	FXModule – oracle-priced retail FX	Should	III
46	Dual-currency facility (fiat ↔ crypto)	Specified	III
47	Cascading repayment / interest across tiers	Should	II
48	NettingEngine – multilateral settlement	Future Work	–
49	Trade finance / escrow letters of credit	Future Work	–
<i>Credit passport and identity</i>			
50	On-chain Credit Passport (Soulbound Token)	Must	II
51	Credit score, tier, and repayment history on SBT	Must	II
52	Tier-based max loan limits and rate modifiers	Should	II
53	Wallet authentication (MetaMask / WalletConnect)	Must	I
54	EIP-712 typed-data login + JWT session	Must	I–II
55	Five-stage retail onboarding funnel	Should	I–II
56	Tiered KYC (Level 1 / Level 2)	Must	I–II
57	Off-chain KYC document storage (hash-linked)	Must	I–II
58	ERC-4337 account abstraction (gasless onboarding)	Stretch	III
59	Groth16 zkKYC / zkAML circuits	Future Work	–
<i>AI/ML risk and explainability</i>			
60	Random Forest fraud probability scoring	Must (C3)	III
61	Isolation Forest anomaly detection	Must (C3)	III
62	Stacking meta-learner (calibrated composite score)	Must (C3)	III
63	SHAP explainability (top- <i>k</i> features)	Must (C3)	III
64	Authority Brief UI for approvers	Must (C3)	II–III
65	Commit–reveal oracle score gate before approval	Must	III
66	LOAN_RISK_ASSESSMENT audit row per inference	Must	III
67	BCCC-DeFiFraudTrans benchmark evaluation	Must	II–III
68	Chainlink Functions DON oracle	Future Work	–
69	GNN (GraphSAGE) ablation	Future Work	–
70	Federated learning across banks	Future Work	–
<i>Conversational agent</i>			
71	Qwen3-8B local LLM chat interface	Must	III
72	MCP tool server (3–5 tools at MVT)	Must	III
73	Human confirmation gate for all write tools	Must	III
74	Read tools: account state, loan status, requirements	Must	III
75	Write tools: loan application, installment payment	Must	III
76	RAG over policy documents (Q&A path)	Must	III
77	Live on-chain context injection per turn	Must	III
78	Three-tier prompt assembly (Stable / Context / Volatile)	Must	III
79	Prompt injection scanning	Must	III
80	AGENT_ACTION_LOG audit trail	Must	III
81	Full 17-tool MCP suite	Future Work	–
82	EIP-7702 scoped session keys for agent	Stretch	III
<i>Frontend, API, and data layer</i>			
83	React + TypeScript dashboard (mobile-first)	Must	I–II
84	Reserve transparency dashboard	Should	III
85	Five-state transaction UX machine (idle → success/error)	Should	II
86	PostgreSQL event listener (on-chain → DB sync)	Must	I–II
87	Real-time notifications (WebSocket / Socket.IO)	Should	III
88	Client–bank chat	Should	II
89	Express.js REST API with RBAC	Must	I–II
90	FastAPI ML inference service	Must	II–III
91	API rate limiting and correlation-ID tracing	Should	II
92	The Graph subgraph (optional query layer)	Future Work	–
<i>Security, governance, and compliance</i>			
93	Five-layer defense-in-depth architecture	Specified	I–IV
94	ReentrancyGuard + CEI pattern on contracts	Must	I–II
95	LiquidationEngine (health factor, liquidator bonus)	Should	III
96	SAR workflow (anomaly → freeze account)	Should	III

Table 3.4 (continued on next page)

Table 3.4 (continued)

#	Feature	Priority	Phase
97	Governance timelock + snapshot voting	Specified	III+
98	Slither + Mythril static analysis	Must	IV
99	Foundry fuzz + invariant test suite	Must	II-IV
100	Tenderly monitoring alerts	Should	III-IV
101	Regulatory read-only audit access	Specified	III+
102	GDPR-style data minimisation (hash-only on-chain)	Specified	I-IV
<i>Evaluation and operations</i>			
103	300-client Foundry / Hardhat simulation	Must	IV
104	LLM red-team evaluation protocol	Should	IV
105	Open-source README with reproduction steps	Must	IV

3.2 Societal Impact

The existing global financial system does not provide equal access to financial lending capabilities; tier-based financial flows are often kept hidden from a general client who has little to no financial literacy, and bankers often try tactics to misdirect clients into taking loans since transparency is not their priority. In general, the rich have the ability to back up loans of large amounts by showing their assets and potential profit margins and are given priorities, and the poor who might need financial help as loans for critical issues are ignored for the higher profit margins of banks. Programmable smart contracts are being utilized to generalise the loan limits and loan request submitting and approval system that improves equal access globally to those who need it regardless of financial class in a society and who have the ability to repay a loan despite their reputation set by typical judgemental views of traditional bank authorities. All the retail clients are treated equally; if the client can provide necessary documents for proof of income and stability and stakes, he or she may get loan approval from the *Crypto World Bank* branch that has been set for that local client. The benefit of the smart contracts in CWB is that they are completely transparent for all the tiers, so clients can understand how currency is being made sustainable and functional for all; the blockchain comes with the added benefit of decentralisation, so no single party holds the ability to stop financial flow for one section of the world as a restriction of equal access to currency. It is not just a project to show technical design; it is made to prove that blockchain technology and cryptocurrencies can be introduced to the general mass for universal usage, and a lending platform can be built to improve the traditional tier-based banks.

3.3 Environmental Impact

One of the widely popular major complaints of using a blockchain network to build projects and products on it is the energy consumption and its effect on the environment. Running a proof-of-work (PoW) blockchain calculation consumes a massive amount of energy by the servers or personal computer systems, and the waste of energy compared to the end results of calculations is hard to justify.

That is why our project, CWB, runs on proof-of-stake (PoS) EVM, which is 99.9% more energy efficient compared to PoW consensus chains. The power consumption of Bitcoin's PoW network is estimated at 120 to 150 TWh annually, which is 0.4–0.5% of total global electricity consumption, where our approach to serve a worldwide unified lending platform should not exceed electricity consumption of a few small commercial buildings to keep the servers and network operational.

During development, Ethereum Sepolia will be used for demonstration and to prepare the testnets, and in future Polygon zkEVM Cardona is preferred for more efficient gas cost optimisation and more efficient settlement. The ML layer risk score containing fraud detection, anomaly detection and explainability runs on mid-tier commodity hardware or free cloud services; these are not very expensive services as we planned with lightweight models. ML models are not processed on-chain with the smart contracts; the loan approval, financial flow and other banking services are off-chain work, which reduces stress of on-chain calculations to minimize complexities. During the pre-thesis stage, the total estimation of carbon emission and electricity usage is hard to determine; this information will be more predictable after the final execution of the prototype.

3.4 Ethical Issues

The ethical statements have been clarified at the beginning of the paper. The intention of CWB is to demonstrate how this technology and development stack can be used to create a blockchain-based lending platform with AI-enhanced security and assistance. There is no intention to store and use user data, monetary profit, or to build services for financial gains. The full development process and code are open source and available for use publicly in the official GitHub repository for the project.

For the banking system, client data are mostly controlled and processed for on-chain use using hashes, and the AI-assisted techniques that have been utilized here only help the clients to navigate and suggest how to use the platform; all major decisions regarding loan approval are controlled by human approval gates. The ML layers that use client data are not stored and only used for each session time when a user visits the platform; the risk scores and SHAP explainability are designed to not expose any critical data about the users and the platform adapters.

For professional-grade usage of the platform in the future, clients are managed by fair rules and regulation; the treatment for the lending system and money allocation system follows the traditional banking system. No extraordinary social, psychological, financial systems or laws have been applied here, as this build is a prototype which may evolve in future for more robust usage and more advanced adaptability strategies may be used.

3.5 Project Management Plan

3.5.1 Implementation Phase Plan and Deliverables

Phase I: Foundation and Infrastructure (Weeks 1–4)

Phase I focuses on the foundation layers, such as the smart contracts of World Bank, National Bank, and Local Bank, setting the right format and design of the database and schemas, and successfully deploying the smart contracts on Sepolia to show the contracts are functional. Priority follows the MoSCoW classification:

Table 3.5: Phase I task register: smart contract and database foundation.

Task	Priority	Development Task	Effort (days)
DT-I.01	Must	World Bank contract — reserve management, national bank registration	5
DT-I.02	Must	National Bank contract — borrow from World Bank, lend to Local Banks	5
DT-I.03	Must	Local Bank contract — borrow from National Bank, lend to users	5
DT-I.04	Must	RBAC — WorldBankAdmin, NationalBankAdmin, LocalBankAdmin, BankUser, RetailClient	3
DT-I.04a	Must	MockUSDC.sol — mintable ERC-20 for testnet loans (Section 3.6)	1
DT-I.05	Should	Gas cost management — initiator-pays pattern; Polygon low-fee design	2
DT-I.06	Should	InsuranceFund — 5% interest capture, claims, default coverage	4
DT-I.07	Should	getReserveSummary() on each tier contract (PSR, reserve, insurance fund)	2
DT-I.08	Should	Chainlink Price Feeds (fiat/USD, ETH/USD via AggregatorV3Interface)	3
DT-I.09	Should	Chainlink Proof of Reserve — WorldBankReserve attestation	2
DT-I.10	Should	Append-only audit_logs PostgreSQL role + RLS policies	2
DT-I.11	Could	Polygon zkEVM Cardona migration — RPC, chain IDs, factory addresses	1
DT-I.12	Must	Phase I schema migration — 13 M1 tables (Section 4.2.4); M2/M3/stub tables deferred	4
DT-I.13	Must	Core indexes and referential integrity constraints	2
DT-I.14	Must	React frontend shell — Vite build, routing, layout components	3
DT-I.15	Must	WalletConnect — connection, address display, network switching	2
DT-I.16	Should	Sessions and assets tables in frontend (dashboard stubs)	2
DT-I.17	Should	Credit tier and interest rate schedule display	2
DT-I.18	Must	Safe 2-of-3 multisig + WorldBankAdmin role transfer (MVT item 15)	1

Phase I effort: 31 Must days; 51 all priorities.

Phase II: Core Banking and On-Chain Lifecycle (Weeks 5–9)

For Phase II, we focus on building the React user interface to be responsive with loan requests and other banking functionalities such as loan approval, installments, registering accounts, and setting borrowing limits. Moreover, this phase builds the building blocks for Phase III that will incorporate the conversational agent and the tools that the agent has access to.

Table 3.6: Phase II task register.

Task	Priority	Development Task	Effort (days)
DT-II.01	Must	Loan request submission with blockchain transaction	5
DT-II.02	Must	Loan approval and rejection workflow with bank approver	5
DT-II.03	Must	Installment payment system with auto-mated schedule generation	8
DT-II.04	Must	Borrowing limit enforcement (on-chain authoritative check per client SBT)	5
DT-II.05	Should	Client–bank real-time chat system	5
DT-II.06	Should	Income verification document upload and review	5
DT-II.07	Must	Hierarchical bank registration (National → Local)	5
DT-II.08	Should	Bank user management and approver designation	3
DT-II.09	Should	Loan history and transaction tracking pages	5
DT-II.10	Could	QR code generation for wallet addresses	2
DT-II.11	Should	Responsive UI polish and error handling	2
DT-II.12	Should	Authority Brief UI (SHAP breakdown for bank approver dashboard)	5
DT-II.13	Should	Installment due-date checks: Node.js cron (MVT demo path) or Chainlink Automation upgrade when LINK funded	5
DT-II.14	Should	Credit tier schedule in Credit Passport SBT (Bronze → Diamond)	3
DT-II.15	Should	interest_rate_tier table integration with Kinked Rate Model	2
DT-II.16	Should	agent_action_log + SESSIONS schema migration (agent audit trail foundation)	3
<i>Phase II total estimated effort (Must-have):</i>			<i>28 days</i>
<i>Phase II total estimated effort (all priorities):</i>			<i>68 days</i>

Phase III: AI/ML Oracle Pipeline and Conversational Agent (Weeks 10–13)

Phase III is the continuation of Phase II, using all lending functionalities such as loan request, approval workflow, and adding the ability of the MCP agent to interact with these tasks. The ML layer is also integrated with the oracle link and a connection between the on-chain and off-chain is established. Implementation of conversational agent harnessing including MCP server, conversational UI, and confirmation gates from both client and bank authority.

Table 3.7: Phase III task register.

Task	Priority	Development Task	Effort (days)
DT-III.01	Future Work	Chainlink Functions oracle (production upgrade): DON-based score commit when supported; commit–reveal is MVT demonstration path	6
DT-III.02	Must	RF + Isolation Forest + SHAP pipeline (FastAPI); Authority Brief explainability	7
DT-III.03	Must	MCP tool server — minimal (3–5) tool subset wired to Express.js API; one write demo E2E	4
DT-III.04	Must	Agent chat interface (Qwen3-8B + tool calling + human-gate)	8
DT-III.05	Should	Session-key management (EIP-7702) and scope enforcement	5
DT-III.06	Must	Three-tier prompt constructor in Express.js	5
DT-III.07	Must	Prompt injection scanning + confirmation audit hook (Hook 1)	5
DT-III.08	Should	Session lineage + <code>get_session_history</code> MCP tool	5
DT-III.09	Should	Context compression path + toolset scoping + Hook 2	9
DT-III.10	Should	(Optional) The Graph query layer + Web-Socket event listener for Reserve Transparency Dashboard	4
DT-III.11	Should	SAR workflow: Isolation Forest → aml-alert → <code>freezeAccount</code> ; compliance review queue integration	4
DT-III.12	Should	Reserve Transparency Dashboard: live queries to event-listener index (Graph optional later)	3
DT-III.13	Could	GraphSAGE GNN ablation	5
DT-III.14	Could	Federated learning aggregator	7
<i>Phase III total estimated effort (Must-have):</i>			<i>29 days</i>
<i>Phase III total estimated effort (all priorities):</i>			<i>77 days</i>

Phase IV: Verification, Evaluation, and Finalization (Weeks 14–16)

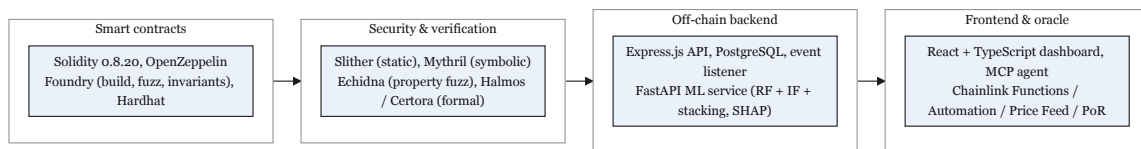
Implementing the Foundry fuzz suite, fully functional 300-client simulation, testnet deployment (local + remote), and the LLM agent evaluation reports, and finally paper refinements and finalization.

Table 3.8: Phase IV task register.

Task	Priority	Development Task	Effort (days)
DT-IV.01	Must	300-client Foundry simulation on Anvil/Hardhat (deterministic seed); gas-cost table and reserve-dynamics manifest to deployments/testnet/; optional Sepolia replay for explorer evidence	7
DT-IV.02	Should	LLM evaluation protocol: task accuracy, action accuracy, human-gate bypass (Section 4.2.17; MVT item 12)	5
DT-IV.03	Future Work	Certora reserve proofs: CVL specs for Invariant 1 (reserve never under-collateralised) and Invariant 2 (no over-allocation downstream); specs in Appendix B	5
DT-IV.04	Should	Foundry invariant + fuzz suite: solvency, role segregation, capital-flow direction; 10,000 fuzz runs per CI build	4
DT-IV.05	Should	AML pre-check (Hook 3): IF anomaly gate before disbursement writes; compliance queue routing; requires Phase III IF pipeline	3
DT-IV.06	Could	Groth16 zkKYC (Circom 2.0): ID possession proof without PII; circuit I/O specification	5
DT-IV.07	Must	Final paper revision: tool-count and phase refs consistent; master checklist complete; bibliography verified	4
DT-IV.08	Must	Red-team testing: 20 injection payloads; 20 bypass attempts; 10 session-scope scenarios; results in evaluation subsections	3
DT-IV.09	Must	Slither + Mythril on World/National/Local Bank contracts; triage and archive (MVT item 13)	2
DT-IV.10	Should	Tenderly alerts: 3 demo-critical triggers + README reproduction (MVT items 10, 16)	2
<i>Phase IV total estimated effort (Must-have):</i>			<i>16 days</i>
<i>Phase IV total estimated effort (all priorities):</i>			<i>40 days</i>

Table 3.9: Four-phase roadmap.

Phase	Focus	Weeks	Tasks	Key deliverables
I	Foundation and infrastructure	1–4	DT-I.01–18	Contract hierarchy, MockUSDC, core DB schema, React shell
II	Core banking and on-chain lifecycle	5–9	DT-II.01–16	Lending workflow, SBT limits, cron/Automation triggers
III	AI/ML oracle pipeline + conversational agent	10–13	DT-III.01–14	Commit–reveal oracle, ML pipeline, MCP agent (minimal toolset), dashboard
IV	Verification, evaluation, finalization	14–16	DT-IV.01–10	Foundry, simulation, evaluation pack, testnet deploy

**Figure 3.1:** Planned development and verification toolchain

3.5.2 SDLC Stage Mapping

Table 3.10: SDLC stage mapping.

#	SDLC Stage	Project Activity	Deliverable
1	Planning	Feasibility studies; professor consultations; guideline review	Feasibility Report; Project Plan
2	Requirements	System analysis; use case definitions; constraints identification	Use case diagrams; UC-1 through UC-11 (eleven grouped use cases)
3	Design	Four-layer architecture; DB schema (51 entities, 3NF); smart contract interfaces	Architecture diagrams; ERD; DFD
4	Development	Phase I–IV: smart contracts, frontend, backend, AI/ML, agent	Source code; DApp prototype
5	Testing	Hardhat unit tests; Foundry fuzz; integration testing; AI/ML + agent evaluation	Test reports; model metrics
6	Deployment	Ethereum Sepolia (Phases I–IV demonstration); Certora on Sepolia (Phase IV stretch); frontend (Vercel); backend local-first (Render optional)	Testnet prototype manifest
7	Maintenance	Monitoring; model retraining; bug fixes; iteration	Updated docs; retrained models

3.6 Risk Management

Table 3.12: Design reference versus demonstration path and key mitigations.

Area	Full design	Demo path (Ph. I–IV)	Risk mitigated
Network	Polygon zkEVM Cardona	Sepolia primary; Amoy secondary	Testnet instability
Stablecoin	USDC / USDT	MockUSDC.sol	No official testnet USDC
Oracle	Chainlink DON	Commit–reveal relay	Oracle cost / latency
ML delivery	Chainlink Functions	FastAPI + local relay	Integration complexity
Simulation	300 wallets live	Foundry/Anvil + 5–10 live demo	Faucet funding
Hosting	Cloud API + ML	Local stack + Vercel frontend	Cold-start / cost

3.7 Economic Analysis

3.7.1 Prototype Cost Model

Table 3.13: Economic feasibility — zero-cost prototype.

Cost Category	Estimate	Notes
Blockchain deployment	\$0	Public testnets — no real cryptocurrency required
Frontend hosting	\$0	Vercel free tier or localhost for demo
Backend hosting	\$0	Render free tier or localhost
AI/ML training	\$0	Local machine or Google Colab free tier
Development tools	\$0	Hardhat, VS Code, Git — all open-source
Total prototype	\$0	Entire prototype operates at zero financial cost

3.7.2 Gas Cost and Break-Even

Table 3.14: Gas cost and break-even assumptions (Polygon zkEVM Cardona design preference).

Parameter	Estimate	Notes
On-chain steps per loan lifecycle	27–32	Full retail lifecycle
Gas price (planning)	≈30 Gwei	—
ETH price (planning)	\$2,500	—
Total gas (Cardona / L2)	≈\$0.011	<0.01% of interest on \$25,000 USDC loan at 8% APR
Total gas (Ethereum mainnet)	\$2.40–\$4.80	Same lifecycle; supports L2 design choice [37]
Average loan size	\$25,000	Break-even base case
Default rate assumption	8%	—
Infrastructure cost	\$500/month	Pilot-scale assumption
Break-even active loans	≈200	Covers default losses at base case

3.7.3 Net Present Value and Return on Investment

Table 3.15: Illustrative three-year NPV and ROI at pilot scale (300 active loans, \$25,000 average).

Year	Benefits (\$k)	Costs (\$k)	Net (\$k)	PV factor @ 10%
0 (setup)	0	6.0	−6.0	1.000
1	48.0	6.0	42.0	0.909
2	72.0	8.0	64.0	0.826
3	96.0	10.0	86.0	0.751
NPV (sum of discounted net)				\$149.6k
ROI (3-yr benefits – costs) / costs				1,247%
Break-even loans (base case)				≈300

Chapter 4

Design Alternatives

This project paper is constructed to match with the final year project evaluation rubrics. Full development work has been distributed to 4 phases, and priority of all the capability sections has been divided to Must, Should and Future work. The Must and Should priority sections are to be completed by the end of the development and testing phase, Future Work sections have been specified only, to show the Potential of the project by extending its functionality and development scopes.

Table 4.1: Capability priority summary.

Capability	Priority	Primary interface / phase
Four-tier smart contracts	Must (Ph. I–II)	React UI + wallet; Hardhat/Foundry tests
On-chain loan life-cycle	Must (Ph. II)	React forms; LoanController state machine
Chainlink Functions oracle	Future Work	FastAPI → DON → on-chain score commit; commit–reveal is MVT demonstration path
RF + Isolation Forest + SHAP	Must (Ph. III)	Approver Authority Brief; client decline reasons
Authority Brief UI	Should (Ph. II–III)	Bank approver dashboard (React) — <i>not</i> agent-only
MCP conversational agent	Must (Ph. III)	Qwen3-8B + 3–5-tool MCP server; one write demo E2E
LLM evaluation protocol	Should (Ph. IV)	Red-team + task accuracy (Section 4.2.17; deferred)

4.1 Design Process or Methodology Overview

We have developed a plan to adapt lightweight Agile/Scrum Methodology for the prototype build with an estimated development window of 16 weeks. This iterative approach enables us to explore and improve the demonstrable features that include the minimum requirements and advanced research oriented features. These feature development phases are organised in four incremental stages (Section 3.5.1).

Agile Methodology application process:

- **Phase duration:** each phase is designed for 3–5 weeks, total 4 phases, total estimated time — 16 weeks.
- **Team Size:** 2 developers

- **Weekly Sync:** One weekly meeting will be conducted on Tuesday. Completion of each feature, testing and next development plans will be discussed in these meetings, based on the opinion and suggestions received from the supervisor.
- **Phase Planning and review:** After completion of each phase, a long reflective phase meeting will be arranged, to organize and plan upcoming weeks.

4.1.1 Process Model Selection

Based on the Software engineering process for a blockchain project, a development plan of Agile/Scrum with incremental phases has been chosen. Other models such as Waterfall, Spiral, V-Model have some flaws that do not meet the requirements to successfully build all the features.

Table 4.2: Process model comparison and selection rationale.

Model	Fit for CWB	Limitation	Decision
Waterfall	Clear phase documents	Cannot absorb evolving smart-contract and ML scope mid-project	Rejected as primary
Incremental	Matches four phases + gates G1–G4	Still needs within-phase flexibility	Adopted (phase structure)
Iterative / Agile	Weekly sync; phase retrospectives	Requires discipline on scope	Adopted (within-phase)
Spiral	Strong for high-risk R&D	Overhead excessive for 16-week student project	Not adopted
V-Model	Maps to test levels in Ch. 5	Too rigid for re-search prototyping	Used for <i>verification planning</i> only

The project is built in the main branch from the root section. It follows a conventional **Model–View–Controller (MVC)** architecture for separation in root. **Model** – contracts/: on-chain state machines and PostgreSQL (off-chain projections), **View** – frontend/ (React components), **Controller** – backend/. For version control the repository is managed with git command, with consistency and compatibility checks, validated by `npm run test:contracts` (Hardhat) before each merge into main branch.

4.1.2 Implementation Feasibility and Demonstration Scope

The full project scope—phase task registers (DT-I.01–IV.10), the MVT checklist (Table 3.3), and the module inventory in Chapter 4—remains the project record. This section states what we will actually build and demo in the 16-week window using free testnets and local tools.

Phase I targets roughly 31 person-days of Must work for a two-person team over four weeks. That is enough for core contracts, database setup, and a basic frontend, but only if Should tasks are not treated as gate blockers. Phase I originally listed about 30 Must person-days; the revised plan keeps the same scope but adds explicit room for testnet debugging.

The Phase I (G1) demo must show:

- Three tier contracts and MockUSDC deployed on Sepolia
- A working capital-flow demo (`allocateCapital` across tiers)
- Passing RBAC tests in Hardhat or Foundry
- A React dashboard connected via WalletConnect that reads live reserve balances
- PostgreSQL M1 tables migrated with seed data (Section 4.2.4)
- A short screen recording (≤ 5 minutes) as the MVT #1 artifact

Production upgrades—Chainlink DON oracles, live USDC, full 51-table schema, and main-net deployment—stay documented in Chapter 4 and Future Work. They are not required for the graded prototype.

4.2 Preliminary Design or Design (Model) Specification

4.2.1 System Design Overview

This chapter documents the technical dependencies and logical design based on in-depth system analysis of the project. The chapter demonstrates a four-tier banking system with on-chain lending flows and off-chain data and AI/ML layer integration and optional conversational AI agent support. All technical specifications and logical explanations of their interconnectivity, and how they work as a complete system, have been documented here sequentially and structurally. The tables contain concise information and the figures are provided for visualization of each complicated section.

4.2.2 High-Level Architecture

The system uses a 4-layer architecture: React for frontend, application services by Express.js and FastAPI, persistence by PostgreSQL, and Solidity for on-chain contracts. The layers are summarized in Table 4.3 and visually demonstrated in Figure 4.1, including how they interact.

Table 4.3: Four-layer architecture: technical components and responsibilities.

Layer	Technology	Key components	Responsibility
Presentation	React, TypeScript, WalletConnect / RainbowKit	Web UI, wallet signing, app-prover dashboard	User interaction and transaction initiation
Application services	Express.js, FastAPI	REST API, ML scoring service, MCP agent server, event listener	Business logic, AI/ML inference, agent tool calls, API orchestration
Persistence	PostgreSQL, Redis (cache)	Lending projections, audit logs, session and agent tables	Off-chain data storage and read models synced from chain events
On-chain	Solidity (EVM)	0.8.20 World / National / Local Bank contracts, LoanController, deposit and pool modules	Authoritative banking state, RBAC, loan lifecycle, reserve rules

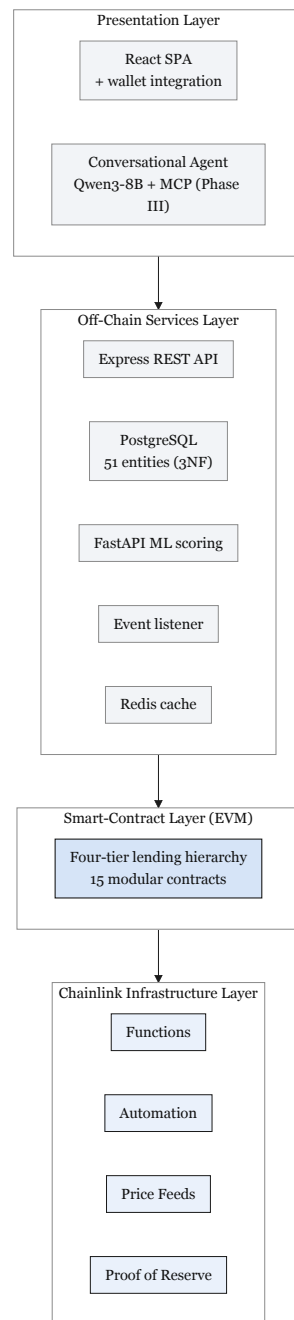


Figure 4.1: Four-layer application architecture

Figure 4.2 shows component-level interactions. Core lending contracts are specified in Phase I; extended modules are documented in Appendix C.

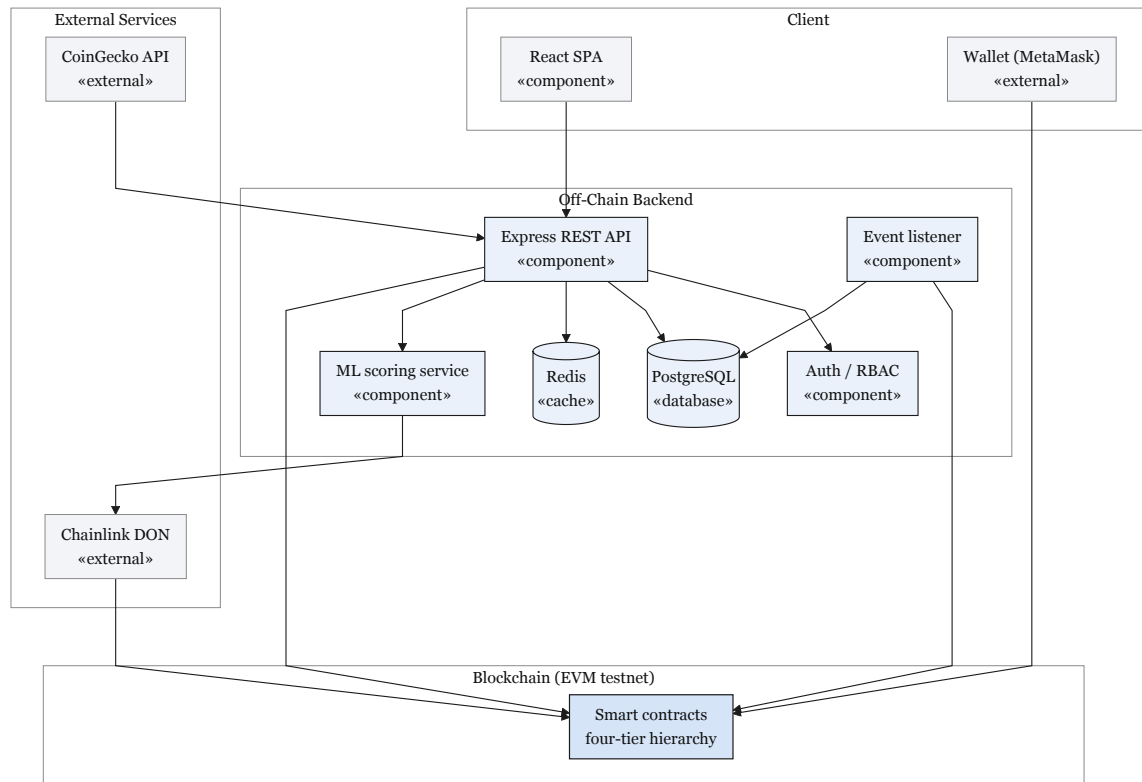


Figure 4.2: Component diagram (planning phase): layered client–server topology with PostgreSQL repository and external oracle feeds.

Figures 4.3 and 4.4 model the core lending subsystem in Gane-Sarson notation: Level-0 (Figure 4.3) shows boundary flows to five external entities; Level-1 (Figure 4.4) decomposes Process 0 into Processes 1.0–5.0 with data stores D1–D6.

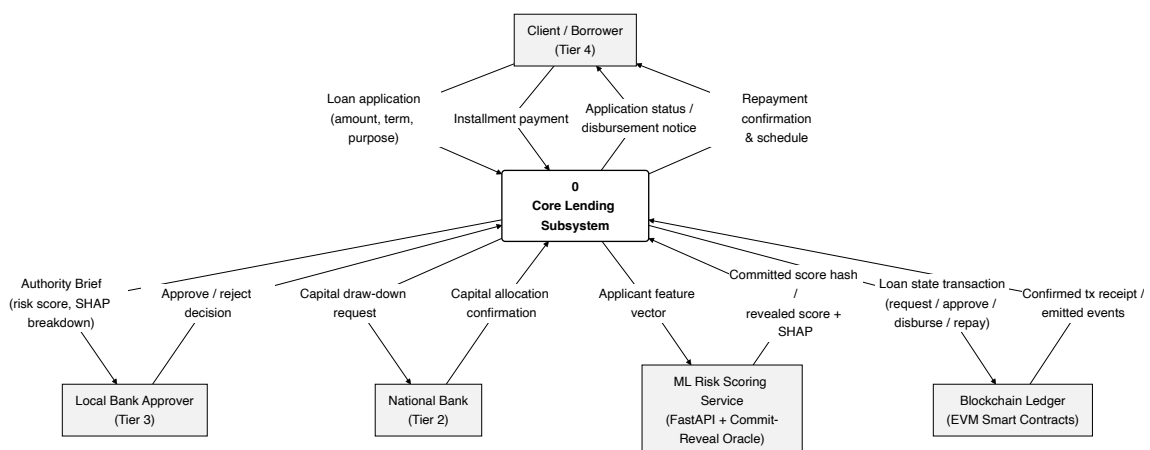


Figure 4.3: Level-0 data-flow diagram (Gane-Sarson notation): context view of the core lending subsystem and its external entities.

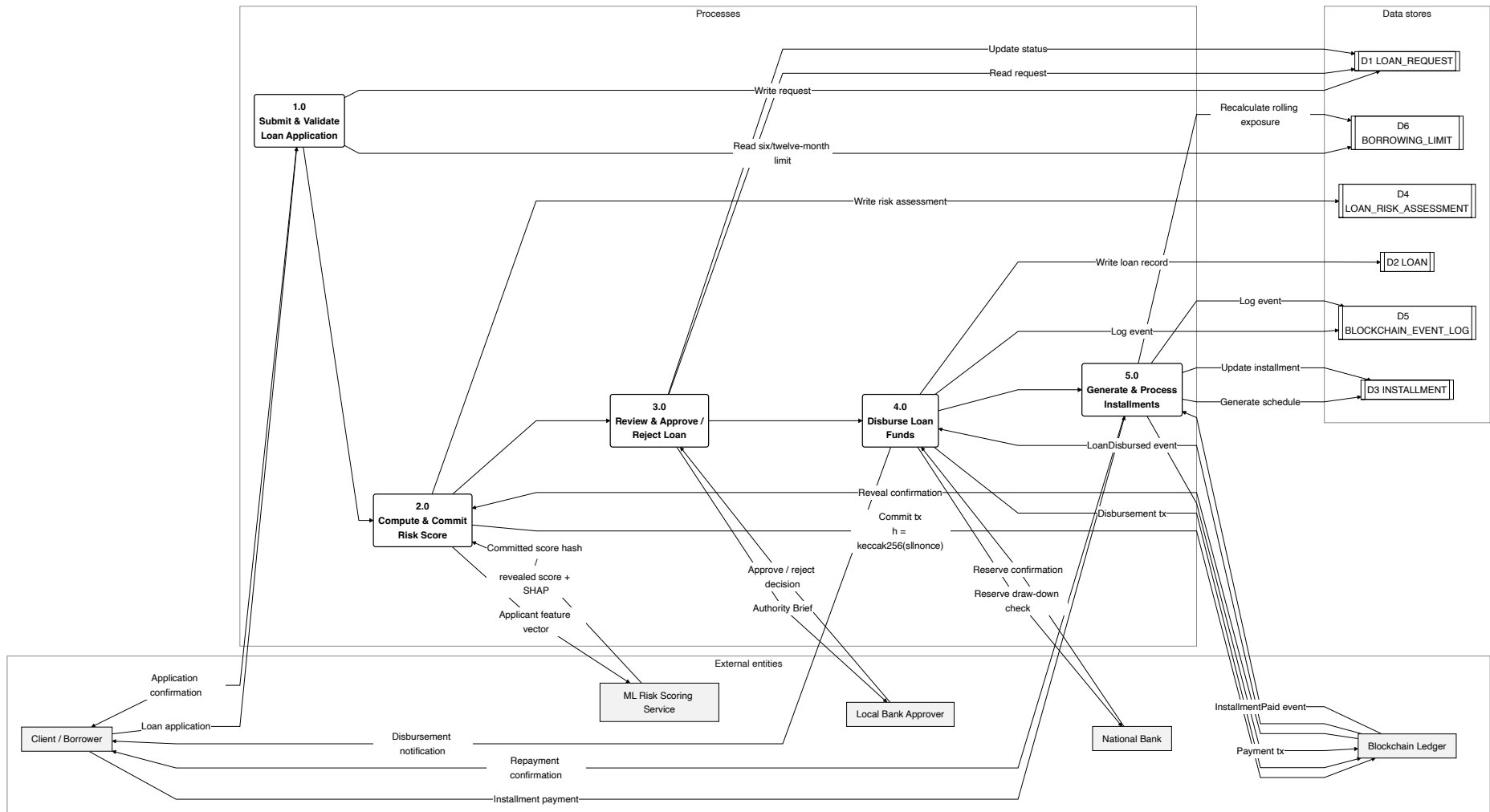


Figure 4.4: Level-1 data-flow diagram (Gane-Sarson notation): decomposition of Process 0 into Processes 1.0–5.0 with data stores D1–D6.

Figure 4.5 is the detailed UML-style component diagram: four layers, required interfaces (lollipop notation), and delegation between Express.js, FastAPI, MCP agent, persistence stores, and on-chain contract modules.

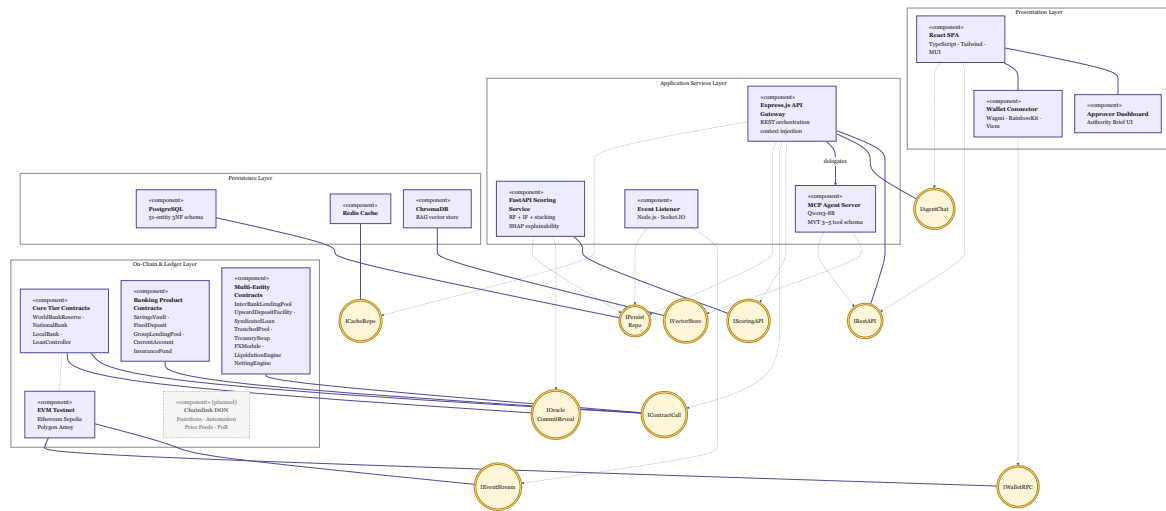


Figure 4.5: System component architecture: presentation, application services, persistence, and on-chain layers with interface dependencies and contract groupings.

Table 4.4: Smart contract inventory: seventeen modular contracts across core, product, and multi-entity modules.

Contract	Module	Status	Primary responsibility
WorldBankReserve	Tier 1 core	Specified (Phase I)	Global reserve custody, national-bank registration, downward capital allocation.
NationalBank	Tier 2 core	Specified (Phase I)	Local-bank registration, upward borrowing, downward allocation.
LocalBank	Tier 3 core	Specified (Phase I)	Client onboarding, approver management, owns <code>LoanController</code> ; <code>freezeAccount</code> (Phase II+).
LoanController	Lending state machine	Ph. I shell; Ph. II	Loan request, approval, disbursement, installment state machine (DT-II.01–03); deployed by <code>LocalBank</code> .
SavingsVault	Deposits	Specified (Phase II)	Variable-yield savings, reserve-gated withdrawals.
FixedDeposit	Deposits	Stub (Phase II+)	Term deposits with deterministic early-withdrawal penalty.
GroupLendingPool	Group credit	Specified (Phase II)	Solidarity-group formation, consent, mutual liability.
FXModule	Treasury / FX	Planned (Phase III)	Oracle-priced retail FX with disclosed spread.
InsuranceFund	Risk buffer	Ph. I (often II)	Default coverage buffer; Chainlink Proof of Reserve.
CurrentAccount	Payments	Stub (Phase II+)	Transactional accounts and atomic peer transfers.
InterBankLending-Pool	Multi-entity	Specified (Phase II)	Same-tier interbank liquidity pools per tier.
UpwardDeposit-Facility	Multi-entity	Specified (Phase II)	Upward surplus repatriation with yield spread.
SyndicatedLoan	Multi-entity	Planned (Phase III)	Multi-lender syndicate consent and disbursement.
TranchedPool	Multi-entity	Planned (Phase III)	Senior/junior tranching and waterfall recovery.
TreasurySwap	Multi-entity	Planned (Phase III)	Institutional oracle-priced asset swaps.
LiquidationEngine	Risk / liquidation	Planned (Phase III)	Health-factor monitoring; third-party liquidator bonus when $HF < 1.0$ (Section 4.2.8).
NettingEngine	Multi-entity	Future Work	ERD stubs only; Section 6.2

4.2.3 Blockchain Platform Selection

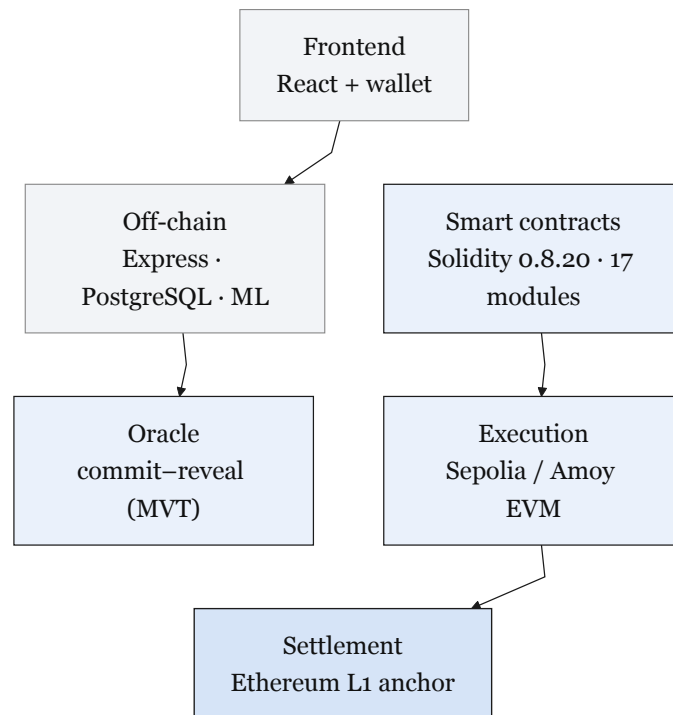


Figure 4.6: Layered blockchain and application stack

Table 4.5: Blockchain platform selection criteria and justification.

Criterion	Selection		Justification
Platform	Ethereum	Virtual Machine (EVM)	Largest EVM ecosystem; mature tooling.
Network	Ethereum (primary) Polygon PoS (secondary prototype implementation)	Sepolia and/or Amoy (secondary implementation)	Free public testnets; zkEVM evaluated, not default prototype chain.
Consensus	Public-testnet consensus (testnet)	consensus (PoS/L1)	Stable testnet consensus; ZK settlement is a design preference only.
Smart contract language	Solidity 0.8.20		Industry standard; overflow-safe compiler; rich libraries.

Table 4.7: Blockchain platform selection (continued): operational and deployment factors.

Criterion	Selection	Justification
Gas cost	\$0.001–\$0.01 per transaction	Far below mainnet; viable for micro-loans.
Block finality	≈2 seconds	Fast retail UX; L1-anchored security.
Developer tooling	Hardhat + OpenZeppelin	Hardhat tests; audited OpenZeppelin libs.
Testnet availability	Amoy (Polygon PoS), Sepolia (Ethereum)	Free faucets; EVM-identical; no real funds.
Security model	Public-testnet security model	Testnet PoS; ZK settlement evaluated separately.
Migration path	EVM-compatible	Portable Solidity across EVM chains.

Transaction Verification and Consensus

The prototype deploys on **Ethereum Sepolia** (primary) with **Polygon Amoy** as a secondary fallback (Tables 4.5). Both are public PoS testnets with free faucets and no real-asset exposure. Polygon zkEVM Cardona remains a **design-preference** target for future ZK-anchored settlement but is not the default MVP chain. The retail loan lifecycle involves multiple on-chain state transitions per client; projected gas cost stays well below 0.01% of interest on a representative micro-loan and is validated in Phase IV (Section 3.7.2).

Oracle Architecture: Off-Chain AI to On-Chain Decision

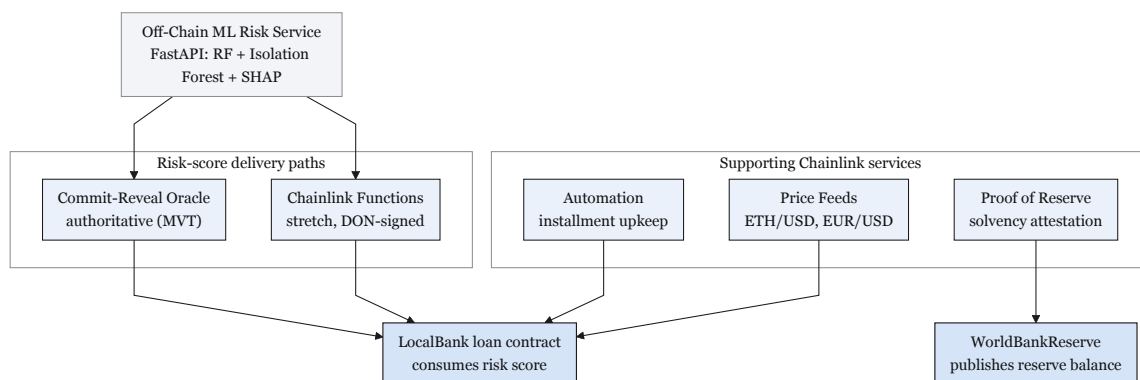
Off-chain ML risk scores must reach the on-chain loan approval workflow without trusting a single party at decision time (the oracle problem [40, 41]). The graded MVT path is a **commit–reveal relay**: the FastAPI scoring service commits $h = \text{keccak256}(s \parallel \text{nonce})$ to LoanController, then reveals the score s and nonce within the decision window. The contract verifies the hash and stores s immutably, giving an audit trail and preventing post-commit score changes. A trusted backend relay may assist integration in Phases I–II only.

Figure 4.7 summarises the authoritative MVT path and the documented Chainlink upgrade routes. Table 4.9 maps each Chainlink service to its production role and the zero-cost demonstration substitute (Section 3.6; Table 3.12).

Table 4.9: Chainlink upgrade paths versus MVT demonstration substitutes.

Service	Production role	MVT / demo substitute
Chainlink Functions	DON-signed fetch of ML risk score from the off-chain service	Commit-reveal relay (authoritative at MVT scope)
Chainlink Automation	On-chain upkeep for overdue installments and interest accrual	Node.js cron calling <code>checkOverdueLoans()</code>
Price feeds	FXModule and fiat-equivalent UI via <code>AggregatorV3Interface</code>	<code>MockPriceFeed.sol</code> with fixed test-net answers
Proof of Reserve	External attestation of <code>WorldBankReserve</code> solvency	On-chain <code>getReserveSummary()</code> view; PoR job is Future Work

Chainlink Functions adds roughly 30–60 s latency and a per-call fee, acceptable at loan origination but unnecessary for the graded prototype. Production contract interfaces for reserve reporting and FX conversion are specified in Appendix D; integration code remains in the repository rather than in this chapter.

**Figure 4.7:** Oracle architecture

4.2.4 Data Model and Database Design

This section describes the PostgreSQL data model for Crypto World Bank. The complete logical schema defines **51 entities** in Third Normal Form (3NF); Table 4.10 summarises the **16 core entities** that drive the prototype, and Table 4.11 groups the remaining extension and stub relations by phase.

The graded prototype **migrates about 32 tables** across Phases I–III (Section 4.2.4); Gate G1 requires 13 tables only. The full 51-entity relational ERD is in Appendix A (Figure A.1) because of its size; Figures 4.8 and 4.9 below provide readable focused views of the core lending graph and banking-product extensions.

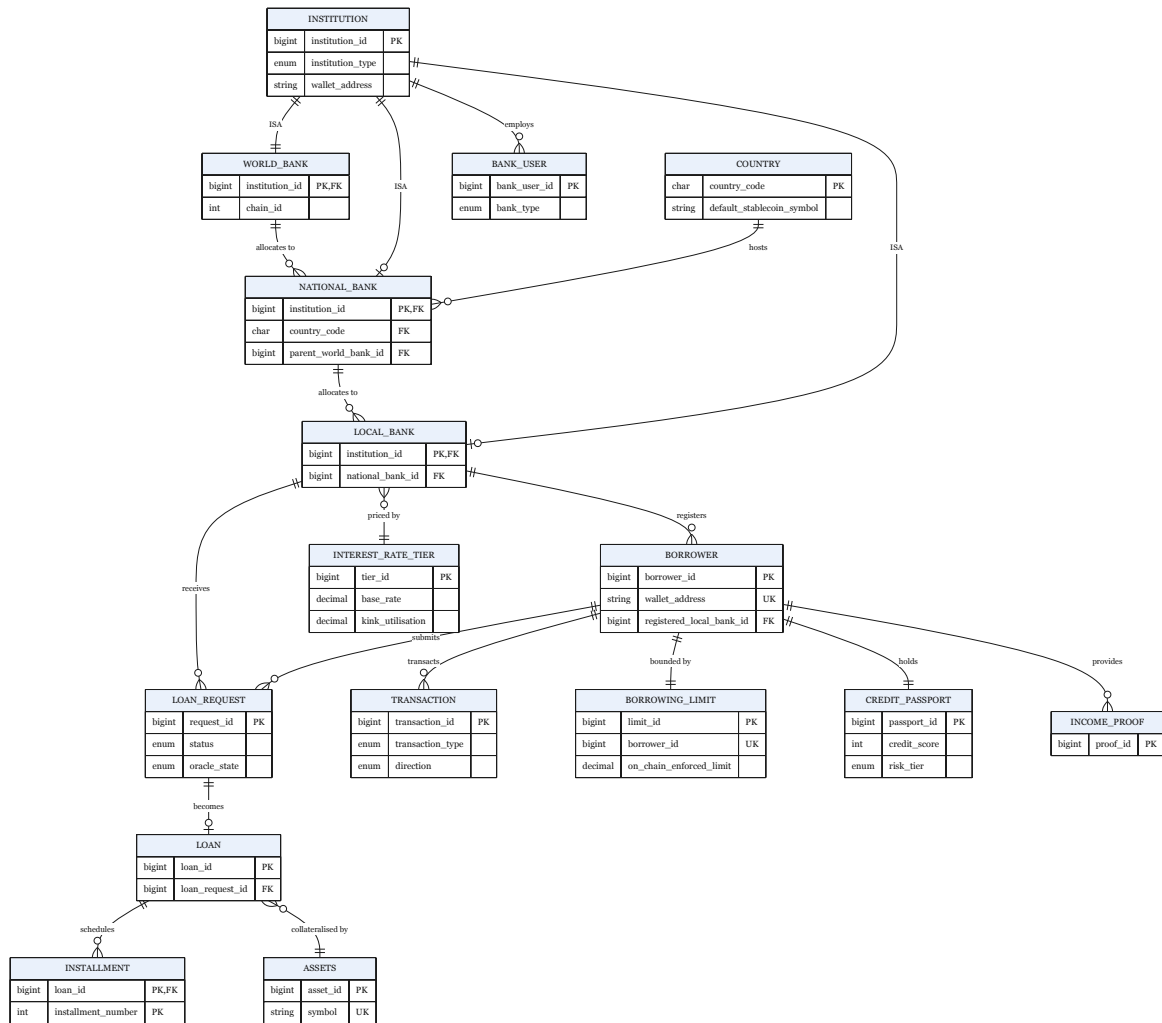


Figure 4.8: Core system entity graph

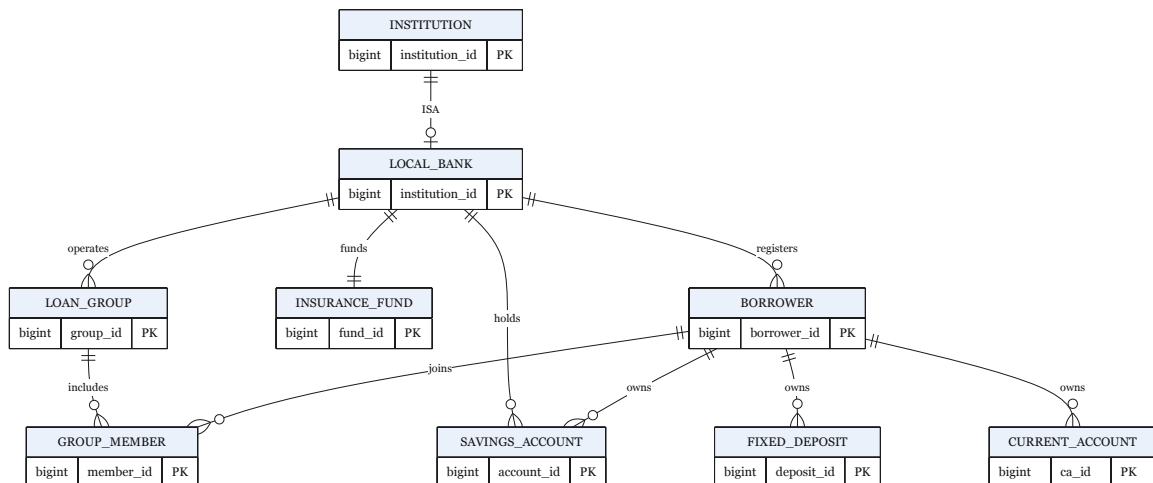


Figure 4.9: Extended ERD: banking-product entities (crow's-foot; PK shown; entity roles in Table 4.11).

Figure 4.9 documents banking-product extensions; FK anchors into core lending entities are in Table 4.23 (Section 4.2.9).

Entity Summary

Table 4.10: Core database entities: primary keys and roles (Phase I–III prototype scope).

Entity	Primary key	Role
<i>Institution hierarchy (M1)</i>		
INSTITUTION	institution_id	Shared supertype for World, National, and Local bank identities; enables single FK target for cross-tier operations.
COUNTRY	country_code	ISO 3166-1 reference; one NATIONAL_BANK per country (UNIQUE).
WORLD_BANK	institution_id	Global reserve subtype; singleton (CHECK institution_id = 1).
NATIONAL_BANK	institution_id	National reserve subtype; FK country_code, parent_world_bank_id.
LOCAL_BANK	institution_id	City-level lending institution; FK national_bank_id.
<i>Actors and lending lifecycle (M1)</i>		
BANK_USER	bank_user_id	Bank staff; exactly one parent institution enforced by CHECK (Table 4.15).
BORROWER	borrower_id	Retail client (UK: wallet_address); home bank via registered_local_bank_id; hash-only KYC fields.
LOAN_REQUEST	request_id	Application workflow (status, oracle_state); links borrower to local bank.
LOAN	loan_id	Active loan after disbursement; FK loan_request_id; on-chain linkage fields.
INSTALLMENT	loan_id, installment_number	Repayment schedule (weak entity on LOAN).
<i>Sync, assets, and audit (M1)</i>		
BLOCKCHAIN_EVENT_LOG	event_id	Canonical on-chain event cache (event-listener owned); FK target for lending tables.
ASSETS	asset_id	Collateral and loan asset registry (UK: symbol).
AUDIT_LOGS	audit_id	Append-only governance and compliance log (Express-owned).
<i>Phase II–III extensions (selected)</i>		
TRANSACTION	transaction_id	Financial ledger for retail and institutional flows.
BORROWING_LIMIT	limit_id	Off-chain projection of on-chain rolling borrow limits.
CREDIT_PASSPORT	passport_id	Event-listener projection of CreditPassport SBT state.
LOAN_RISK_ASSESSMENT	assessment_id	Per-loan ML inference record; FK model_id → MODEL_REGISTRY.
SESSIONS	session_id	Retail agent wallet session (Phase III); FK borrower_id.

Table 4.11: Deferred and extension entities (35 relations): grouped by implementation phase. Full attribute lists are omitted; see ERD figures and Section 4.2.4.

Tier	Entities	Purpose
M2	CREDIT_PASSPORT_HISTORY, INCOME_PROOF, INTEREST_RATE_TIER	SBT history, Level 2+ document hashes, normalised kinked rates
M2	GROUP_CONSENT, LOAN_GROUP, GROUP_MEMBER	Group lending (if GroupLendingPool demo chosen)
M2	INTERBANK_LOAN, UPWARD_DEPOSIT, SAVINGS_ACCOUNT	One multi-entity PoC path
M3	MODEL_REGISTRY, SECURITY_EVENT_LOG	ML lineage and system-wide security events
M3	AGENT_ACTION_LOG, AGENT_CONVERSATION_TURN, SESSION_ANCESTOR	Agent audit and session lineage
Stub	FIXED_DEPOSIT, CURRENT_ACCOUNT, INSURANCE_FUND	Deposit and risk-buffer products
Stub	SYNDICATE, SYNDICATE_MEMBER, TRANCHED_POOL, TRANCHED_POOL_SUBSCRIPTION	Syndication and tranching lending
Stub	TREASURY_SWAP, NETTING_BATCH, NETTING_ENTRY	Treasury swap and netting (Future Work)
Future	CHAT_MESSAGE, INSTITUTION_MESSAGE, PROFILE_SETTING	UX messaging and preferences
Future	MARKET_DATA, CHAINLINK_PRICE_ROUND, ORACLE_HEARTBEAT_MONITOR	Oracle caches (mock at MVT)
Future	COLLATERAL_POSITION, KYC_VERIFICATION_REQUEST, NETTING_COORDINATOR_STATE	Liquidation, ZKP KYC, settlement coordinator
Future	CHAIN_REGISTRY, SESSION_KEY_PERMISSION	Phase IV platform configuration

Logical Schema versus Physical Migration Tiers

The 51-entity count describes the **complete logical data model** for architectural consistency with the seventeen-contract inventory. The graded prototype does *not* require 51 physical PostgreSQL tables at Gate G1. **Phase I (M1, 13 tables)** covers the institution hierarchy, actors, lending lifecycle, and event sync—the Gate G1 minimum. **Phase II (~12 tables)** adds ledger, credit passport, and rate-tier tables plus one optional product path. **Phase III (~7 tables)** adds ML, agent session, and security-log tables. Remaining entities stay as ERD stubs or Future Work (Table 4.11).

Platform-configuration relations (outside the 51-entity count). CHAIN_REGISTRY (deployment chain IDs, RPC endpoints, EIP-7702 delegation contract addresses) and SESSION_KEY_PERMISSION (normalised MCP tool allowlists per session) are Phase IV platform-configuration tables. They are referenced in Sections 4.2.7 and 4.2.17 but are *not* included in the 51-entity lending schema because they hold deployment metadata rather than banking-domain state.

Table 4.12: Audit and logging table taxonomy: separate relations for on-chain events, governance actions, ML inference, and system security.

Table	Writer	Scope	Typical contents
BLOCKCHAIN_- EVENT_LOG	Event listener	On-chain receipts	tx_hash, event_name, raw_data; FK target for LOAN_REQUEST, LOAN, INSTALLMENT
LOAN_RISK_- ASSESSMENT	FastAPI ML service	Per-loan inference	fraud_probability, shap_vector, FK model_id → MODEL_REGISTRY
AUDIT_LOGS	Express API	Governance / compliance	Travel Rule packets, role grants, manual approver actions (Section 4.2.12)
SECURITY_EVENT_- LOG	ML / monitoring	System-wide security	AML alerts, anomaly spikes, agent injection blocks
AGENT_ACTION_LOG	Express agent hooks	Agent write tools	tool_name, confirmation_turn_id, onchain_tx_hash

EER Constructs Applied

Table 4.13: EER constructs applied: specialisation, hierarchy, and constraints.

EER Construct	Applied To	Notes
Specialisation (dis-joint)	BANK_USER → WorldBankUser, NationalBankUser, LocalBankUser	A bank user belongs to exactly one bank type (world, national, or local); enforced by CHECK constraint (Table 4.15).
Generalisation / specialisation	INSTITUTION → WORLD_BANK, NATIONAL_BANK, LOCAL_BANK	ID-based table-per-type inheritance: subtype PK = FK to INSTITUTION.institution_id; shared identity for cross-tier FKs (Table 4.10).
Weak entity	INSTALLMENT depends on LOAN	Existence and identity determined by parent loan.
Multi-valued attribute	INCOME_PROOF per BORROWER	Modelled as separate entity to maintain 1NF.
Aggregation	LOAN_RISK_ASSESSMENT ← LOAN_REQUEST; SECURITY_EVENT_LOG ← runtime monitors	Per-loan ML scores and system-wide security events are separate append-only relations, not a conflated aggregate.
Association entity	LOAN_REQUEST links BORROWER + LOCAL_BANK	Captures many-to-many borrowing relationships; distinct from BORROWER.registered_local_bank_id (home bank at onboarding).
Participation constraint	BORROWER.registered_local_bank_id NOT NULL at onboarding	Home Local Bank is mandatory; LOAN_REQUEST participation for lending services enforced at application layer.
Append-only table (policy)	AGENT_ACTION_LOG, AUDIT_LOGS, SECURITY_EVENT_LOG	DB-level INSERT-only role + Row-Level Security policy enforces that no UPDATE or DELETE is permitted on audit records.

Normalization

The schema has been normalized to Third Normal Form (3NF) and checked against Boyce-Codd Normal Form (BCNF):

- **1NF:** There are no repeating groups in any of the attributes. Separate entities store multi-valued attributes, such as proof of income.
- **2NF:** There are no partial dependencies. The full composite key (loan_id, installment_number) determines the non-key attributes of INSTALLMENT.

- **3NF:** No dependencies that go through other dependencies. Instead of storing redundant values like `total_borrowed` in bank entities, they are calculated at query time.
- **BCNF:** All non-trivial functional dependencies have a superkey as their determinant. For example, in `INTEREST_RATE_TIER`, `tier_id` $\rightarrow$ `{base_rate, kink_utilisation, rate_above_kink, max_rate}`: the determinant `tier_id` is the primary key, satisfying BCNF. Prior to the normalisation fix in this section, interest-rate parameters were columns on the three bank entities, determined by `bank_tier_id` (a non-key attribute), violating BCNF.

A transitive dependency was identified during schema review: `interest_rate_parameters` $\rightarrow$ `bank_tier_id` (interest rate values were stored as columns in the World Bank, National Bank, and Local Bank entities, making them dependent on a non-key attribute via the tier relationship). This violates 3NF. The fix is to extract these parameters into a dedicated `INTEREST_RATE_TIER` table keyed by `tier_id`, which each bank entity references via FK. This normalisation change ensures that updating a base rate does not require touching multiple bank-tier rows.

Relational Integrity Constraints

Table 4.15: Relational integrity constraints.

Constraint Type	Examples
Primary Key	One surrogate or composite key per entity (Tables 4.10, 4.11): e.g. <code>institution_id</code> , <code>borrower_id</code> , <code>request_id</code> , (<code>loan_id</code> , <code>installment_number</code>), ...
Foreign Key	Tier subtypes $\rightarrow$ INSTITUTION; BORROWER.registered_local_bank_id $\rightarrow$ LOCAL_BANK; LOAN_RISK_ASSESSMENT $\rightarrow$ LOAN_REQUEST, MODEL_REGISTRY; multi-entity stubs $\rightarrow$ INSTITUTION.institution_id.
UNIQUE	BORROWER.wallet_address; NATIONAL_BANK(country_code); INSTITUTION.contract_address; LOAN_REQUEST.blockchain_tx_hash; BORROWING_LIMIT.borrower_id; ASSETS.symbol; BLOCKCHAIN_EVENT_LOG.tx_hash; partial unique on active INTEREST_RATE_TIER(bank_tier_type).
CHECK	WORLD_BANK: singleton (<code>institution_id = 1</code>). BANK_USER: exactly one parent FK matching bank_type. TRANSACTION: at least one of <code>borrower_id</code> , <code>origin_institution_id</code> , <code>counterparty_institution_id</code> non-null.
NOT NULL	INSTITUTION.name, INSTITUTION.institution_type; BORROWER.wallet_address, registered_local_bank_id, kyc_level; status; oracle_state (default NONE).
APPEND-ONLY	AGENT_ACTION_LOG, CREDIT_PASSPORT_HISTORY, SECURITY_EVENT_LOG, AUDIT_LOGS: INSERT-only role + RLS; no UPDATE/DELETE.
PROJECTION (RLS)	CREDIT_PASSPORT, BORROWING_LIMIT, COLLATERAL_POSITION: event_listener role only may INSERT/UPDATE; Express API SELECT-only.
FOREIGN KEY	LOAN $\rightarrow$ ASSETS (collateral, loan asset); AGENT_ACTION_LOG $\rightarrow$ SESSIONS, AGENT_CONVERSATION_TURN; LOAN.disbursement_event_log_id $\rightarrow$ BLOCKCHAIN_EVENT_LOG.

This integrity constraints table summarizes referential and domain constraints that keep loan, repayment, and identity records consistent. These constraints reduce operational risk by preventing orphaned records and invalid lifecycle states, which is critical for auditability in financial systems.

4.2.5 On-Chain and Off-Chain Data Partitioning

Table 4.17: Data partitioning between on-chain and off-chain storage.

Data Category	Storage	Rationale
Reserve balances, loan requests, approval/rejection events, repayment transactions	On-chain	Immutability, public auditability, trustless verification.
User profiles, income verification documents, chat messages, loan risk assessments, security event logs	Off-chain (database)	Data privacy, query flexibility, storage cost optimisation.
Borrowing limit computations	Off-chain with on-chain enforcement	Complex temporal aggregation; results committed as on-chain constraints via <code>LocalBank.updateBorrowingLimit(borrowerId, newLimit)</code> (see sync protocol below).
Cryptocurrency market data (non-Chainlink)	Off-chain (MARKET_DATA)	CoinGecko and other fallback feeds; high-frequency updates; external API dependency.
Chainlink oracle price data	Off-chain (CHAINLINK_PRICE_ROUND)	Each <code>latestRoundData()</code> round cached with <code>answered_in_round</code> for staleness detection; on-chain <code>AggregatorV3Interface</code> is authoritative. <code>MARKET_DATA</code> does <i>not</i> store Chainlink rounds.
Agent action execution log	Off-chain (append-only DB)	Immutable audit trail of every agent write-tool call, linked to on-chain tx hash; not stored on-chain to save gas.
Session key material	Off-chain (sessions table)	EIP-7702 session key hash, scope JSON, and TTL stored off-chain; the scope restrictions are enforced on-chain by the session key contract at signing time.
Session lineage chains (SESSIONS, message history, compression summaries)	Off-chain (PostgreSQL)	Full transcript history is too large for on-chain storage. The lineage chain enables searchable long-term memory without gas cost. <code>AGENT_ACTION_LOG</code> provides the on-chain audit link via confirmed transaction hashes.

Borrowing limit sync protocol. Borrowing limits are enforced on-chain, but the rolling exposure that feeds those limits is derived from off-chain transaction history. When a `LoanDisbursed` or `LoanRepaid` event arrives, the event listener first records it in `TRANSACTION` and recalculates the borrower's six-month and one-year windows. If the new

figure no longer matches `on_chain_enforced_limit` stored in `BORROWING_LIMIT`, the Express API submits an on-chain update through `LocalBank.updateBorrowingLimit`. Once the chain emits `BorrowingLimitUpdated`, the listener refreshes the projection row and clears any `sync_discrepancy_flag`, keeping the database aligned with the authoritative contract state.

Projection vs. application tables. The database splits cleanly between tables the application writes and tables the event listener maintains as read models. Projection tables—`CREDIT_PASSPORT`, `CREDIT_PASSPORT_HISTORY`, `BORROWING_LIMIT`, and `COLLATERAL_POSITION`—are populated only from chain events; Express.js may read them for dashboards and API responses but cannot insert or update them directly, a rule enforced by PostgreSQL row-level security (Table 4.15). Workflow tables such as `LOAN_REQUEST`, `CHAT_MESSAGE`, and `PROFILE_SETTING` remain Express-owned. `LOAN_REQUEST` is the one hybrid case: most columns are application-writable, but `oracle_state` is reserved for the listener, with column-level privileges preventing the API role from modifying it while still allowing the listener to track commit-reveal oracle progress independently of user-facing application state.

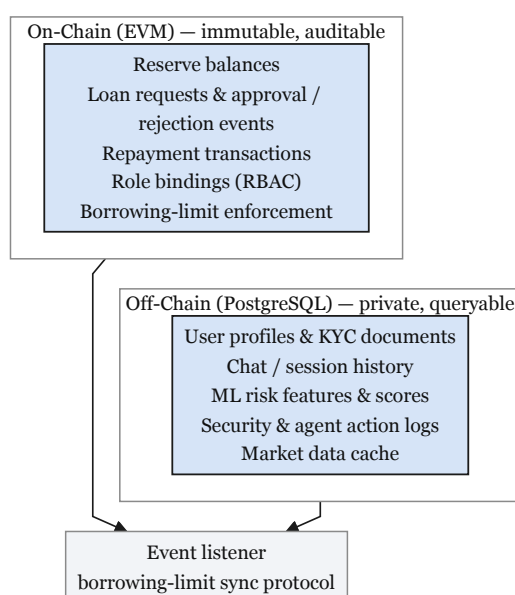


Figure 4.10: On-chain versus off-chain data partitioning

Financial Data Authority and Lifecycle

On-chain contract state is the **authoritative source of truth** for settlement: reserve balances, loan lifecycle transitions, role bindings, and enforced borrowing limits. PostgreSQL holds workflow metadata, PII, ML inference records, and **read-only projections** synchronised from chain events by the event listener (FR-07; Section 4.2.18). Where the two layers disagree after a confirmed transaction, the on-chain record prevails; projection tables exist for fast queries and dashboards, not for independent financial ledgering.

Retail loan write path. The end-to-end flow for a standard retail loan is: (1) the client submits an application (wallet-signed `requestLoan` on `LocalBank`, with optional off-chain

document hash in `LOAN_REQUEST`); (2) the ML service scores features off-chain and the commit-reveal oracle records the risk score on-chain (Section 4.2.3); (3) the Local Bank approver signs `approveLoan`, which atomically updates loan state and disburses funds; (4) the event listener ingests `LoanRequested`, `LoanApproved`, and `LoanDisbursed` events into `BLOCKCHAIN_EVENT_LOG` and updates `LOAN`, `INSTALLMENT`, and projection tables (Figure 4.22). Repayments follow the same pattern: `payInstallment` on-chain, then listener projection.

At MVT scope, loan denomination may use `MockUSDC` or native `ETH` on the demonstration testnet (Table 3.12); the authority rules above are unchanged—only the settlement asset differs.

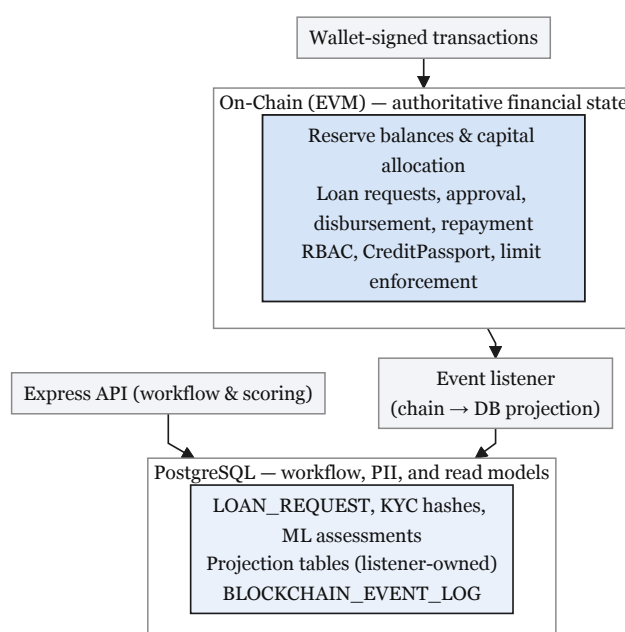


Figure 4.11: Financial data authority and lifecycle: on-chain settlement state, off-chain workflow and projections, and the event-listener bridge.

4.2.6 Operating Context

The Crypto World Bank is specified as a **blockchain-based institutional finance platform** on EVM-compatible infrastructure, not as a geographic pilot for a single rural market. The primary contribution is programmable hierarchical banking—reserve enforcement, tiered capital flow, on-chain auditability, and modular lending products—deployable on a stability-first prototype testnet (Ethereum Sepolia as primary, Polygon Amoy PoS as secondary). Polygon zkEVM remains an evaluated option in the platform comparison tables rather than the default prototype deployment chain narrative.

Three design choices anchor the operating context. First, **stablecoin-denominated retail lending** (Section 1.7) limits liability-side volatility when collateral and loans use different asset types. Second, **low per-transaction gas cost** on zkEVM L2 makes high-frequency installment and reserve operations economically viable relative to Ethereum mainnet. Third, wallet-based onboarding (Section 4.2.7) keeps retail access simple without changing contract role logic. Solidarity group lending (Section 4.2.11) is a programmable product module informed by microfinance literature (BRAC, Grameen); it is not scoped to a specific coun-

try deployment. Institutional onboarding is planned through academic pilots, open-source replication, and regulatory sandbox partnerships (Section 4.2.12).

4.2.7 User Taxonomy and Onboarding Flows

On-chain roles are bound to wallet addresses; KYC document hashes stay off-chain. Tables 4.19–4.21 and Figure 4.12 define actors and permissions. Groth16 zkKYC is Future Work (Section 6.2).

Table 4.19: Operational user taxonomy: KYC level, wallet type, and data residence by user type.

User Type	Tier	KYC Level	Wallet Type	Data Residence
Anonymous Visitor	—	None	None	Session only (Redis)
Registered Retail User	Tier 4	Level 1 (gov. ID + selfie)	MetaMask or optional ERC-4337	Off-chain primary; on-chain role only
Group Client	Tier 4	Level 1 + group verification	ERC-4337 abstracted	Off-chain + on-chain group contract
Local Bank Operator	Tier 3	Full institutional KYC	MetaMask (institutional)	Off-chain + on-chain role binding
Local Bank Approver	Tier 3	Full institutional + background check	MetaMask + Safe co-signer	Off-chain + on-chain role binding
National Bank Admin	Tier 2	Full institutional + regulatory license	Safe multisig mandatory	Off-chain + on-chain role binding
World Bank Admin (Governance)	Tier 1	Founding team + supervisor attestation	Safe 3-of-5 multisig	On-chain governance only

Table 4.20: Formal actor taxonomy (nine actors).

A1a	Retail Client (pre-KYC)	Browsing only; no banking transactions.
A1b	Retail Client (KYC Tier 1)	Bronze/Silver credit tier; small loans up to the KYC-1 cap.
A1c	Retail Client (KYC Tier 2)	Gold/Platinum/Diamond; larger limits, group lending.
A2	Local Bank Admin (Approver)	Loan approver; risk officer; reviews AML alerts.
A3	National Bank Admin	Capital allocator; compliance officer; SAR review; freeze authority.
A4	World Bank Admin (Governance)	Parameter governor; system operator via multi-sig.
A5	AI Agent (retail)	Phase III Must deliverable; read tools always permitted; write tools require explicit human confirmation gate before execution.
<i>Secondary Actors (external systems or authorities)</i>		
A6	Regulatory Authority	Read-only audit access via encrypted data package.
A7	Chainlink DON	Oracle, price feeds, automation.
A8	Blockchain Validator	Polygon zkEVM validity proof generation.
A9	External Auditor	Chainlink PoR verification.

Table 4.21: Actor permission matrix (tabular summary of primary actions).

Action	Client	LB Ad-min	NB Ad-min	WB Ad-min	AI Agent
View own loan status	YES	YES	YES	YES	READ
Submit loan application	YES	NO	NO	NO	WRITE
Pay installment	YES	NO	NO	NO	WRITE
Submit KYC documents	YES	NO	NO	NO	NO
Approve loan application	NO	YES	NO	NO	NO
Reject loan application	NO	YES	NO	NO	NO
Freeze client account	NO	YES	YES	YES	NO
Set borrowing rate	NO	NO	YES	YES	NO
Change reserve ratio	NO	NO	NO	YES	NO
Upgrade LB borrowing cap	NO	NO	YES	YES	NO
View audit logs (all)	NO	YES	YES	YES	NO
Generate SAR report	NO	YES	YES	YES	NO
View reserve summary (public)	YES	YES	YES	YES	READ

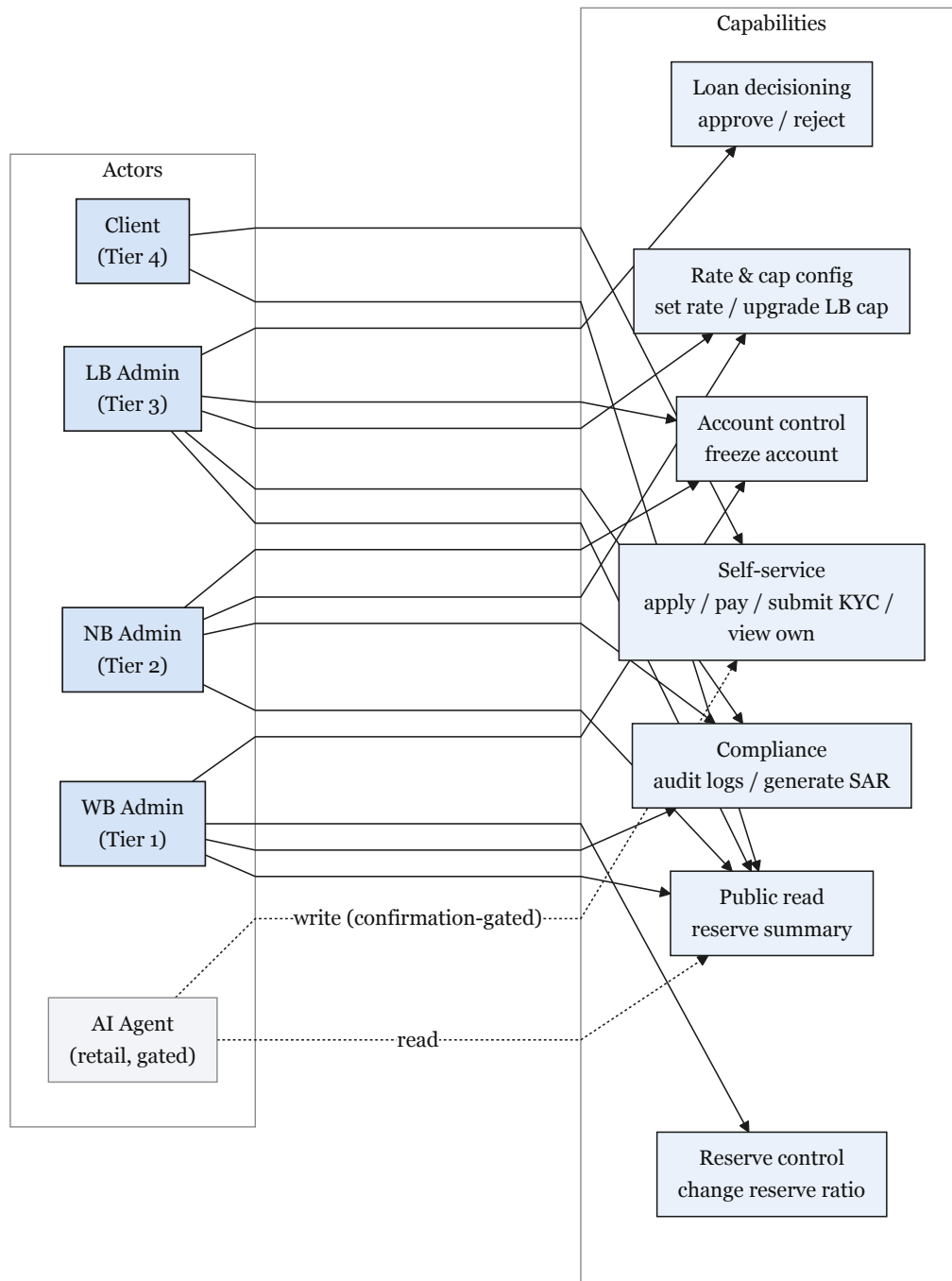


Figure 4.12: Actor permission matrix (primary actions)

4.2.8 Banking Modules

Kinked Interest Rates

Borrowing cost follows a kinked utilization model (Compound/Aave pattern [42]) with optimal utilization $U^* = 80\%$. Below the kink the rate rises gently; above it the rate jumps to incentivise repayment (Formulas 18–19, List of Formulas). Figure 4.13 shows the curve.

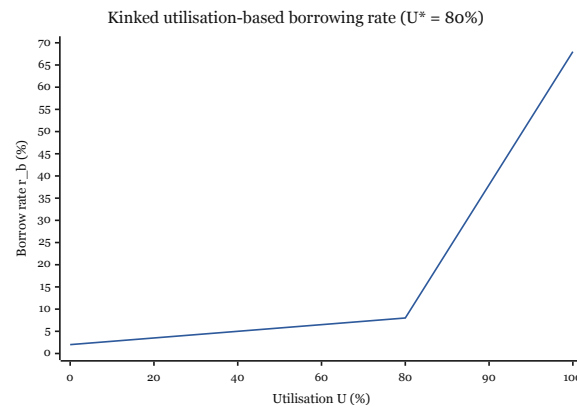


Figure 4.13: Kinked utilization-based borrowing rate

Liquidation Engine

LiquidationEngine monitors health factor (HF) and pays third-party liquidators a 5–8% bonus when $HF < 1.0$. Retail over-collateralised loans use $HF = (C \times LT)/D$; group pools use pool-level HF; credit-based loans trigger SBT downgrade instead of collateral seizure. Figure 4.14 summarises tier models.

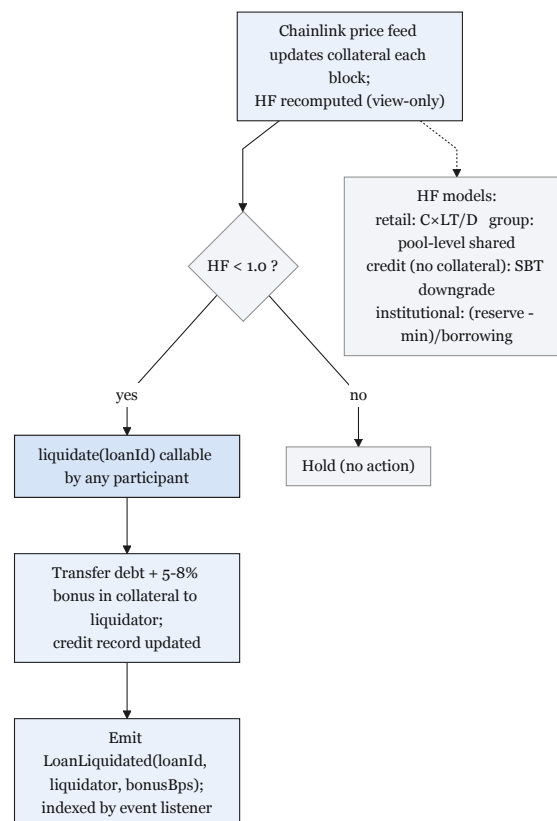


Figure 4.14: LiquidationEngine tier models and lifecycle

Savings and Fixed Deposits

SavingsVault accepts USDC deposits with utilization-linked yield (ERC-4626). FixedDeposit locks principal for a fixed term (ERC-7540). Interest splits among

depositor yield, InsuranceFund, and protocol revenue (Formula NetInterest).

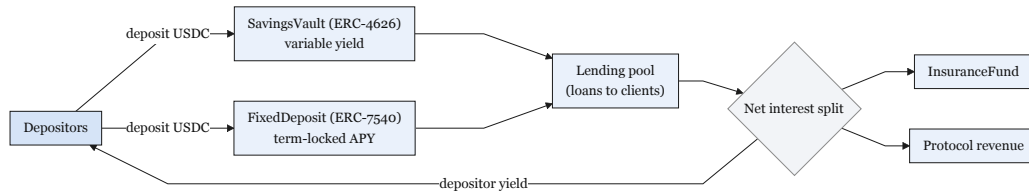


Figure 4.15: SavingsVault and FixedDeposit closed loop

Credit Passport (SBT)

Each KYC-verified client receives a non-transferable SBT [94] encoding repayment history (creditScore, riskTier). Table 4.22 governs loan limits by tier; the token upgrades on full repayment and downgrades on default.

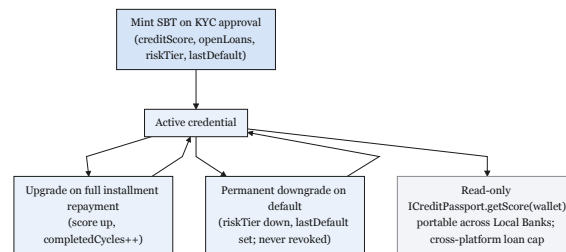


Figure 4.16: On-chain Credit Passport SBT lifecycle

Table 4.22: Credit Passport tier schedule governing maximum loan limits and interest modifiers.

Tier	Label	Score Range	Max (USDC)	Loan	Interest Modifier
1	Bronze	0–299	50		Base rate
2	Silver	300–549	250		Base – 0.5%
3	Gold	550–749	1,000		Base – 1.0%
4	Platinum	750–899	5,000		Base – 1.5%
5	Diamond	900–1000	25,000		Base – 2.0%

4.2.9 Multi-Entity and Cross-Tier Capital Operations

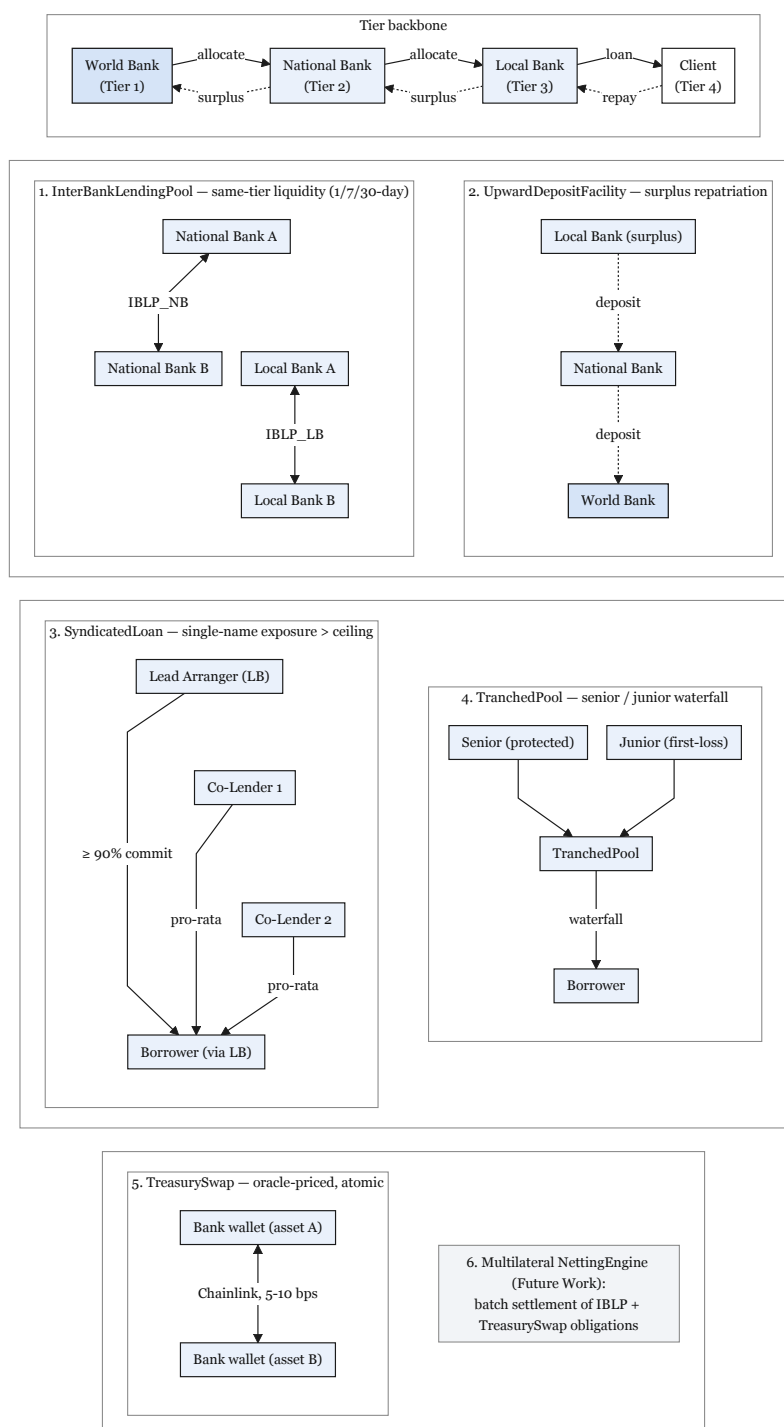


Figure 4.17: Multi-entity and cross-tier capital operations

The architecture framing of multi-directional flows promises downward, same-tier, and upward lending plus bank-level syndication for large exposures. Earlier specifications left same-tier and upward flows at design-level only. A paper that claims a *complete* banking system must specify these flows—and the genuinely multi-entity operations they enable—at the same depth as the downward flow. This section closes that gap with five concrete contract-and-flow specifications: a fully-specified InterBankLendingPool (Section 4.2.9), up-

ward surplus repatriation (Section 4.2.9), syndicated / club lending (Section 4.2.9), senior–junior tranching lending pools (Section 4.2.9), and cross-tier treasury FX swap (Section 4.2.9). Multilateral settlement netting (NettingEngine) is scoped to Future Work (Section 6.2). The corresponding ERD entities, contract list, implementation phase plan, and Future Work are updated downstream so the database design, software-engineering stages, and architecture remain a single consistent system.

InterBankLendingPool: Fully Specified Same-Tier Lending

One InterBankLendingPool (IBLP) per tier (IBLP_NB, IBLP_LB) supplies same-tier short-tenor (1/7/30-day) USDC liquidity among peer banks, gated by active tier role (Section 4.2.13).

Rate model. $r_{IB}(U)$ follows the kinked curve of Section 4.2.8 (Formulas 18–19) with kink $U_{IB}^* = 0.90$. The interbank rate is capped by the parent-tier downward rate:

$$r_{IB}(U) \leq r_{\text{downward}}(U) - \delta \quad (4.1)$$

δ is the governance-set spread that blocks arbitrage against the tier above; checked live via `getCurrentBorrowRate()` at each rate update.

Default cascade. On maturity default: (1) reserve buffer debited, (2) InsuranceFund drawn, (3) parent tier absorbs residual and queues role suspension via TimeLock.

Upward Surplus Repatriation

Banks with reserves above the minimum by 20% may deposit excess into the parent tier via UpwardDepositFacility, earning parent deposit yield. The upward rate is capped below the downward rate:

$$r_{\text{up}} < r_{\text{down}} - \delta \quad (4.2)$$

Withdrawals revert if they breach the depositor's minimum reserve ratio; daily withdrawals are capped at 20% of outstanding upward liabilities; net upward exposure is capped at 30% of on-chain assets.

Syndicated and Club Lending

When single-name exposure exceeds a bank's lending ceiling, multiple banks co-fund one loan through SyndicatedLoan: Lead Arranger, Co-Lenders, and Borrower (client via Local Bank only). Funding requires aggregate commitments $\geq 90\%$ of `totalAmount` and Co-Lender confirmation $\geq 75\%$ by capital share. Each lender's share is recorded as `shareBps[lender][syndicateId]`; principal and interest (and default recovery) distribute pro-rata by that share.

Senior–Junior Tranching Lending Pools

TranchingPool splits capital into senior (lower yield, protected) and junior (higher yield, first-loss) tranches. Repayment waterfall: senior interest $\rightarrow$ junior interest $\rightarrow$ senior principal $\rightarrow$ junior principal; losses reverse (junior written down first). Subordination ratio (junior/senior) is fixed at deposit between 10/90 and 30/70.

Cross-Tier Treasury FX Swap

TreasurySwap executes oracle-priced atomic cross-asset swaps for banking-role wallets:

$$\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread}) \quad (4.3)$$

P is the Chainlink feed price; treasury spread is 5–10 bps. Post-swap reserve ratio must remain above the tier minimum; swaps above \$1M require Safe two-signature confirmation.

Multilateral Settlement Netting Engine (Future Work)

Future Work. A NettingEngine contract to compress bilateral IBLP and TreasurySwap obligations into batch settlement—including partial settlement (settlePartial) and challenge-period dispute resolution—is listed in Section 6.2. ERD stubs (NETTING_BATCH, NETTING_ENTRY) remain in the schema for forward compatibility but are not specified at implementation depth in this report.

Database, Phase, and Contract Consistency for Multi-Entity Operations

Eight ERD entities (INTERBANK_LOAN, UPWARD_DEPOSIT, SYNDICATE, SYNDICATE_MEMBER, TRANCHED_POOL, TREASURY_SWAP, NETTING_BATCH, NETTING_ENTRY) extend Figure 4.9, together with GROUP_CONSENT (Phase II, required before GroupLendingPool) and TRANCHED_POOL_SUBSCRIPTION (Phase II). All institution-participant FKs anchor to INSTITUTION.institution_id (Table 4.11). Five contracts are specified at implementation depth in this report: InterBankLendingPool, UpwardDepositFacility, SyndicatedLoan, TranchedPool, and TreasurySwap; NettingEngine is Future Work (Section 6.2). Extended entities include planned on_chain_address and last_synced_block columns for event-listener mapping.

Table 4.23: FK anchors connecting extended entities to core schema.

Entity	Foreign Keys into Core Schema
SAVINGS_ACCOUNT, FIXED_DEPOSIT, CURRENT_AC- COUNT	borrower_id → BORROWER; local_bank_id → LOCAL_BANK(institution_id)
LOAN_GROUP	local_bank_id → LOCAL_BANK(institution_id)
GROUP_MEMBER	group_id → LOAN_GROUP; borrower_id → BORROWER
INSURANCE_FUND	local_bank_id → LOCAL_BANK(institution_id)
INTERBANK_LOAN	lender_institution_id, borrower_ institution_id → INSTITUTION; tier-type CHECK per IBLP instance
UPWARD_DEPOSIT	depositor_institution_id, parent_ institution_id → INSTITUTION; adjacent-tier CHECK
SYNDICATE	lead_institution_id → INSTITUTION; borrower_id → BORROWER
SYNDICATE_MEM- BER	syndicate_id → SYNDICATE; member_ institution_id → INSTITUTION
TRANCHED_POOL	sponsor_institution_id → INSTITUTION
TRANCHED_POOL_ SUBSCRIPTION	pool_id → TRANCHED_POOL; subscriber_id → BANK_USER
TREASURY_SWAP	initiator_institution_id → INSTITUTION; from_asset_id, to_asset_id → ASSETS
NETTING_BATCH	coordinator_id → BANK_USER
NETTING_ENTRY	batch_id → NETTING_BATCH; participant_ institution_id → INSTITUTION
INSTITUTION_MES- SAGE	sender_institution_id, receiver_ institution_id → INSTITUTION
LOAN_RISK_ASSESS- MENT	loan_request_id → LOAN_REQUEST; model_id → MODEL_REGISTRY
GROUP_CONSENT	loan_request_id → LOAN_REQUEST; member_id → GROUP_MEMBER
COLLATERAL_POSI- TION	borrower_id → BORROWER; loan_id → LOAN; asset_id → ASSETS
BLOCKCHAIN_ EVENT_LOG	Referenced by: LOAN_REQUEST.submission_ event_log_id, approval_event_log_ id; LOAN.disbursement_event_log_id; INSTALLMENT.payment_event_log_id

4.2.10 System Modeling

This section presents the system analysis diagrams that model the platform's structure, behavior, and data flow. Loan state transitions are formalised in Section 4.2.3; the diagrams below show actor interactions and operational flows at the use-case and sequence level.

Use Case Diagram

The system involves nine actors across eleven representative use cases (Figure 4.18). Four **core primary** actors initiate banking operations: Retail Client (A1, three KYC sub-types), Local Bank Admin (A2), National Bank Admin (A3), and World Bank Admin (A4). A5 (AI Agent) is a **planned Phase III primary** actor with restricted write permissions gated by the human confirmation protocol (Table 3.3, item 11). Four secondary actors are external systems or authorities: Regulatory Authority (A6), Chainlink DON (A7), Blockchain Validator (A8), and External Auditor (A9). Note that the primary actor classes in the use-case diagram are a condensation of the formal nine-actor taxonomy defined in Table 4.20: Visitor and Registered Retail User are pre-conditions for the CLIENT (BORROWER) role rather than independent diagram actors, and Local Bank Operator / Approver are merged into the Bank Approver role at the diagram level.

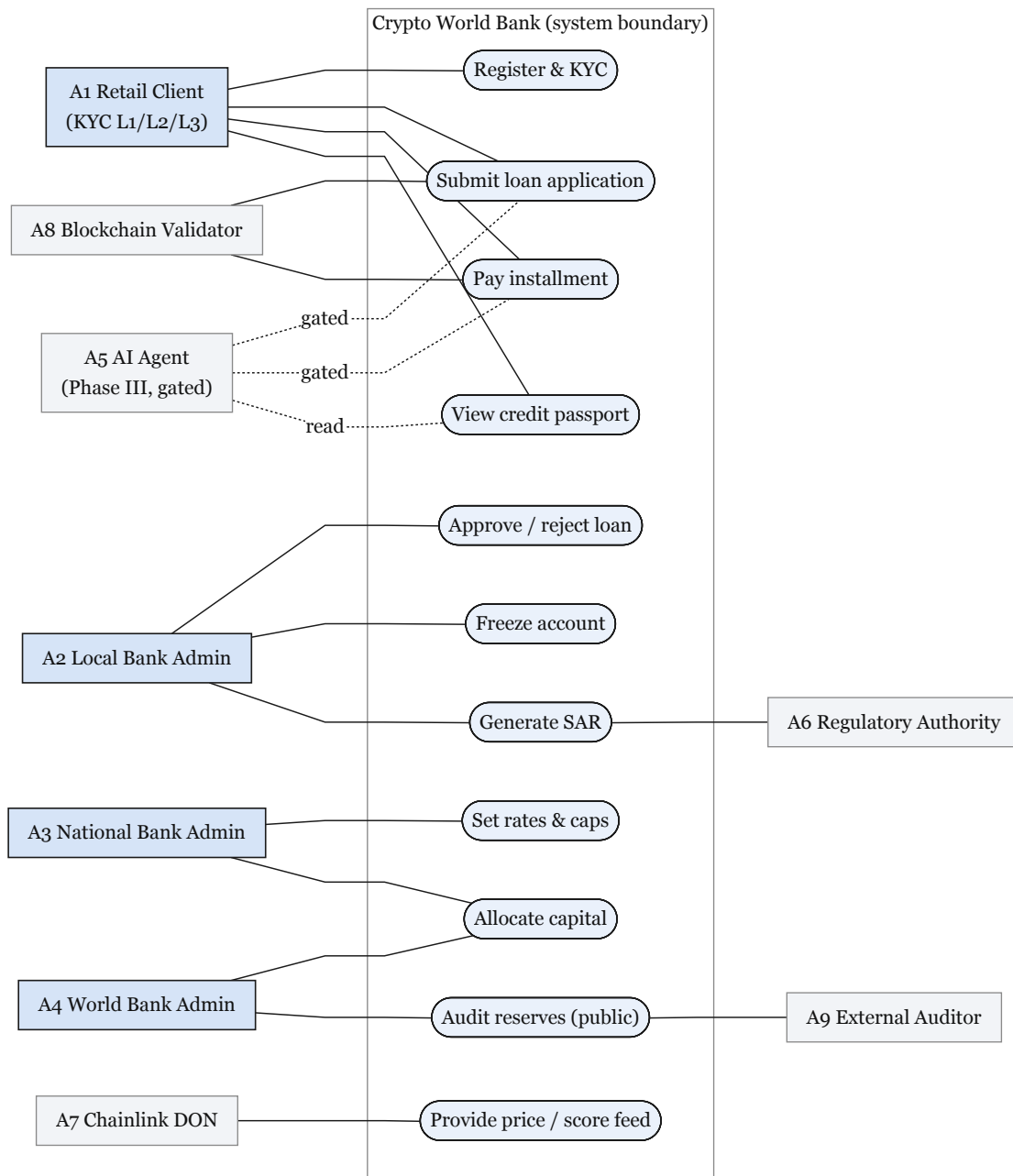


Figure 4.18: Use-case diagram (nine-actor taxonomy)

Activity Diagrams

Figures 4.19–4.21 present three end-to-end activity flows. Each diagram has a single start and a single end node; decision diamonds route exceptional paths (limit exceeded, rejection, verification retry) back to the terminal state without spawning parallel lifelines.

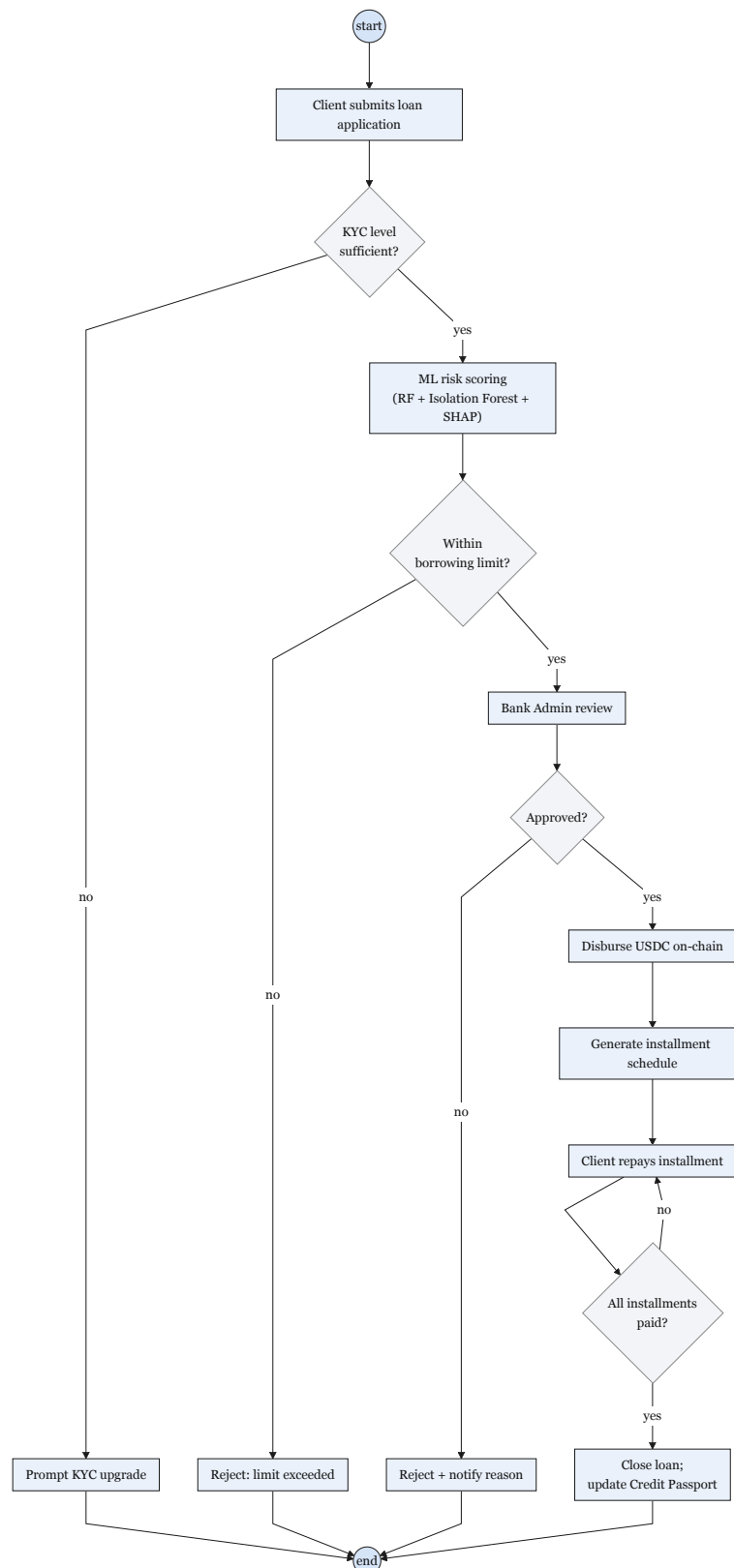


Figure 4.19: USDC loan lifecycle activity flow

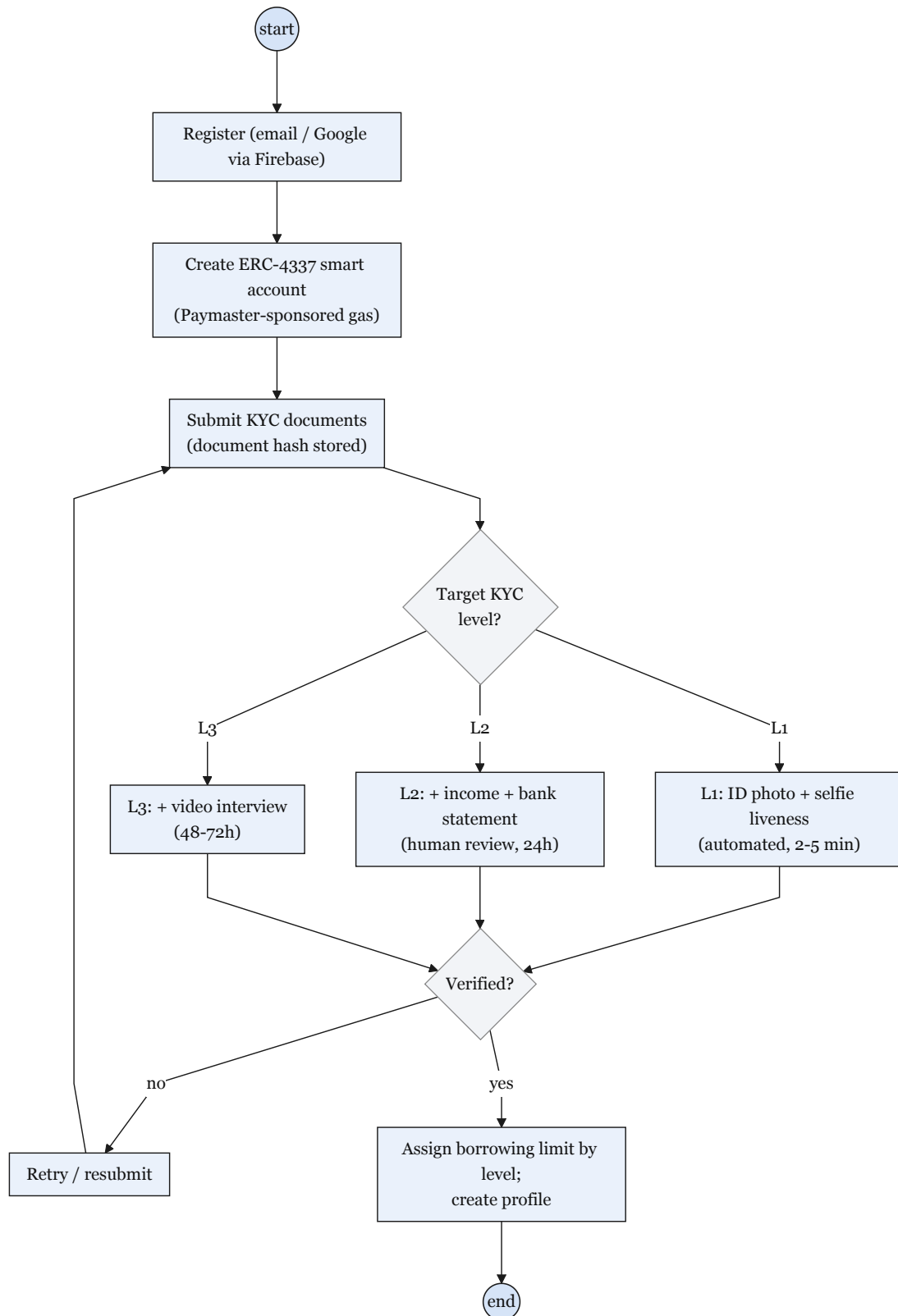


Figure 4.20: Retail onboarding and identity activity flow

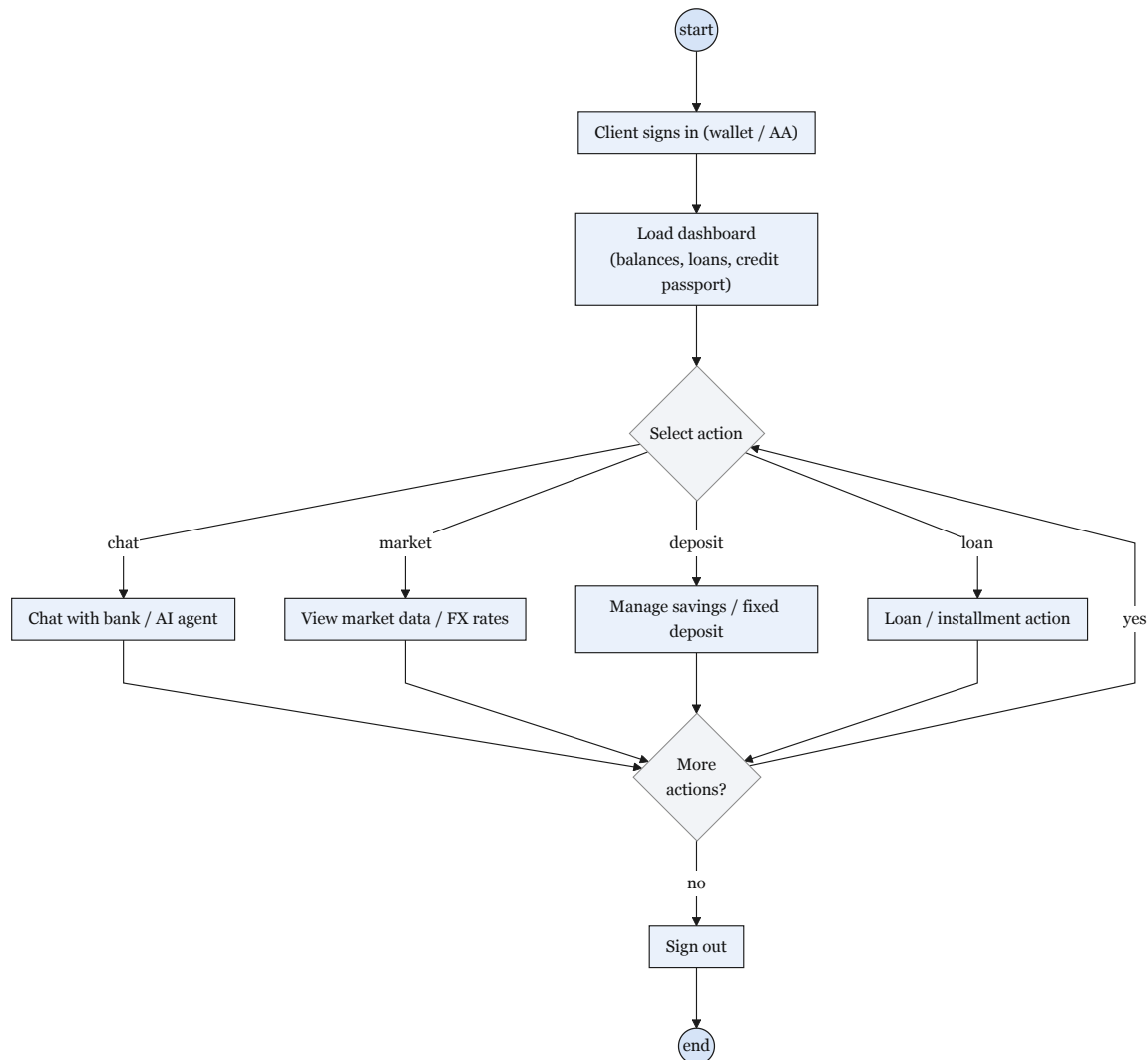


Figure 4.21: Client banking session activity flow

SAR activity flow. The Suspicious Activity Report (SAR) workflow is triggered when the Isolation Forest anomaly detector (Section 4.2.14) flags a wallet with an anomaly score above the 0.75 threshold. The five-step flow is:

1. Isolation Forest flags the wallet: `anomaly_score > 0.75`.
2. `LOAN_RISK_ASSESSMENT` records the per-loan detection (`FK model_id → MODEL_REGISTRY`); `SECURITY_EVENT_LOG` records the system-wide AML alert event.
3. Express.js backend emits a planned event-bus message (`aml-alert` topic); the compliance dashboard consumes it.
4. Bank officer reviews the alert in the admin compliance queue:
 - **False positive:** Dismiss and document reason (audit trail in `audit_logs`).
 - **Confirmed:** Generate SAR record in `audit_logs`; insert `INSTITUTION_MESSAGE` (`message_type='SAR_ESCALATION'`) to notify the tier above (National Bank); freeze wallet via `LocalBank.freezeAccount(clientId)`.
5. The `freezeAccount` function is gated by the `onlyApprover` modifier and is Foundry-tested (Phase III invariant suite). A frozen wallet cannot initiate new loan disbursements or installment payments until the freeze is lifted by the tier above.

Data Flow Diagrams

The core lending data flows are specified in Figures 4.3 and 4.4 (Section 4.2.2). Level-0 defines subsystem boundary flows to the Client, Local Bank Approver, National Bank, ML Risk Scoring Service, and Blockchain Ledger. Level-1 expands Process 0 into submit/validate (1.0), risk scoring with commit–reveal oracle (2.0), approver review (3.0), disbursement (4.0), and installment processing (5.0), reading and writing stores D1–D6 (LOAN_REQUEST, LOAN, INSTALLMENT, LOAN_RISK_ASSESSMENT, BLOCKCHAIN_EVENT_LOG, BORROWING_LIMIT).

Sequence Diagrams

Figures 4.22–4.25 consolidate the platform’s nine interaction flows into four panels: loan approval with reject alternative, installment and income, hierarchical banking with market data and borrowing-limit, and chat with AI-chatbot.

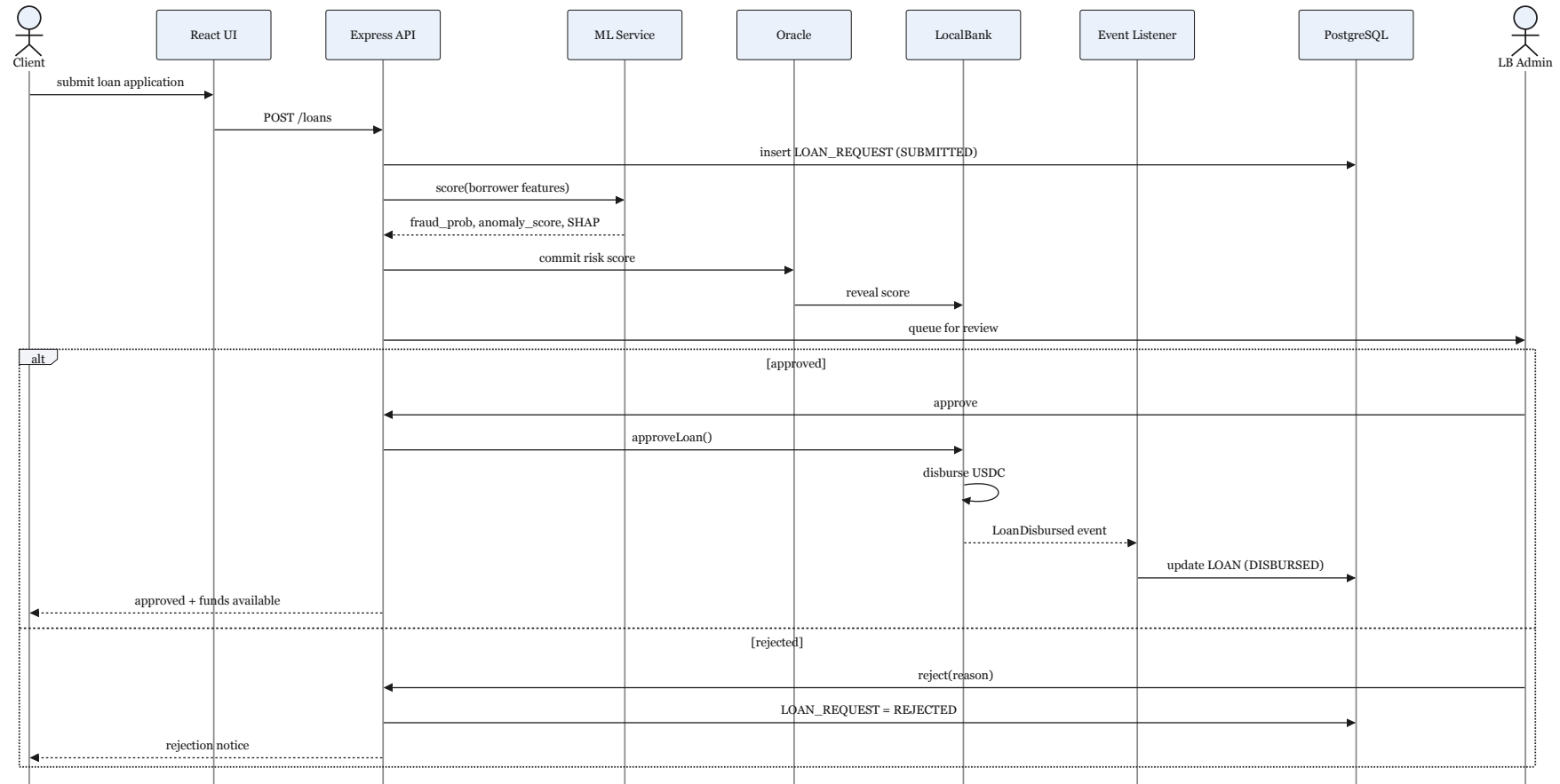


Figure 4.22: Loan approval sequence: application submission, ML scoring, commit–reveal oracle, approver review, and on-chain disbursement or rejection.

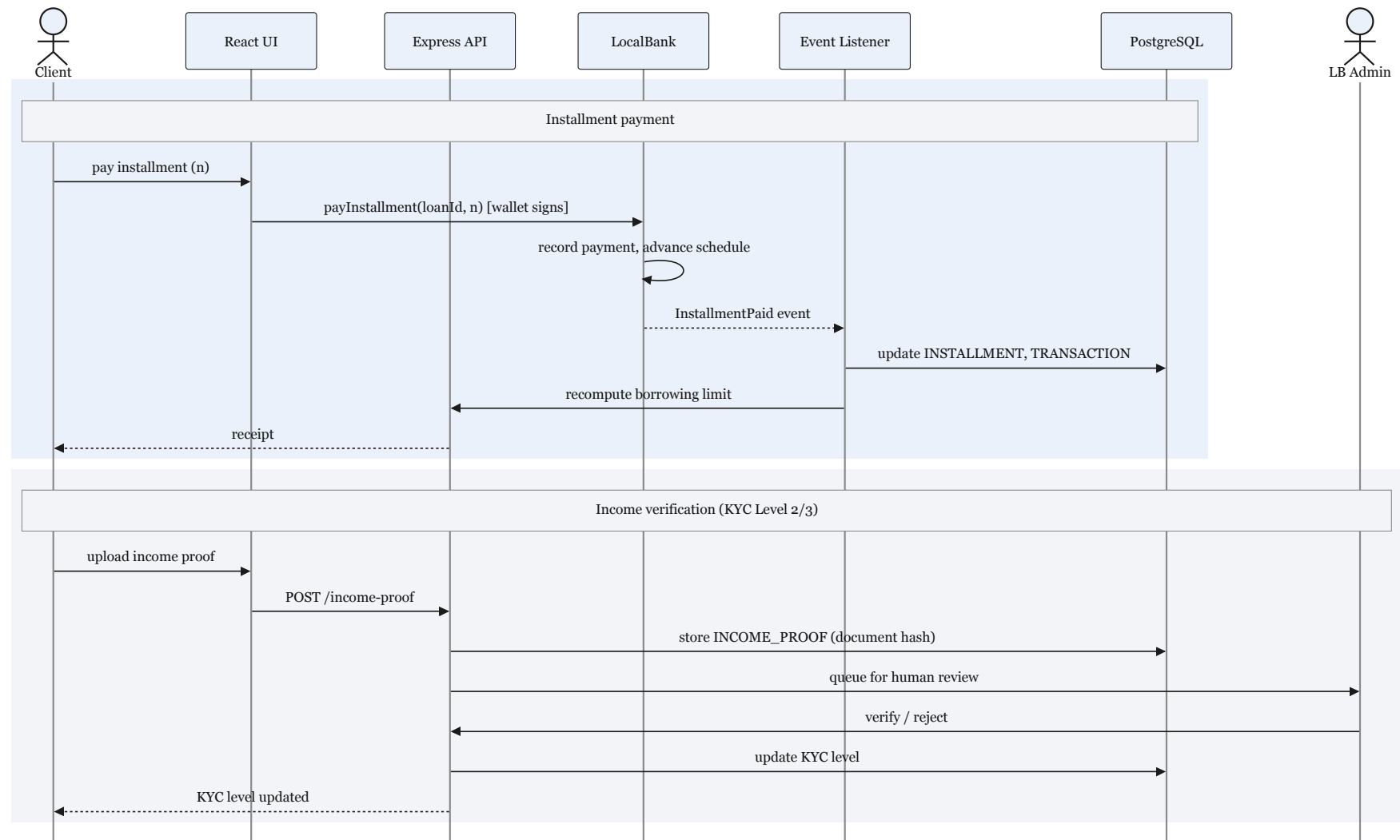


Figure 4.23: Installment payment via wallet-signed `payInstallment` with listener projection, and income-proof upload for KYC Level 2/3 officer review.

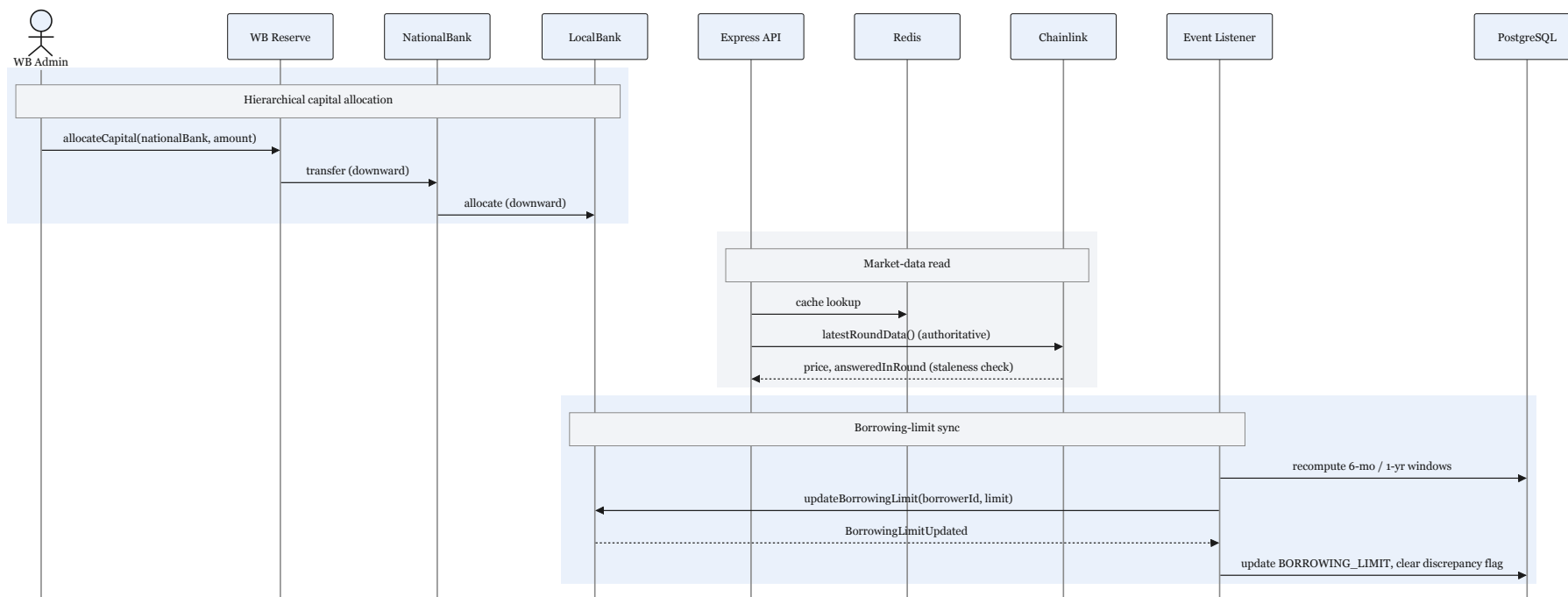


Figure 4.24: Downward capital allocation across tiers, Chainlink price read with Redis cache, and event-driven borrowing-limit synchronisation to PostgreSQL.

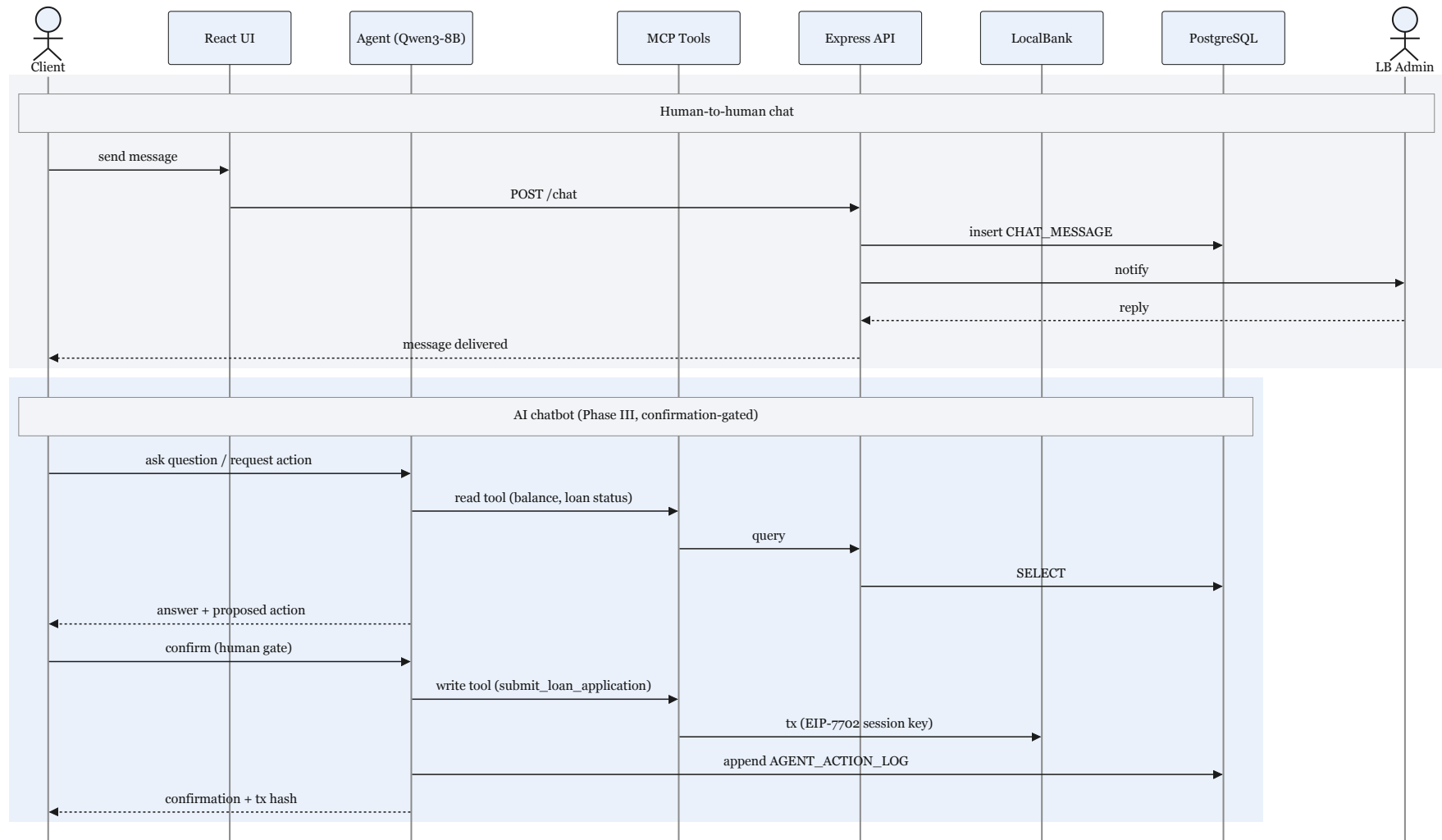


Figure 4.25: Human client–officer messaging through the Express API, and Phase III confirmation-gated conversational agent with MCP read/write tools.

UML Class Diagram

Figure 4.26 is a **Software Engineering** UML class diagram for the *core lending domain*. It is not another system-architecture drawing (those are Figures 4.1 and 4.2). Instead it answers: *which software classes exist, what data they hold, and how they relate*—the same question CSE470 asks when moving from requirements to implementation.

The diagram has three layers of classes:

- **Institution hierarchy** — Institution is an abstract supertype; WorldBankReserve, NationalBank, and LocalBank are its subtypes (generalisation), matching Figure A.1 and the Solidity contracts in contracts/.
- **Retail lending** — Borrower and LoanRequest mirror PostgreSQL tables; LoanController is the on-chain state machine that enforces the oracle gate and disbursement rules.
- **Off-chain services** — LoanService (Express.js) and ScoringService (FastAPI) are controller/service classes that the React UI calls; they delegate to contracts and the ML pipeline without duplicating business logic in the frontend.

Why include it? Use-case and sequence diagrams (above) show *who talks to whom over time*; the class diagram shows *the static object model* that developers implement. Examiners assessing System Analysis and Design and Software Engineering coursework can trace each class back to a requirement in Table 3.1.

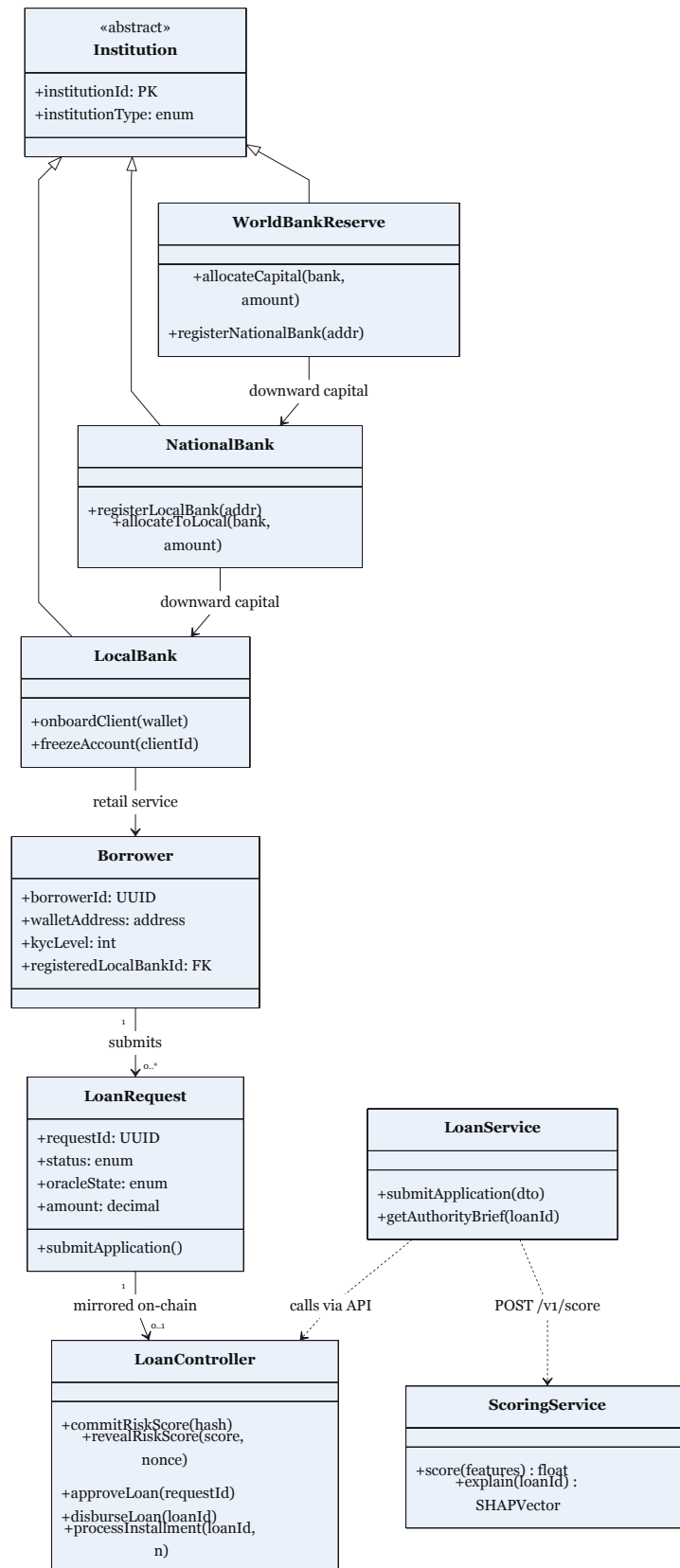


Figure 4.26: UML class diagram (core lending domain): institution hierarchy, loan entities, on-chain controller, and off-chain services.

User Interface Design

The retail and approver interfaces follow a mobile-first Material Design 3 layout implemented in frontend/ (React 18, TypeScript, Tailwind, MUI). Two primary surfaces are specified:

- **Retail loan flow** — wallet connect, KYC status banner, loan application form with plain-language APR and installment preview, and post-submission status tracking.
- **Authority Brief dashboard** — approver view with composite risk score s , SHAP waterfall chart, anomaly deviation table, and approve/decline actions gated on oracle SCORE_REVEALED state.

High-fidelity Figma prototype exports (screen flows, component layout, and approver dashboard) are collected in **Appendix F** (Section C) so the main chapters stay specification-focused; Figure C.1 and Figure C.2 will be inserted there before final submission.

Four-Tier Capital Flow

Figure 4.27 illustrates the hierarchical capital flow and cascading repayment structure.

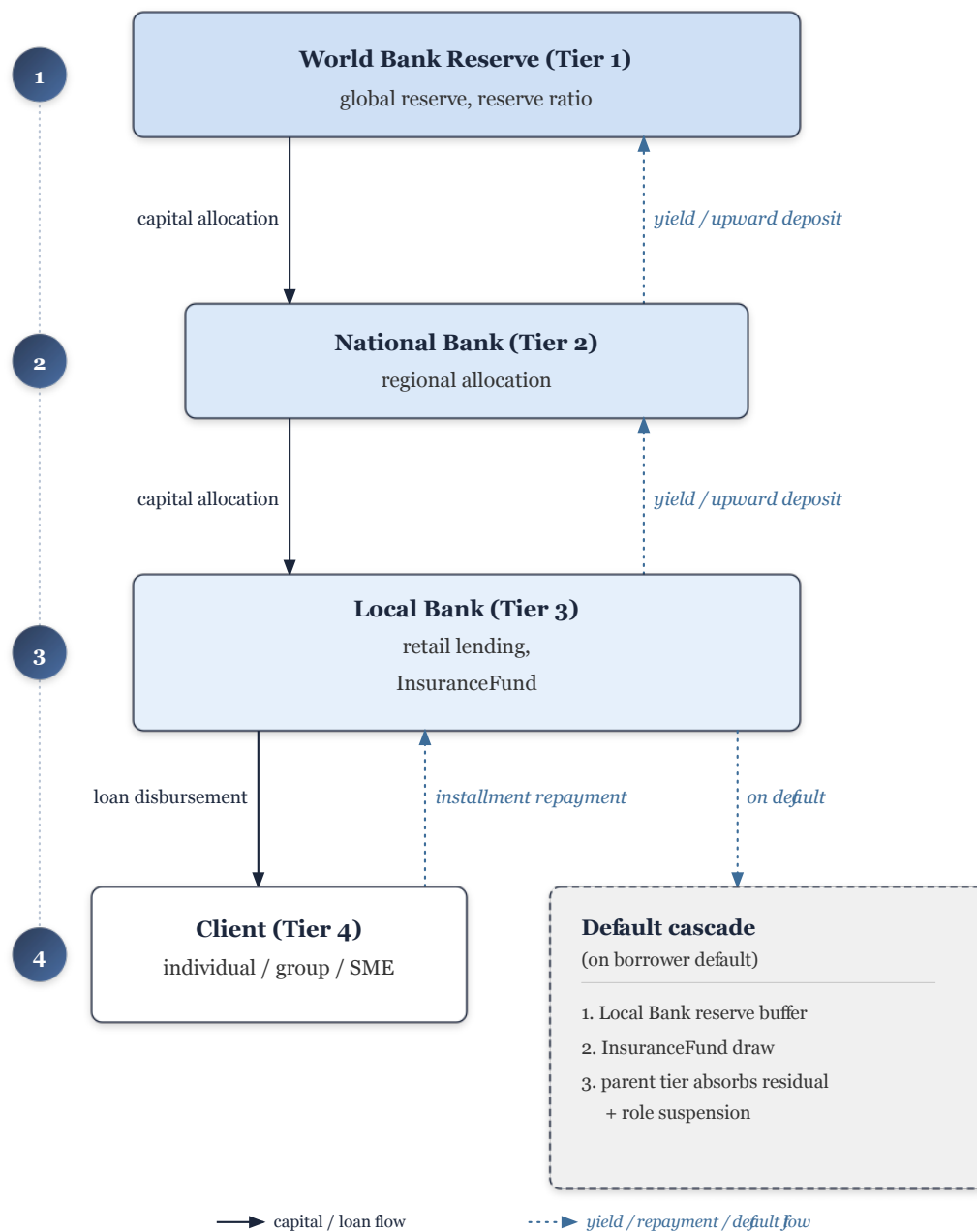


Figure 4.27: Four-tier hierarchical capital flow

4.2.11 Group Solidarity Lending

Groups of 3–20 clients borrow via `GroupLendingPool` with shared collateral. Per-member share is $\text{LoanTotal}/N_{\text{members}}$. Default uses collateral first, then SBT riskTier downgrade [94].

Over-indebtedness risk control. Max two simultaneous group loans, 30-day cooling-off, and cross-bank check via `ICreditPassport.getScore()`. Reject if debt-to-income $\text{DTI} > 0.40$ (Formula in List of Formulas).

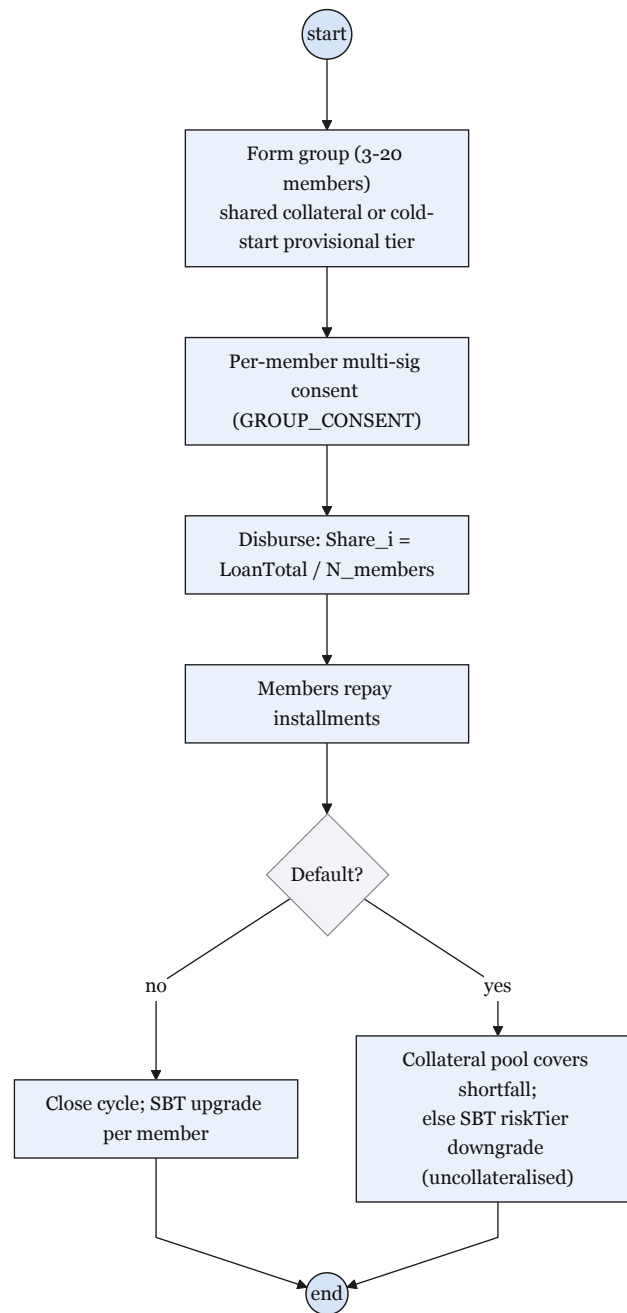


Figure 4.28: Solidarity group lending lifecycle

4.2.12 Governance Framework

Parameter changes route through a 48-hour TimelockController with snapshot voting and 60% quorum. Emergency pause uses a 2-of-3 multisig; production admin keys use Safe multisigs (Table 4.25). Prototype runs on public testnets; SAR workflow: Isolation Forest → compliance queue → freezeAccount (Section 4.2.10).

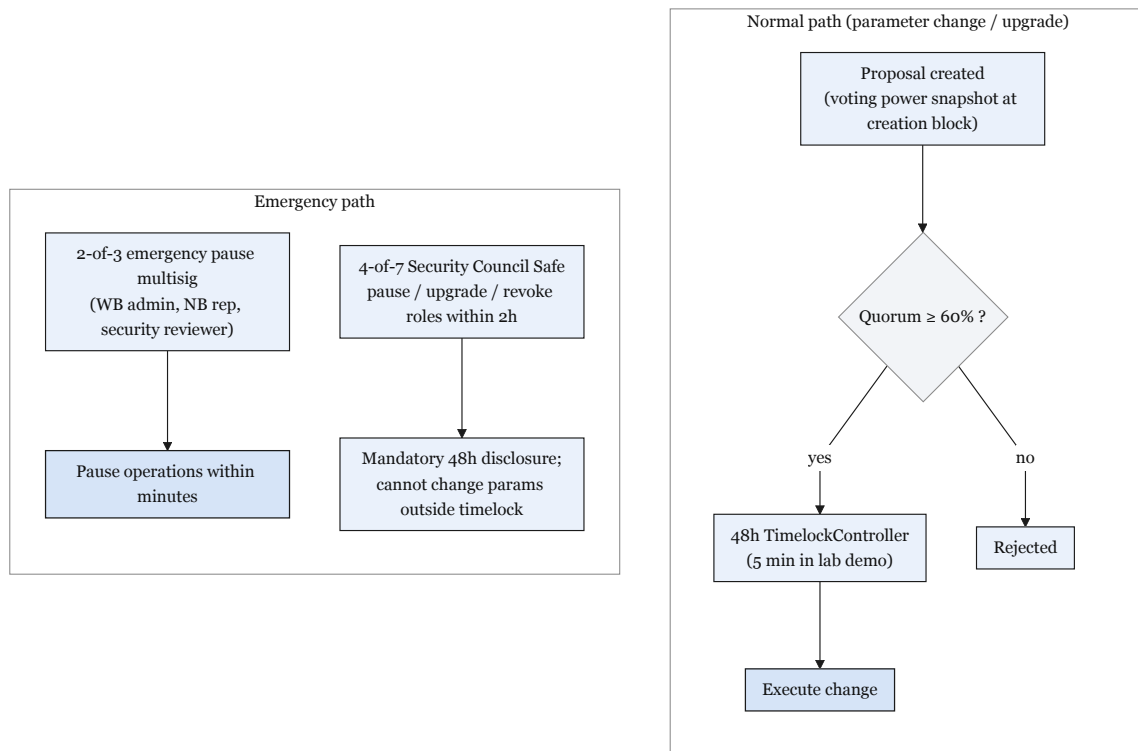


Figure 4.29: Governance dual-path

Table 4.25: Governance framework summary.

Area	Implementation
Network membership	World Bank registers National Banks; National Banks register Local Banks; hierarchical RBAC on-chain
Business operations	Reserve management, loan lifecycle, event notifications; testnet best-effort SLA
Technology	EVM contracts (decentralised); Express API (centralised); event listener syncs PostgreSQL projections
Emergency controls	Per-function pause registry; 2-of-3 emergency multisig; upgrade via UUPS + Safe
Regulatory	Audit logs via contract events; sandbox pilots for production; zkKYC Future Work

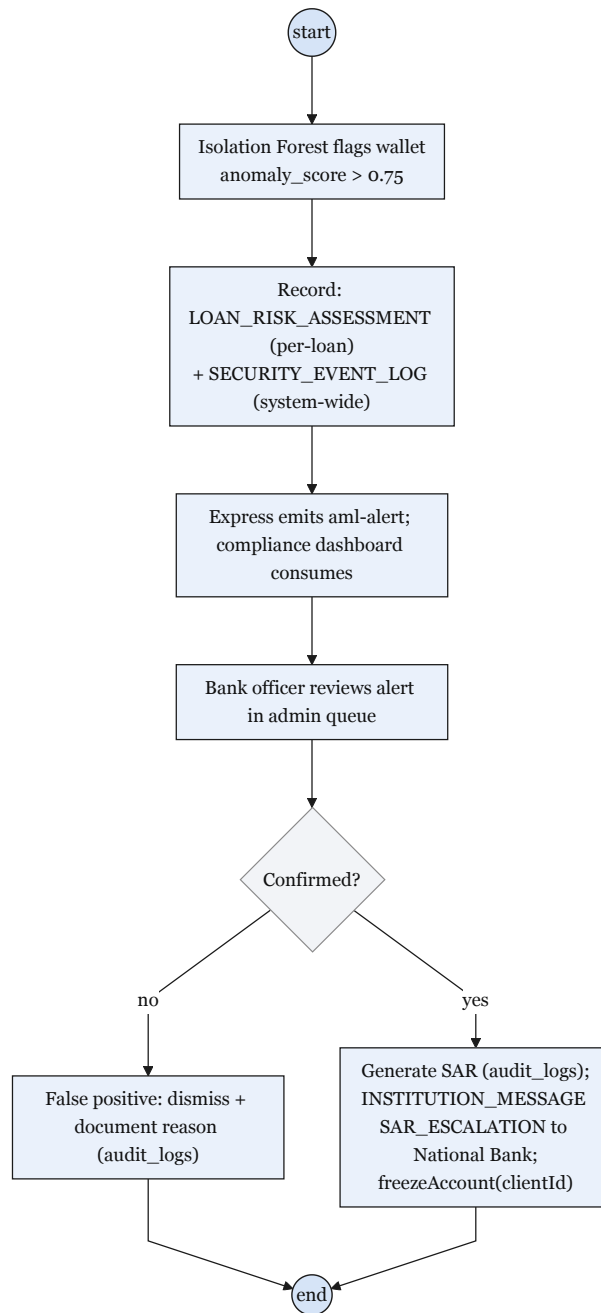


Figure 4.30: SAR and AML compliance workflow

4.2.13 Five-Layer Defense-in-Depth Security Architecture

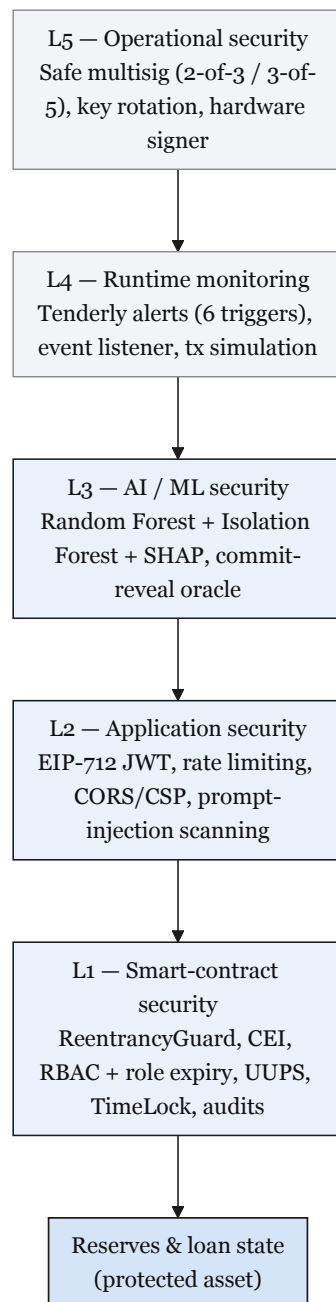


Figure 4.31: Five-layer defense-in-depth security architecture

Security spans five layers (Table 4.26): L5 operational keys (Safe multisigs), L4 runtime monitoring (Tenderly alerts), L3 ML fraud scoring with human review, L2 API auth and rate limits, L1 contract RBAC + ReentrancyGuard + Foundry fuzz tests. Each layer fails closed: if ML is unreachable, loans route to manual review rather than auto-approval.

Table 4.26: Five-layer defense-in-depth model.

Layer	Component	CWB control (MVT)
L5	Operational	Safe 2-of-3 for WorldBankAdmin
L4	Monitoring	Tenderly alerts on large disbursement, low reserve, pause
L3	AI/ML	RF + IF + SHAP; commit–reveal oracle; human approver gate
L2	Application	JWT auth, rate limits, prompt-injection scan on agent input
L1	Smart contract	OpenZeppelin RBAC, CEI, pause, Hardhat + Foundry tests

4.2.14 Threat Model and Security Controls

Table 4.27 maps primary threats to mitigations across the five layers.

Table 4.27: Threat model and mitigations (summary).

Threat	Layer	Vector	Mitigation
Reentrancy	L1	Recursive external call	ReentrancyGuard, CEI, Foundry tests
Oracle manipulation	L1	Bad price feed	Chainlink feeds; mock feed on testnet
Governance capture	L1+L5	Privileged role abuse	Safe multisig, TimeLock, role expiry
JWT / API abuse	L2	Stolen token, DDoS	Short JWT TTL, <code>slowapi</code> limits
ML evasion	L3	Adversarial features	SHAP review, human approver gate
Admin-key theft	L5	Compromised EOA	Safe multisig; no single-key admin

4.2.15 Design Decisions and Alternatives

Table 4.28 demonstrates the chosen stack for frontend, backend, and all dependencies for the stability and completion of the project. The comparison table is shown below:

Table 4.28: Design decisions and alternatives considered.

Decision Area	1st Choice	2nd Choice	Key Criterion
Methodology	Agile / Scrum	Incremental	Evolving scope, milestones
Architecture	DApp + Off-chain AI	Hybrid with Oracle	Gas cost, ML flexibility
Frontend	React + TypeScript	Vue + TypeScript	Web3 ecosystem maturity
Smart Contract	EVM (Solidity)	Solana (Rust)	Gas, control size, free test-nets
Fraud Detection	Random Forest	XGBoost	SHAP compatibility, simplicity
Anomaly Detection	Isolation Forest	Autoencoder	Unsupervised; no labelled data needed
XAI Method	SHAP	LIME	Theoretical guarantees, regulatory fit
Database	PostgreSQL	SQLite	3NF support, async queries
Hosting	Vercel + Render	Localhost only	\$0 cost, publicly accessible URL
Network (primary)	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary)	Polygon zkEVM Cardona	Stability-first prototype vs. ZK-anchored design preference
Oracle mechanism	Commit-reveal relay (MVT)	Chainlink Functions (DON)	Always-available audited relay vs. DON-based upgrade when supported
Agent tool framework	MCP tool server	Direct Express.js API	Defined schema = safety boundary; Phase III Must
Agent session auth	Human confirmation + standard wallet signing	EIP-7702 session keys	MVT relies on explicit consent; scoped session keys are Future Work
Prompt construction	Three-tier assembly	Single ad-hoc string	Phase III Must
Session memory	Lineage model	10-turn window	Phase III Should; cross-session continuity
Tool exposure per turn	Named toolsets	Full 17-tool list	Phase III Should; attack-surface reduction
Write-tool enforcement	Confirmation audit hook	In-pipeline check	Phase III Must; agent write-path

Table 4.30: Design alternative evaluation criteria (selected vs. rejected).

Decision	Selected	Rejected	Criteria (weight)	Outcome
Methodology	Agile + incremental phases	Pure Waterfall	Adaptability (40%), milestone clarity (30%), team size (30%)	Agile wins on evolving scope
Settlement chain	Sepolia / Amoy (demo)	zkEVM Cardona (day-1)	Stability (50%), cost (30%), ZK anchor (20%)	Demo path stability-first
ML delivery	Off-chain FastAPI + commit-reveal	On-chain Chainlink Functions	Latency (35%), cost (35%), auditability (30%)	Off-chain for MVT
Fraud model	Random Forest + SHAP	XGBoost + LIME	Explainability consistency (50%), tabular fit (30%), imbalance (20%)	RF exact TreeExplainer
Database	PostgreSQL 3NF	SQLite	Concurrency (40%), window queries (35%), FK integrity (25%)	PostgreSQL for lending projections

*Justification of Selected Technologies***Table 4.31:** Selected technology justifications (1st choice).

Technology	Justification
Agile/Scrum	Supports evolving contract, UI, and ML scope within short iteration cycles.
DApp + off-chain AI	ML runs off-chain for speed and cost; critical lending decisions remain on-chain.
React + TypeScript	Strong Web3 library support (Wagmi, RainbowKit, Viem, ethers.js).
EVM (Solidity)	Mature security tooling; Sepolia/Amoy testnets; OpenZeppelin libraries.
MUI (Material Design 3)	Ready-made tables, forms, and dashboards for banking UI.
Random Forest	Exact SHAP TreeExplainer; strong on tabular lending data; handles class imbalance.
Isolation Forest	Unsupervised anomaly detection with few labeled fraud samples.
SHAP	Consistent, theory-grounded feature attribution for explainability.
PostgreSQL	ACID compliance, window queries for borrowing limits, JSON for prototyping.
Vercel + Render	Free hosting for live supervisor and examiner demos.
Sepolia / Amoy testnets	Stability-first deployment path within the thesis timeline.
Commit-reveal oracle	Simple MVT path with immutable on-chain ML score audit trail.
MCP tool server	Typed tool schema bounds agent reads and writes.
EIP-7702 session keys	Future Work: scoped session keys for agent on-chain actions.

Table 4.32: Retained alternatives (2nd choice) and why they were not selected.

Alternative	Why retained but not primary
Waterfall	Clear phases, but too rigid for research prototyping.
Hybrid on-chain oracle	Higher latency and cost; external oracle dependency.
Vue + TypeScript	Smaller Web3 ecosystem than React.
Solana (Rust)	Steeper learning curve and weaker audit tooling within 16 weeks.
Tailwind CSS	More custom UI work than MUI for banking layouts.
XGBoost	Approximate SHAP; weaker explanation consistency.
Autoencoder	Needs more data and tuning than Isolation Forest.
LIME	Less stable explanations than SHAP for audit use.
SQLite	Limited concurrency and window-query support.
Localhost-only deploy	Blocks remote supervisor and examiner access.
Polygon Amoy PoS (2nd network)	Fallback testnet if Sepolia is unstable.
Chainlink Functions	Stretch upgrade when supported; higher latency and fees.
Direct Express.js API	Weaker safety boundary than MCP tool schema.
ERC-4337 paymasters	Weaker scope control than EIP-7702 for agent actions.

4.2.16 Planned AI/ML Support and Risk-Score Wiring

The purpose of the AI/ML layer is to provide a risk score per loan approval that will be delivered on-chain through the commit–reveal oracle (Section 4.2.3). The oracle gate design has been shown in Figure 4.32. ML scoring and explainability outputs have been shown in Section 5.3.1, and the rest of the benchmarks and evaluations have been reported in Chapter 5 (Sections 5.1 and 5.3).

Risk Scoring: A sequential step combines Random Forest classifier and Isolation Forest to provide fraud probability and anomaly scores, and these scores are evaluated within $s \in [0, 1]$.

Commit–reveal oracle: This commits a hash of (s, nonce) to LoanController, then each transaction comes with a risk score.

Decision threshold: A minimum and maximum threshold of the value of score (s) is maintained for loan approval decisions: approval consideration ($s < 0.4$), manual review ($0.4 \leq s < 0.7$), or rejection ($s \geq 0.7$).

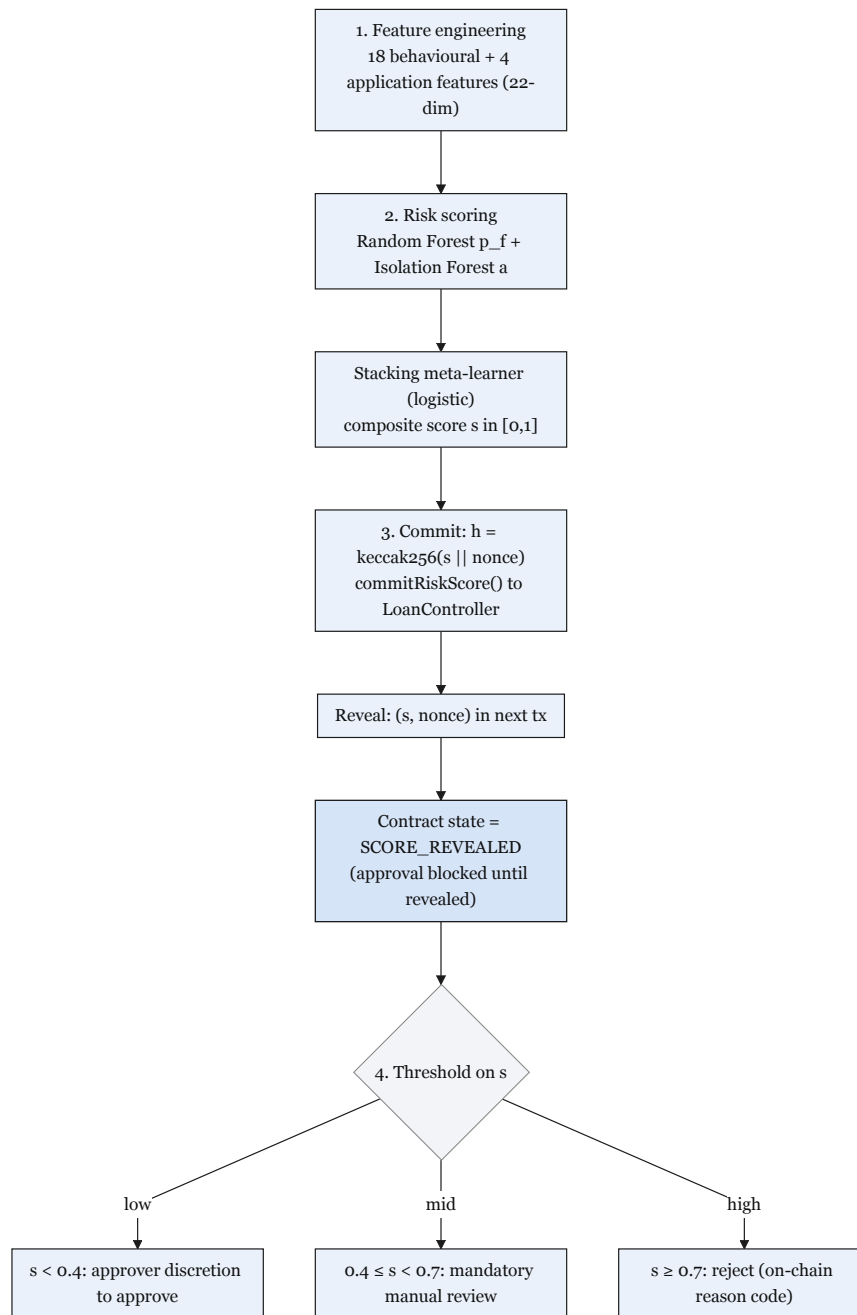


Figure 4.32: ML oracle commit–reveal gate

4.2.17 Conversational Agent (Phase III–IV)

The conversational agent that has been designed to assist the clients to better navigate the platform is planned to be completed by Phase III. It is an auxiliary feature that works alongside the standard web UI for those who may not be familiar with the banking system of the project and have less domain knowledge and want to understand the platform and how to navigate with it. The schema of the full MCP agent has been shown in Table 4.33; explainability for approvers is documented in Section 5.3.1.

Agent six-step pipeline. Clients are expected to interact with the frontend UI like any other typical user-friendly banking website. The agent works as a guiding assistant and its

tool-calling abilities use the same Express.js API endpoints; the same smart contracts are considered for functionalities. This six-step pipeline lets the Qwen3-8B model have access to the live on-chain context injection layer that helps the agent process related instant and consistent information that is relevant to how a client is interacting with the platform. The layers have been described shortly below:

1. **User enters the platform and interacts with the agent:** frontend interaction is recorded and forwarded via the Express.js backend over Server-Sent Events (SSE).
2. **Agent model with live on-chain context:** The Express.js context injection layer provides the client's information to the model (credit score, active installments, loan status, SBT risk tier, pool utilisation). The model receives this information about the user before the conversation prompt by the user, to process relevant information about the client to assist them.
3. **Q&A:** For this part of the conversation between the agent and client, the agent only processes relevant information about the client via RAG (ChromaDB) and maintains formal conversation with the client, and does not call any tools.
4. **Action request:** After a formal conversation with the client, if the client requests a state-modifying action such as requesting a loan, the agent will choose and prepare MCP write tools, assemble the full parameter set from the injected context, and show a concise summary of action on behalf of the client. Ex: *"You are requesting a loan of 150 USDC for 6 months at 8.5% APR. Your monthly installment would be 26.25 USDC. Shall I proceed?"*
5. **Human confirmation gate:** The agent will wait until a confirmation of affirmation for the action that has been prepared to initiate the loan process. The MCP agent writing tools will not be initiated until a positive response has been received from the client side. If a client responds with vague actions three times, the agent does not make the tool calls, pauses the agent service for 10 minutes, and logs this interaction to AGENT_ACTION_LOG.
6. **MCP write tool functionality:** If a successful conversation is established with the client, the agent takes confirmation from the client to initiate the loan request process; with a positive response the agent initiates write tools and accesses the Express.js banking API, then checks for an EIP-7702 session key—if active, it signs the on-chain transaction approval request. The agent gives a confirmation that the request has been forwarded and notifies the client that the loan application has been submitted, shows a reference number, and how long the approval might take.

MCP tool server. For the MVT scope the agent has been designed for 3–5 tools for a minimal typed tool schema. The specified 17 tools have been designed for future work, and some of these tools may be included in this prototype build if they can be delivered in the allocated time, after the MVT scope has been secured.

Table 4.33: MCP tool server (MVT subset). The full 17-tool suite is Future Work.

Tool	Type	Maps to / Description
get_account_state	READ	SBT tier, loan count, savings balance
get_loan_status	READ	Active loans + next installment
get_requirements	READ	Documents + KYC needed for a given loan amount
submit_loan_application	WRITE	POST /api/loan/apply
pay_installment	WRITE	POST /api/installment/pay

In-product assistant: local large language model (LLM) integration (prototype)

Purpose. The in-product assistant is a **planned Phase III Must deliverable** (Section 4.2.17), providing a plain-language interface alongside the React dashboard. It combines a locally hosted, privacy-preserving large language model (Qwen3-8B, Apache 2.0 licence) with a Model Context Protocol (MCP) tool server exposing a minimal MVT subset of **3–5 tools** (Section 4.2.17). The full **17-tool** suite (9 read tools, including a session history search tool, and 8 write tools requiring human confirmation) remains a specified Future Work expansion rather than the graded prototype baseline. The agent can both answer questions—via retrieval-augmented generation (RAG) from policy documents—and execute on-chain banking operations via the MVT write-tool subset, subject to an explicit human confirmation gate before any state-modifying action is taken. Local hosting keeps conversation data on institutional infrastructure, supporting data-sovereignty requirements in regulated pilots. A client who prefers the React UI experiences no reduction in available functionality, since every operation the agent can perform is equally available through the standard dashboard.

Model and runtime (local development). The inference endpoint follows the **OpenAI Chat Completions** contract (`/v1/chat/completions`) with `stream: true`, proxied from the project backend to a local service such as `http://127.0.0.1:1234`. During early development, the assistant used a locally hosted **Gemma-4-E4B** checkpoint (Google mixture-of-experts model, ~26B total / 4B active parameters) via LM Studio in GGUF form (e.g., Q4_K_M quantization) for a laptop-friendly footprint. For the final thesis evaluation, the assistant is replaced by **Qwen3-8B** (Alibaba Cloud Qwen team, released April 28 2025, Apache 2.0 licence) [67], a dense 8.19B-parameter causal language model with a 32K-token native context window (extendable to 131K tokens via YaRN scaling) and support for 100+ languages and dialects. A distinguishing architectural feature of Qwen3-8B is its *switchable reasoning mode*: the model operates in **thinking mode** (chain-of-thought wrapped in `<think>` tokens, suited to multi-step compliance or policy questions) and **non-thinking mode** (direct, low-latency responses for general navigation queries), selectable per-turn with `/think` or `/no_think` in the system prompt. For Q&A and retrieval-augmented generation (RAG) queries, non-thinking mode is the default; thinking mode is activated only for the red-team regulatory-compliance prompts in the evaluation protocol of Section 4.2.17. For agent tool-calling, non-thinking mode is the default for read tools and confirmation summaries; thinking mode is activated for complex multi-step operations such as EMI calculation or group signature coordination. The per-user context namespace (see Section 4.2.16)

is prepended to every system prompt as a structured JSON block, so the model has full account context before reading the user's message. The API model identifier is environment-driven; the integration layer remains identical for both models.

End-to-end behavior. The user interface assembles a short transcript (user/assistant messages) and posts it to the backend streaming route. Before the user message reaches the model, the context assembly layer: (a) selects the active toolset based on intent classification, (b) applies prompt injection scanning to all user-sourced fields, and (c) assembles the three-tier system prompt (Stable + Context + Volatile), injecting the compression summary from the parent session into the Volatile tier if this is a continuation session. The active toolset name and any injection scan flag are recorded in `AGENT_ACTION_LOG` for every turn. The backend then opens a streaming request to the local model server, converts upstream token events into **server-sent events (SSE)** for the browser, and the UI incrementally appends text. The message renderer uses Markdown (including GitHub-flavored extensions), math (KaTeX) for $$$ \dots $$$ blocks, and line-break rules appropriate for chat transcripts.

For write-tool requests, the agent assembles the full parameter set from the user's request and the injected on-chain state, then presents a confirmation summary. Only after the user sends explicit affirmative consent does the agent call the corresponding MCP write tool via the Express.js banking API layer. The EIP-7702 session key (if active) signs the resulting on-chain transaction within its approved scope. Every executed write tool is logged to `agent_action_log` with the confirmation message ID as the audit reference.

Live on-chain context injection. The Express.js backend injects the client's full on-chain state as a structured JSON context block into every system prompt before forwarding the user's message to the model. The per-user context namespace is:

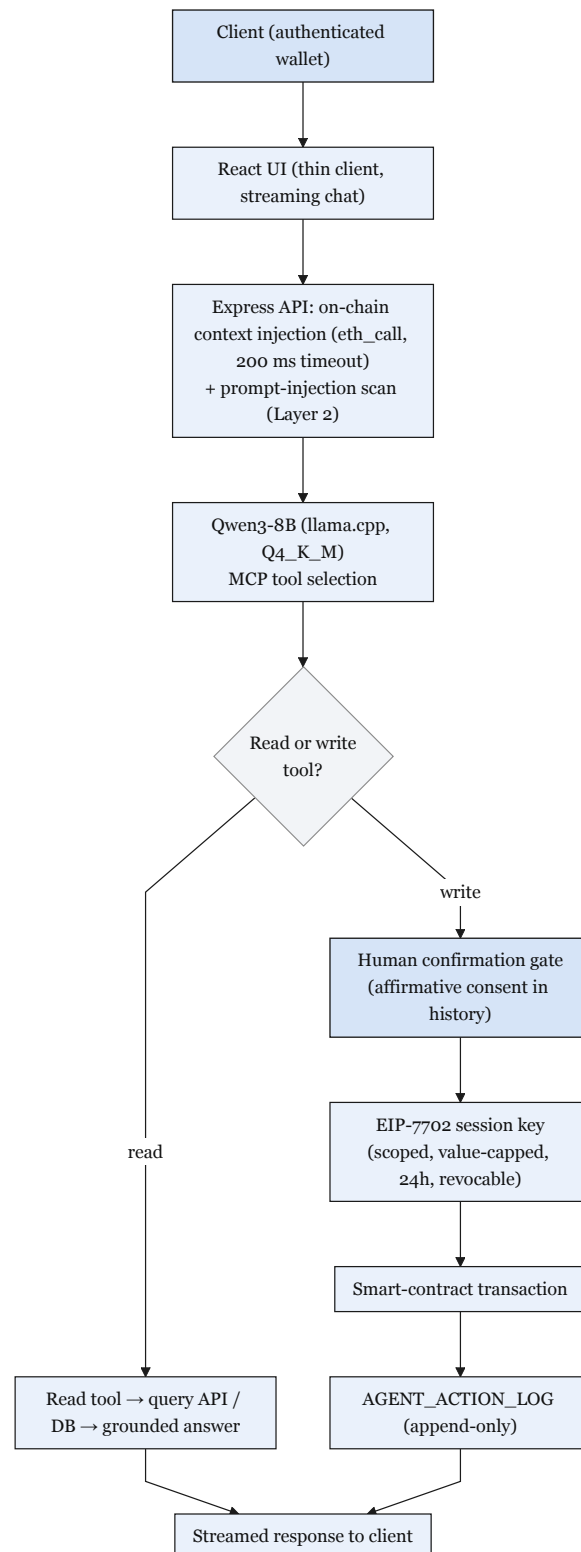
```
{
  "tier": "volatile",
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "session_id": "sess_20260602_xyz",
  "parent_session_id": "sess_20260515_abc",
  "compression_summary": "Client asked about Gold tier eligibility.
```

```
Was told 2 more on-time repayments required. No action taken.",
"conversation_history": [ "...last 10 turns..." ]
}
```

The `compression_summary` field is injected only when this is a child session (i.e., when `parent_session_id` is non-null). It provides the prior-session context without exceeding the Volatile tier token budget. This context block is injected only when the user is authenticated and the `eth_call` round-trip completes within a 200 ms timeout (falling back to a static-context response otherwise). The model uses the injected context to ground its answers in the user's actual account data rather than generic policy text, materially reducing hallucination risk for account-specific queries.

Security and limitations (prototype stance). The agent's safety posture is enforced at four levels. (1) **Tool-schema boundary:** the agent interacts with CWB exclusively through the MVT MCP tool schema (**3–5 tools**) and cannot execute arbitrary code, make unapproved API calls, or read data outside that schema. The full 17-tool suite remains a specified but undeployed Future Work expansion. (2) **Human confirmation gate:** every write tool requires explicit affirmative consent in the conversation history; the confirmation turn ID is logged as the audit reference. (3) **Session key scope:** the EIP-7702 session key is scoped to specific tools, value-capped, time-bound to 24 hours, and revocable at any time. A session that attempts to call an out-of-scope tool is rejected at the key level, not just at the application level. (4) **Prompt injection scanning and lifecycle hook middleware:** user-sourced content is scanned for injection patterns before entering the prompt (Layer 2 control, Section 4.2.16), and every write-tool HTTP POST is intercepted by a middleware stack that enforces the confirmation invariant at the application layer independently of the model's output. Together, these four levels form a defence-in-depth posture in which no single point of failure can circumvent a critical safety constraint. Hallucination risk is bounded by: (a) the injected on-chain state grounding account-specific answers, and (b) the refusal layer for regulatory and legal queries (100% target on the red-team set, Section 4.2.17).

Architecture diagram. Figure 4.33 shows the compact end-to-end request path; Figure 4.34 expands the same flow with component boundaries and authentication context.

**Figure 4.33:** AI banking agent request path

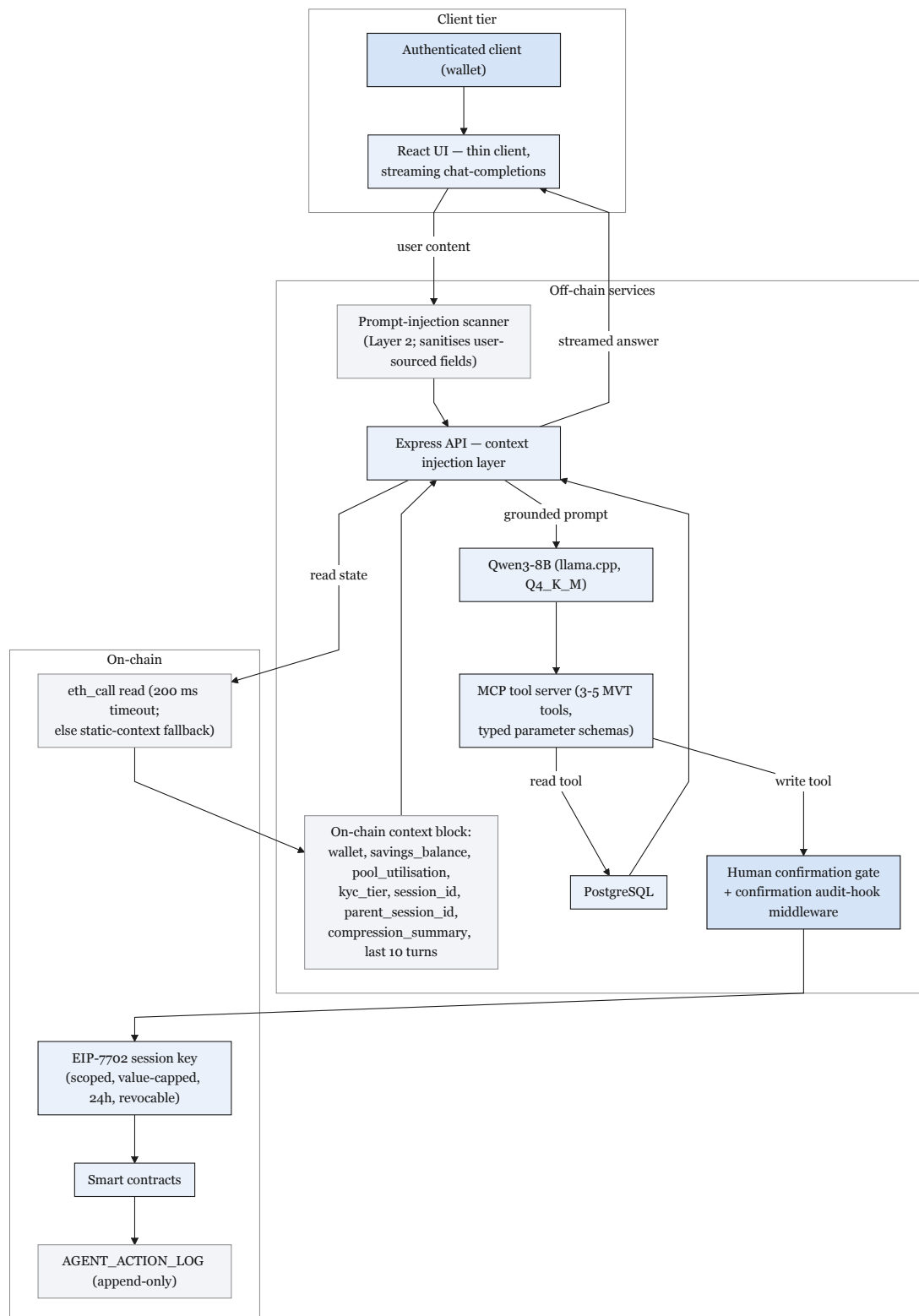


Figure 4.34: AI agent data flow

Table 4.34: Technology stack summary.

Layer	Technology	Version / Notes
Smart Contract	Solidity, OpenZeppelin	0.8.20; Ownable, ReentrancyGuard
Frontend	React, TypeScript	Material UI; Design 3
Wallet Integration	Wagmi, RainbowKit, Viem	EIP-1193 compliant
Build and Test	Hardhat	Automated test suite; deployment scripts
Backend API	Express.js, Node.js	REST API; middleware architecture
Database	PostgreSQL (designed)	51 entities (31 core + 6 extension + 14 stubs), 3NF; relational integrity
Target Network	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary single-chain prototype)	Stable public testnets for delivery; Polygon zkEVM retained as evaluated design preference
AI Agent Engine	Qwen3-8B (llama.cpp, Q4_K_M)	MCP tool server; per-user context namespace; human confirmation gate
MCP Tool Server	Minimal MVT tool subset (3–5 tools); prompt injection scanner; confirmation audit hook middleware	Exposes a small read+write surface to the agent with typed parameter schemas
Session Keys	EIP-7702 (Future Work)	Scoped, time-bound, value-capped agent wallet authorisation
Oracle Network	Commit–reveal (MVT) + Chainlink Functions (stretch)	Commit–reveal enforces auditable score commitment; DON upgrade when supported
Price Feeds	Chainlink AggregatorV3Interface	Fiat/USD pairs (e.g. EUR/USD) and ETH/USD; 8-decimal precision
Automation	Chainlink Automation	Trustless installment due-date checking; replaces centralised cron job
Proof of Reserve	Chainlink PoR	Cryptographically verifiable World-BankReserve balance
Event Indexing	PostgreSQL event listener	Stability-first indexing baseline (GraphQL over The Graph optional later)

4.2.18 Real-Time Dashboard Pipeline and Runtime Monitoring

For stability, lightweight data indexing strategies are used to reduce complexities. After the correct implementation of the core contracts and event schemas, an optional query layer is to be added for GraphQL access for better demonstration of analytics. A Node.js service is utilized for WebSocket event listener via Socket.IO. Tenderly monitoring and MVT-graded time and attempt triggers are added for monitoring and inconsistency checks.

Functional Verification of Design Alternatives

Chapter 5

Evaluation, Analysis, and Final Design

5.1 Performance Evaluation

- **Verification:** Hardhat contract tests (`test/`, 42 cases); Vitest API tests (`backend/tests/`); Foundry invariants (solveny, role segregation, capital-flow direction) in Phase IV.
- **Validation:** Phase gates G1–G4; BCCC fraud benchmarks (Table 5.4, Figure 5.1).

Table 5.1: Implementation evidence (repository root).

Component	Path	Status
Smart contracts	<code>contracts/*.sol</code>	World/National/Local Bank compilable
Contract tests	<code>test/*.test.ts</code>	42 Hardhat cases (four suites)
Backend API	<code>backend/src/</code>	Loans, banks, auth, income routes
API tests	<code>backend/tests/</code>	Vitest auth, banks, loans, chat
Frontend	<code>frontend/src/</code>	React + Wagmi wallet shell
ML pipeline	<code>Documentation/.../ML pipeline - Mac M4/</code>	BCCC subsample; <code>metrics.json</code>
ML service	<code>ml-service/app/</code>	FastAPI <code>/health</code> , <code>/score</code> scaffold
Deploy	<code>scripts/deploy.ts</code>	Sepolia/Amoy/local ready

5.2 Analysis of Design Solutions

Table 5.2: Selected design vs. CO6 evaluation factors.

Factor	Selected	Evidence	Status
Cost	\$0 prototype; L2 gas $\approx$ \$0.011/loan	Tables 3.13, 3.14	Met
Efficiency	Off-chain ML; single-chain	NFR-01; Phase IV gas table	ML trained; live p95 pending
Usability	React + wallet; MCP agent	NFR-05; FR-08	UI shell; agent Phase III
Impact	Tier-transparent flow; group lending	Section 3.2	Specified
Sustainability	PoS testnets; off-chain ML	Section 3.3	Qualitative
Maintainability	Monorepo; OpenZeppelin; G1–G4	Table 5.1	42 tests passing
Performance	RF+IF+stacking; F1 = 0.891	Table 5.4	Benchmark met; E2E Phase II–IV

5.2.1 Requirements Traceability Matrix

Table 5.3: Requirements traceability matrix (excerpt).

Req.	Use case / diagram	Module	Test / evidence	MVT
FR-01	Figure 4.27	WorldBankReserve	hierarchy.test.	#1
FR-02	Figures 4.19, 4.22	LocalBank.sol, loans API	LocalBank.test.t	#2–3
FR-04	Table 4.15	BORROWING_LIMIT + SBT	Foundry invariant (planned)	#5
FR-05	Figure 4.32	ml-service/	Table 5.4	#4, #8
FR-07	Figures 4.3–4.4	Event listener	PostgreSQL tests	DT-I.12–13
FR-08	Figure 4.25	MCP agent	Section 4.2.17	#11–12
NFR-02	Table 4.26	OpenZeppelin RBAC	WorldBankReserve	#13–14

5.2.2 Final Design Adjustments

Multi-chain bridging (Sepolia $\leftrightarrow$ zkEVM) was rejected due to bridge attack surface. **Final design:** single-chain deployment on Sepolia/Amoy; Polygon zkEVM Cardona remains the design-preference target without a live cross-chain bridge. Details in Tables 4.28, 4.30 (Chapter 4).

5.3 Statistical Analysis

5.3.1 Datasets and Benchmark Results

Table 5.4: BCCC-DeFiFraudTrans-2025 subsample benchmark (address-grouped hold-out; evidence/mac-m4/metrics.json). Primary dataset; Elliptic [68] secondary only; on-chain CWB log for future retrain.

Split	Precision	Recall	F1	ROC-AUC	n_{test}
Held-out test	0.874	0.908	0.891	0.991	5,140

TN = 4,621, FP = 60, FN = 42, TP = 417; fraud rate $\approx 8.7\%$; 21 features; leakage column `flag` removed.

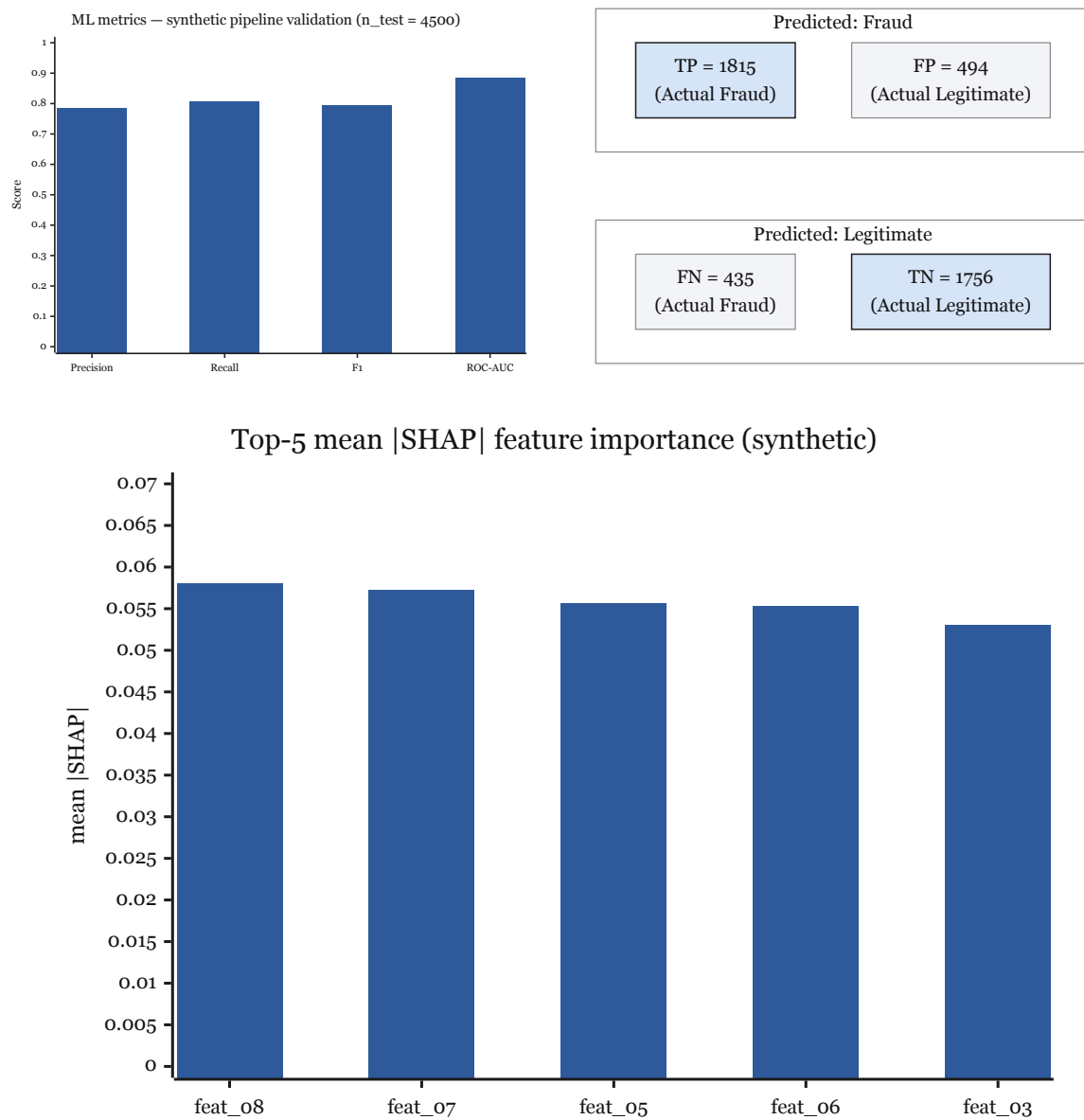


Figure 5.1: ML evaluation (F1 = 0.891, ROC-AUC = 0.991) and SHAP feature importance.

5.4 Comparisons and Relationships

Table 5.5: Selected vs. rejected design choices and linked metrics.

Dimension	CWB (selected)	Rejected / base-	Metric / requirement line
Architecture	Four-tier hierarchy	Flat Aave/Compound	FR-01/FR-03; Table 2.3
Settlement	Single-chain Sepolia/Amoy	Multi-chain bridge	Table 3.14; Section 5.2.2
ML delivery	Off-chain RF+IF+SHAP	On-chain-only scoring	F1 = 0.891; NFR-01
Oracle	Commit-reveal relay	Chainlink Functions DON	Human approver gate (FR-02)
Gas	L2 Cardona (design)	Ethereum mainnet	≈\$0.011 vs \$2.40–\$4.80
Human vs ML	Decision support	Auto-disburse	NFR-02; Section 3.4

5.5 Discussion

Table 5.6: Research limitations and mitigations (summary).

ID	Limitation	Mitigation / scope
G1	Partial DeFi precedents (Maple, Goldfinch)	Claim: combined four-tier + ML + group lending
G2	BCCC flat-pool training ≠ hierarchical CWB flows	Baseline F1 only; CWB retrain after testnet traffic
G3	Pre-thesis evidence = spec + benchmarks	MVT testnet E2E and Foundry suite (Phase IV)
G4	On-chain group lending ≠ social solidarity	No BRAC/Grameen repayment claims
G5	Oracle collusion; model staleness	Commit-reveal MVP; Certora proofs in Future Work
G6	Technical default only	No sovereign preferred-creditor enforcement

Chapter 6

Conclusion

6.1 Conclusion

This paper is a complete specification of all the core sections that will be used in development of the project. It introduces a problem–solution view of how decentralized financing can be made adaptable to the institutional layers of our traditional financing system that is not flat, rather layered. Flat DeFi is hard to be made adaptable among the general mass, so our system provides a tier-based architecture for financial flow, with the added benefit of transparency with programmable smart contracts for the betterment of the client, and less control for the capitalistic organizations. The paper shows credible studies from the past that have contributed to the field before, and uses that evidence and those architectures as proof that a tier-based DeFi system can be built with better security layers that can contribute to the development finance sector globally. The core architecture stack follows mostly a traditional DeFi financial platform build, but the AI/ML layer integration and the tier-based layers are completely novel; no other paper or project shows all four-layer integration before. This project idea is enhanced by the existing systems that have two-tier-based banking systems.

A detailed plan has been introduced for how new clients can be onboarded and attracted to the platform, how clients will interact with the system, how banking authorities can approve loans, and how ML and AI layers can communicate in the middle with both clients and authorities to make this whole process of lending decentralized cryptocurrencies easier and more secure. The potential of such a decentralized system to be popular and adaptable is huge if implemented correctly and if the higher authorities show interest and invest heavily in this idea for the good of humanity. The purpose of this project is to push the idea of decentralized currencies forward for future generations to show more interest, and to support the building blocks for better financing systems in the future.

6.2 Future Work

This section has been specified in the above chapters; it focuses on the potential of the project and the technical abilities and functional capabilities that can be improved and enhanced in the future. First priority is completing the MVT checklist of the development cycle for the defense of the project, and the rest are mostly added stretch to show how many features can be incorporated in the given time. A small summary of the future work:

Deployment on Polygon zkEVM Cardona, which remains experimental for production grade currently; as zkEVM may prove to be more stable, the platform will be deployed on Polygon zkEVM as it is more cost efficient for transactional activities.

Cross-tier features such as InterBankLendingPool and group lending, upward deposit facilities will be improved and made functional. For security and monitoring, multisig admin for World Bank operations and a live reserve transparency dashboard will be integrated.

zkKYC (currently unstable globally), Certora proofs, federated learning, and GraphSAGE (quite complex to implement now) have been specified in the paper and will be more attainable with more time and effort; these have been moved to future work.

This field requires much research, as financial flow is heavily manipulated and misused globally by capitalists; transparency in capital flow is vital for equal access to resources and opportunity to live in harmony. The long-term goal of this project is to evolve this developed system to a more robust level, with more proper and efficient architecture and system design, a regulation-aware system while keeping transparency, equal access, and beneficial to humanity as core design principles.

Appendix A

Database Schema Reference

This appendix records the physical-schema details moved from Chapter 4 to keep the main design chapter focused on conceptual design.

A.1 Full Relational ERD

Figure A.1 is the complete logical data model: 51 documented entities plus two Phase-IV configuration tables (CHAIN_REGISTRY, SESSION_KEY_PERMISSION), with crow's-foot cardinalities, migration-tier colour coding, and a notation key. It is placed here rather than in Chapter 4 because the diagram requires a full landscape page at high magnification to remain legible.

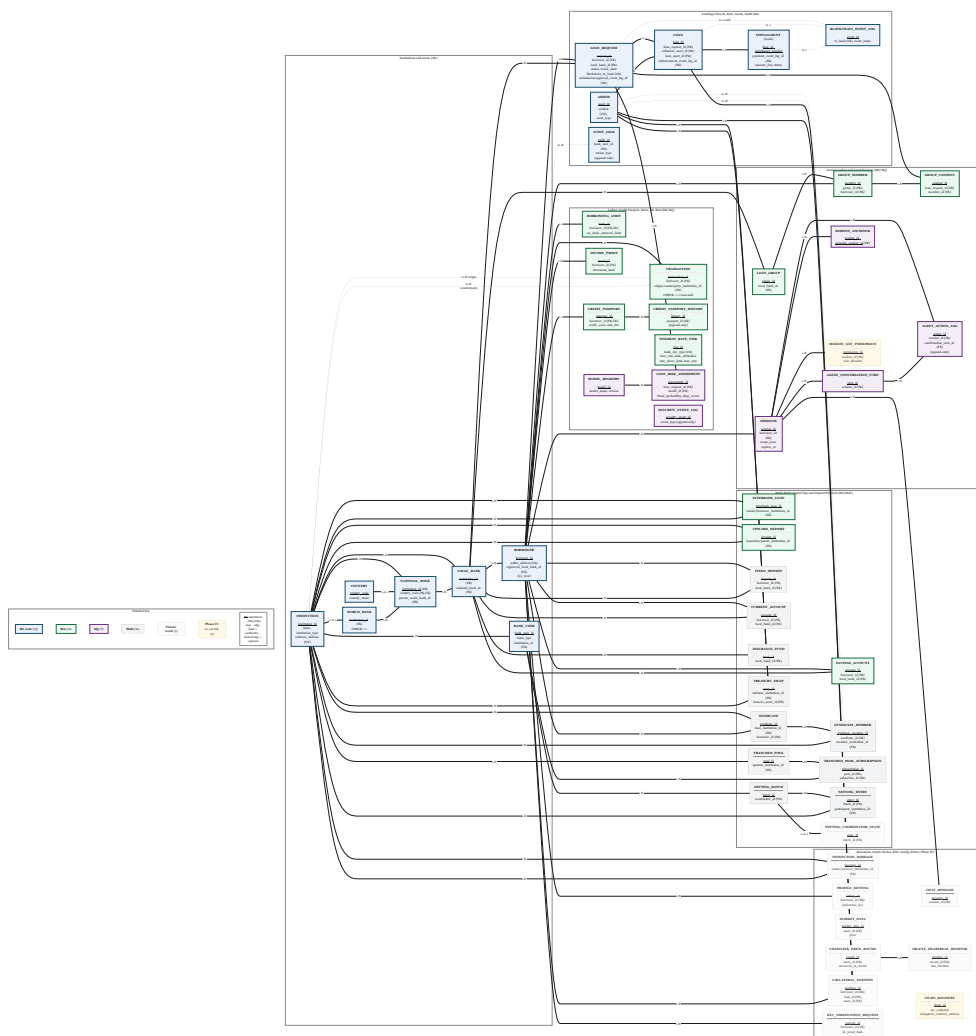


Figure A.1: Full relational ERD: 51 documented entities plus two Phase-IV configuration tables (CHAIN_REGISTRY, SESSION_KEY_PERMISSION), grouped by migration tier (M1 core, M2, M3, stub, future work).

A.2 Indexing Strategy

B-tree indexes (the PostgreSQL default) support efficient retrieval on frequently queried columns:

Table A.1: Indexing strategy: B-tree indexes for time-window and high-frequency queries.

Index	Table / Column(s)	Rationale
idx_loan_- borrower	LOAN_REQUEST(borrower_id)	High-frequency lookup for borrower loan history.
idx_loan_status	LOAN_REQUEST(status, oracle_state)	Efficient filtering of application lifecycle and oracle pipeline states.
idx_- installment_due	INSTALLMENT(due_date, status) WHERE status = 'PENDING'	Partial-index range query for pending overdue installment detection.
idx_txn_created	TRANSACTION(created_at)	Time-window aggregation for 6-month and 1-year borrowing limits.
idx_txn_- borrower	TRANSACTION(borrower_id)	Per-borrower transaction history retrieval.
idx_market_- symbol	MARKET_DATA(symbol, recorded_at)	Composite index for price feed time-series queries.
idx_loan_bank_- status	LOAN(local_bank_id, status, created_at DESC)	Loan lifecycle queries by bank and status.
idx_loan_- borrower_active	LOAN(borrower_id, status) WHERE status = 'ACTIVE'	Partial index for per-borrower open-loan count.
idx_agent_- session_date	AGENT_ACTION_LOG(session_id, created_at DESC)	Per-session agent action history retrieval (join to SESSIONS for borrower).
idx_agent_- status	AGENT_ACTION_LOG(status) WHERE status = 'PENDING'	Partial index for the agent status-monitoring loop.
idx_risk_score	LOAN_RISK_ASSESSMENT(anomaly_score DESC) WHERE anomaly_score > 0.5	Partial index for AML alert queue (SAR workflow).
idx_ib_loan_- status	INTERBANK_LOAN(status, lender_institution_id)	IBLP default cascade and maturity monitoring.
idx_- institution_- type	INSTITUTION(institution_type, status)	Tier-scoped institution lookups for multi-entity operations.
idx_txn_- institution	TRANSACTION(origin_institution_id, created_at DESC)	Institution-to-institution ledger history retrieval.
idx_chainlink_- feed	CHAINLINK_PRICE_ROUND(feed_address, updated_at DESC)	Latest round lookup per Chainlink feed for LTV staleness checks.
idx_session_- parent	SESSIONS(parent_session_id) WHERE parent_session_id IS NOT NULL	Partial index for session lineage chain traversal.
idx_collateral_- borrower	COLLATERAL_POSITION(borrower_id, loan_id)	Health-factor lookup per borrower and per loan.
idx_event_log_- txhash	BLOCKCHAIN_EVENT_LOG(tx_hash)	Deduplication guard: event listener checks this before INSERT.
idx_event_log_- block	BLOCKCHAIN_EVENT_LOG(block_number, chain_id)	Catchup-sync replay from a given block.

A.3 Functional Dependencies

Table A.3: Representative functional dependencies.

Relation	Functional Dependency	Notes
LOAN_REQUEST	request_id → all attributes	Application row keyed by request identifier.
LOAN_REQUEST	borrower_id, local_bank_id → status, oracle_state, amount, ...	Lifecycle status and oracle pipeline oracle_state are separate columns.
LOAN	loan_id → all attributes	Disbursement record keyed by loan identifier.
LOAN	loan_request_id → loan_id	1:1 link from approved request to disbursed loan.
BORROWING_LIMIT	borrower_id → six_month_limit, one_year_limit, on_chain_enforced_limit, last_synced_block, ...	1:1 with BORROWER; event-listener owned projection.
CREDIT_PASSPORT	passport_id → all attributes; borrower_id → passport_id	Read-model projection of on-chain SBT; event-listener owned.
BORROWER	wallet_address → borrower_id, all attributes; registered_local_bank_id → home Local Bank; kyc_level → borrowing-cap tier (Section 4.2.7)	Unique candidate key in the prototype; Phase IV may use (wallet_address, chain_id).
INSTITUTION	institution_id → institution_type, name, wallet_address, contract_address, status	Supertype for cross-tier FK anchors.
NATIONAL_BANK	country_code → institution_id	One national reserve per nation (UNIQUE).
MODEL_REGISTRY	model_id → model_name, version, training_date, metrics, is_active	Model lineage for explainability audit.
LOAN_RISK_ASSESSMENT	assessment_id → all attributes; model_id → fraud_probability, anomaly_score, shap_vector	Per-loan ML inference; FK to MODEL_REGISTRY.
INSTALLMENT	loan_id, installment_number → amount, due_date, status, payment_event_log_id	Composite primary key.
SESSIONS	session_id → borrower_id, wallet_address, session_key_hash, session_key_scope, session_key_expires_at, expires_at, revoked	Retail-client agent sessions only at MVT; bank-authority sessions are Future Work.
AGENT_ACTION_LOG	action_id → session_id, tool_name, parameters, confirmation_turn_id, onchain_tx_hash, status, created_at	Links every write-tool execution to the user's consent turn in AGENT_CONVERSATION_TURN.
ASSETS	asset_id → symbol, name, asset_type, network, decimals, is_active; symbol → asset_id	symbol is a unique candidate key.
INTEREST_RATE_TIER	tier_id → base_rate, kink_utilization, rate_above_kink, max_rate, bank_tier_type, effective_from, effective_to, is_active	Eliminates transitive dependency; temporal versioning for governance changes.

A.4 Materialized Views and Agent Query Optimisation

The MCP tool `get_account_state` joins `BORROWER`, `CREDIT_PASSPORT`, `COLLATERAL_POSITION`, active `LOAN` rows, pending `INSTALLMENT` rows, `SAVINGS_ACCOUNT`, and `BORROWING_LIMIT`. To avoid six-to-seven table joins on every agent call, the schema defines a materialized view refreshed by the event listener after any event that modifies component tables:

```
CREATE MATERIALIZED VIEW mv_client_dashboard AS
SELECT
    b.borrower_id,
    b.wallet_address,
    b.kyc_level,
    cp.credit_score,
    cp.risk_tier,
    bl.six_month_remaining,
    bl.one_year_remaining,
    count(l.loan_id) FILTER (WHERE l.status = 'ACTIVE') AS active_loan_count,
    max(i.due_date) FILTER (WHERE i.status = 'PENDING') AS next_installment_due,
    sum(sa.balance) AS total_savings_balance,
    sum(col.locked_amount) AS total_collateral_locked
FROM borrower b
LEFT JOIN credit_passport cp USING (borrower_id)
LEFT JOIN borrowing_limit bl USING (borrower_id)
LEFT JOIN loan l USING (borrower_id)
LEFT JOIN installment i USING (loan_id)
LEFT JOIN savings_account sa USING (borrower_id)
LEFT JOIN collateral_position col USING (borrower_id)
GROUP BY b.borrower_id, b.wallet_address, b.kyc_level,
         cp.credit_score, cp.risk_tier,
         bl.six_month_remaining, bl.one_year_remaining;

CREATE UNIQUE INDEX ON mv_client_dashboard (borrower_id);
```

Refresh strategy: `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_client_dashboard` triggered by the event listener after processing events on projection or lending tables (Phase III).

A.5 Representative SQL Queries

The following queries illustrate Database Design coursework competencies (joins, aggregates, subqueries, and views) on the prototype schema. They map to FR-02, FR-04, and FR-07 in Table 3.1.

Q1 – Active loans per Local Bank with borrower KYC level (INNER JOIN + aggregate).

```
SELECT lb.name AS local_bank,
       COUNT(l.loan_id) AS active_loans,
       AVG(l.principal_amount) AS avg_principal
```

```

FROM loan l
JOIN local_bank lb ON l.local_bank_id = lb.institution_id
JOIN borrower b ON l.borrower_id = b.borrower_id
WHERE l.status = 'ACTIVE'
GROUP BY lb.institution_id, lb.name
ORDER BY active_loans DESC;

```

Q2 – Overdue installments with borrower contact hash (JOIN + filter).

```

SELECT i.loan_id, i.installment_number, i.due_date, i.amount,
       b.wallet_address, b.kyc_level
FROM installment i
JOIN loan l ON i.loan_id = l.loan_id
JOIN borrower b ON l.borrower_id = b.borrower_id
WHERE i.status = 'PENDING' AND i.due_date < CURRENT_DATE
ORDER BY i.due_date ASC;

```

Q3 – Borrowers exceeding six-month rolling limit (subquery).

```

SELECT b.borrower_id, b.wallet_address,
       bl.six_month_limit,
       (SELECT COALESCE(SUM(t.amount), 0)
        FROM transaction t
        WHERE t.borrower_id = b.borrower_id
              AND t.transaction_type = 'LOAN_DISBURSEMENT'
              AND t.created_at >= NOW() - INTERVAL '6 months') AS disbursed_6mo
FROM borrower b
JOIN borrowing_limit bl ON b.borrower_id = bl.borrower_id
WHERE (SELECT COALESCE(SUM(t.amount), 0) FROM transaction t
       WHERE t.borrower_id = b.borrower_id
             AND t.transaction_type = 'LOAN_DISBURSEMENT'
             AND t.created_at >= NOW() - INTERVAL '6 months')
       > bl.six_month_limit;

```

Q4 – High-risk assessments awaiting approver review (JOIN + ORDER BY).

```

SELECT lr.request_id, lra.composite_score, lra.anomaly_score,
       lra.created_at, mr.model_name, mr.version
FROM loan_request lr
JOIN loan_risk_assessment lra ON lr.request_id = lra.loan_request_id
JOIN model_registry mr ON lra.model_id = mr.model_id
WHERE lr.status = 'UNDER_REVIEW'
      AND lra.composite_score BETWEEN 0.4 AND 0.7
ORDER BY lra.composite_score DESC;

```

Q5 – Client dashboard via materialized view (simplified SELECT).

```

SELECT borrower_id, wallet_address, credit_score, risk_tier,
       active_loan_count, next_installment_due, total_savings_balance

```

```
FROM mv_client_dashboard  
WHERE borrower_id = $1;
```

Appendix B

Agent Harness Reference

B.1 Per-User Context and Prompt Assembly

The Express.js context injection layer:

```
{
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "conversation_history": [ "...last 10 turns..." ]
}
```

B.2 Session Lineage, Compression, and Toolset Scoping

Table B.1: MCP toolset scoping.

Toolset	Tools visible
read_only	Full-suite taxonomy (Future Work): all 9 read tools; the MVT deploys only the subset needed for the selected 3–5 tools.
loan_actions	read_only + loan, installment, and group write tools.
account-management	read_only + deposit, KYC, and reminder write tools.

B.3 Lifecycle Hooks and Optional Capabilities

Lifecycle hook middleware. Write-tool POST requests pass through a hook stack: (1) confirmation audit hook (Phase II) verifies a logged confirmation turn or returns HTTP 403; (2) session-key scope pre-check (Phase III); (3) AML pre-check against LOAN_RISK_ASSESSMENT and SECURITY_EVENT_LOG (Phase IV).

Agent capabilities.

- **Proactive installment reminders** via `get_installment_schedule` and `pay_installment`.
- **Credit Passport coaching** via `get_credit_score` and Table 4.22.
- **Loan calculator** with live FX from `get_market_data`.
- **Group lending coordination** and **KYC tier upgrade guidance** via write/read tools.
- **Cross-session context retrieval** via `get_session_history`.

Appendix C

Smart Contract Capabilities

Table C.1: Smart contract capabilities summary (17 modular contracts).

Contract	Module	Ph.	Status	Primary capability
WorldBankReserv	Tier 1	I	Specified	Reserve custody, national-bank registration, downward allocation
NationalBank	Tier 2	I	Specified	Local-bank registration, borrow upward, allocate downward
LocalBank	Tier 3	I	Specified	Client onboarding, approvers, LoanController, freezeAccount
LoanController	Lending	I-II	Shell / Ph. II	Request, approve, disburse, installment state machine
SavingsVault	Deposits	II	Specified	Variable-yield savings; reserve-gated withdrawals
FixedDeposit	Deposits	II+	Stub	Term deposits; early-withdrawal penalty
GroupLendingPool	Group	II	Specified	Group consent, shared liability, per-member disbursement
InsuranceFund	Risk	I-II	Should	5% interest capture; default coverage; PoR attestation
CurrentAccount	Payments	II+	Stub	Atomic transfers; recurring payments
InterBankLending	Multi-entity	II	Specified	Same-tier peer pools (1 / 7 / 30 day)
UpwardDepositFee	Multi-entity	II	Specified	Surplus repatriation to parent tier
SyndicatedLoan	Multi-entity	III	Planned	Multi-bank co-funding and consent vote
TranchedPool	Multi-entity	III	Planned	Senior / junior tranches; waterfall recovery
TreasurySwap	Multi-entity	III	Planned	Institutional oracle-priced asset swap
FXModule	Treasury / FX	III	Planned	Retail FX conversion and audit trail
LiquidationEngine	Risk	III	Planned	Health-factor monitor; liquidator bonus
NettingEngine	Multi-entity	—	Future Work	Multilateral settlement netting (stub only)

Appendix D: WorldBankReserve Contract Interface

The following Solidity interface documents the public API of the WorldBankReserve contract—the formally specified Tier 1 contract (Phase I deployment target). Three design annotations follow the listing:

```
1  pragma solidity ^0.8.20;
2
3  import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
4  import "@openzeppelin/contracts/access/AccessControl.sol";
5
6  contract WorldBankReserve is ReentrancyGuard, AccessControl {
7
8      bytes32 public constant NATIONAL_BANK_ROLE = keccak256("NATIONAL_BANK_ROLE");
9      bytes32 public constant AUDITOR_ROLE      = keccak256("AUDITOR_ROLE");
10
11     uint256 public totalReserve;
12     uint256 public minimumReserveRatio;
13     mapping(address => uint256) public allocatedTo;
14
15     event CapitalAllocated(address indexed nationalBank, uint256 amount);
16     event RepaymentReceived(address indexed nationalBank, uint256 amount);
17     event ReserveRatioUpdated(uint256 newRatio);
18
19     function allocateCapital(address nationalBank, uint256 amount)
20         external onlyRole(DEFAULT_ADMIN_ROLE) nonReentrant;
21
22     function recordRepayment(uint256 amount)
23         external onlyRole(NATIONAL_BANK_ROLE) nonReentrant;
24
25     function utilizationRate() external view returns (uint256);
26
27     function isReserveAdequate() external view returns (bool);
28
29     function getReserveSummary() external view returns (
30         uint256 totalDeposited,
31         uint256 totalLoaned,
32         uint256 reserveRatio,
33         uint256 insuranceFundBalance
34     );
35 }
```

Listing C.1: WorldBankReserve.sol – Tier 1 public interface. Full implementation in project repository.

Appendix E: Internal Industry Research Notes

Repository: `github.com/Yuvraajrahman/Crypto-World-Bank`; path `Documentation/2nd Phase (Bokhtiar)/Improvements/`.

Table C.2: Internal industry research corpus.

ID	Document	Thesis use
IR-1	<code>Binance_FTX_Full_Report.md</code>	CEX vs. CWB comparison; FTT/BNB token lesson; SAFU and Binance Earn analogues; PoR and commingling safeguards
IR-2	<code>blockchain platforms research.csv</code>	Five-exchange matrix (Binance, Bybit, Coinbase, OKX, Kraken)
IR-3	<code>Binance Business Analysis.md</code>	Exchange revenue taxonomy (fees, listing, OTC)
IR-4	<code>binance_software_engineering_architecture.md</code>	Nine-actor taxonomy (Section 4.2.7); use-case and sequence-diagram conventions
IR-5	<code>binance_architecture_research.md</code>	Off-chain PostgreSQL/Redis patterns (background; not cited in body text)

Appendix F: UI Prototype Wireframes (Figma)

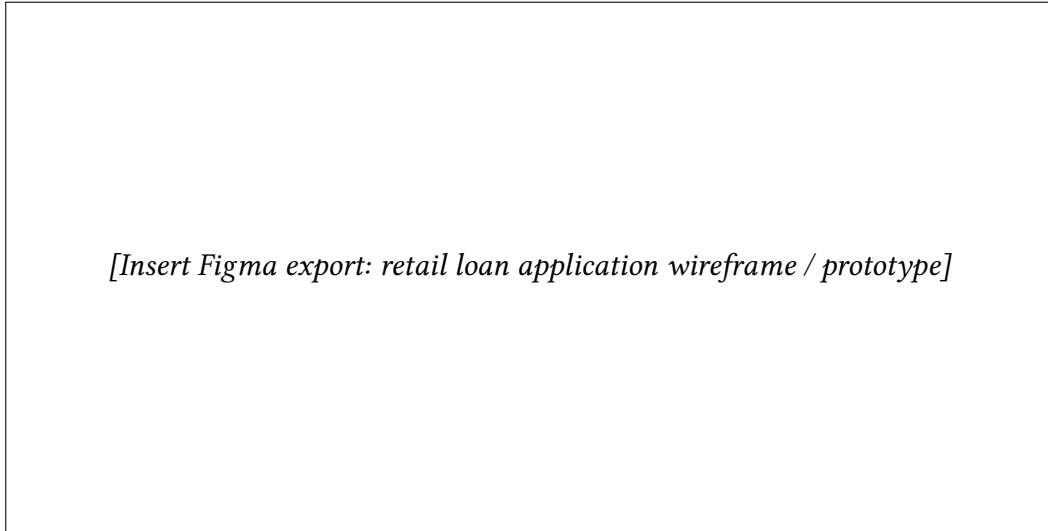


Figure C.1: Retail loan application prototype (Figma).

Retail loan application flow.

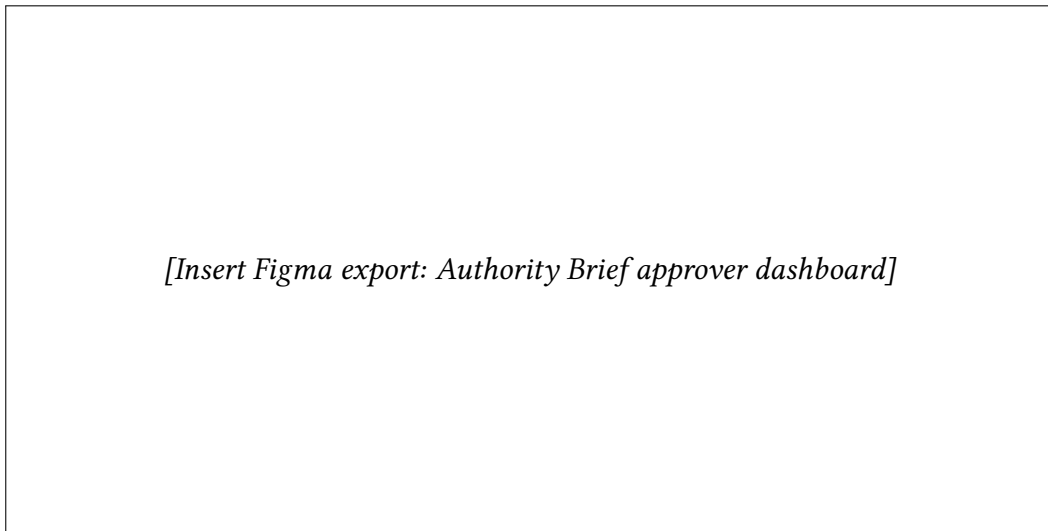


Figure C.2: Authority Brief approver dashboard prototype (Figma).

Authority Brief approver dashboard.

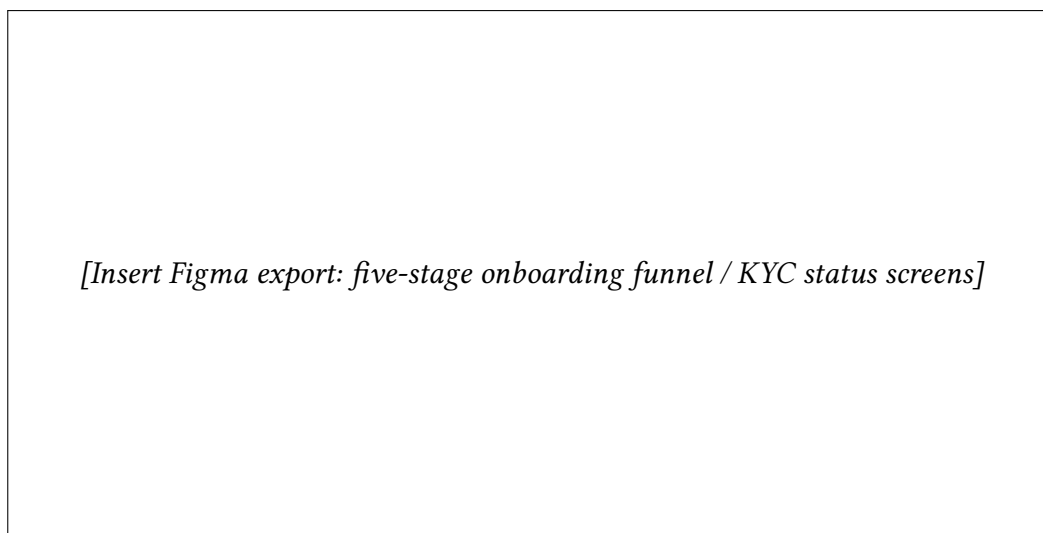


Figure C.3: Client onboarding funnel prototype (Figma).

Client onboarding funnel (KYC levels).

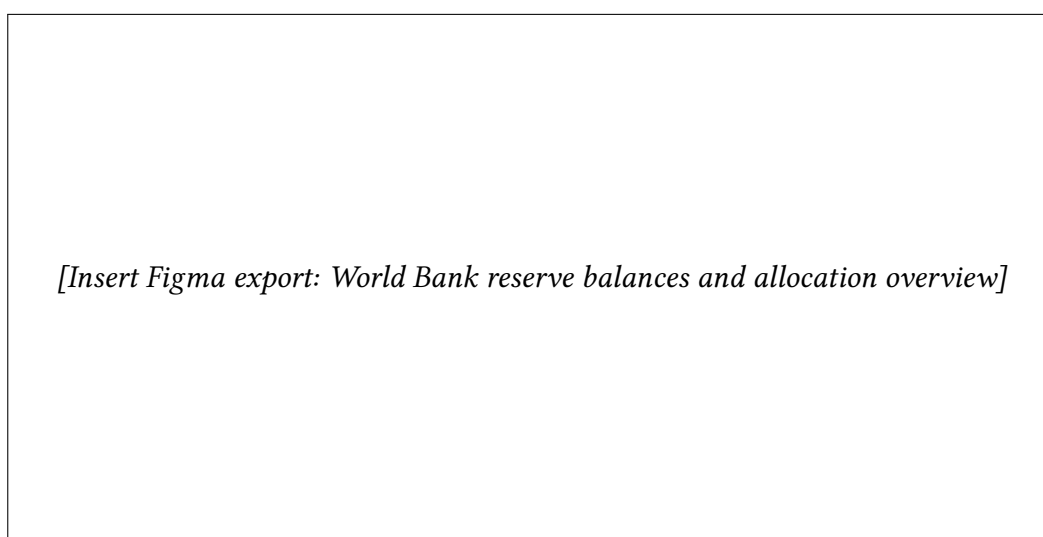


Figure C.4: Reserve transparency dashboard prototype (Figma).

Reserve transparency dashboard (Tier 1).

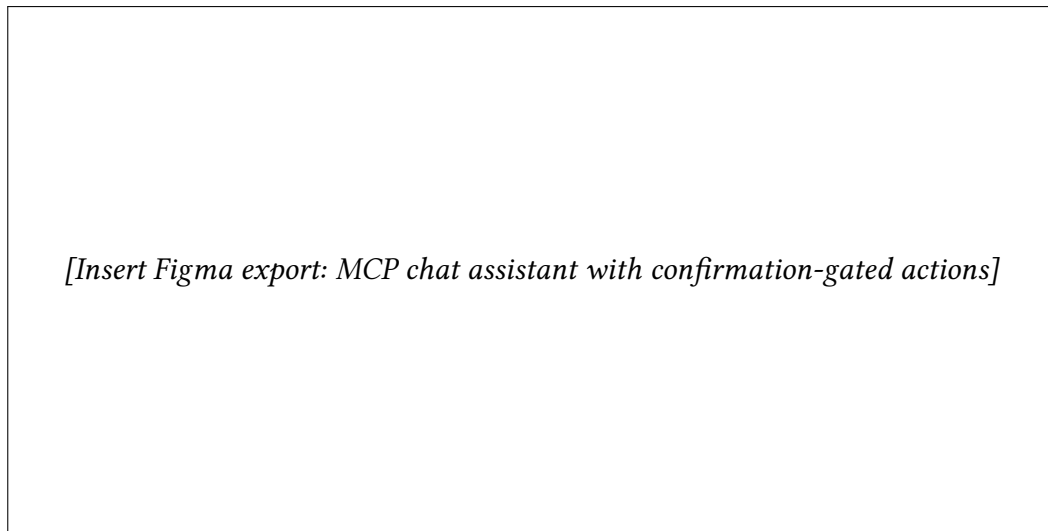


Figure C.5: Conversational banking agent prototype (Figma).

Conversational banking agent (chat).

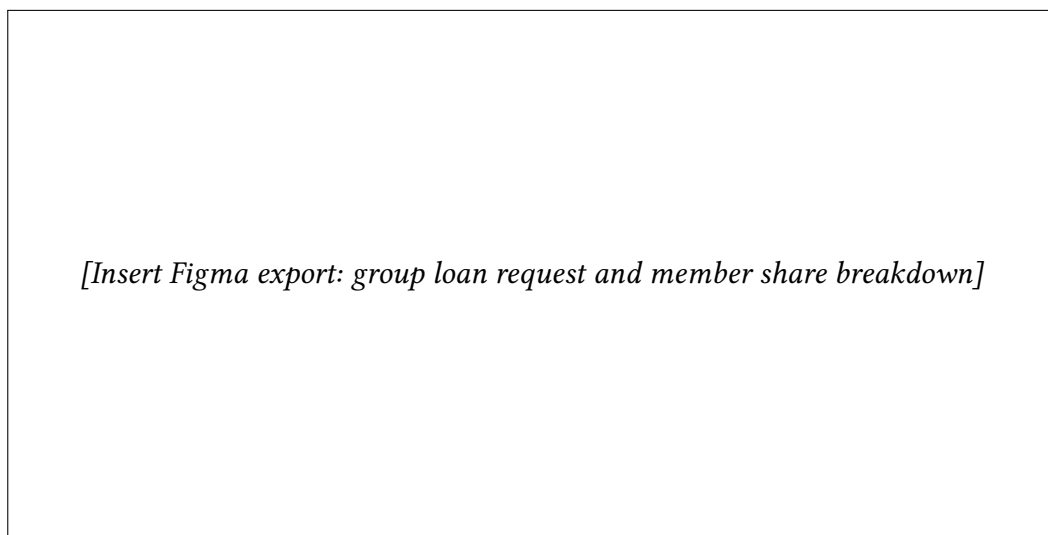
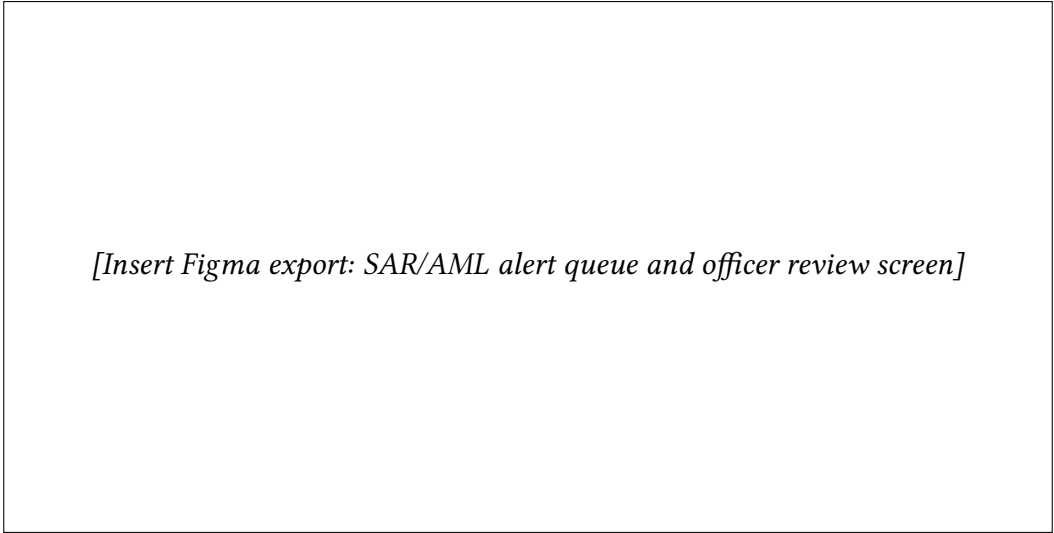


Figure C.6: Group solidarity lending prototype (Figma).

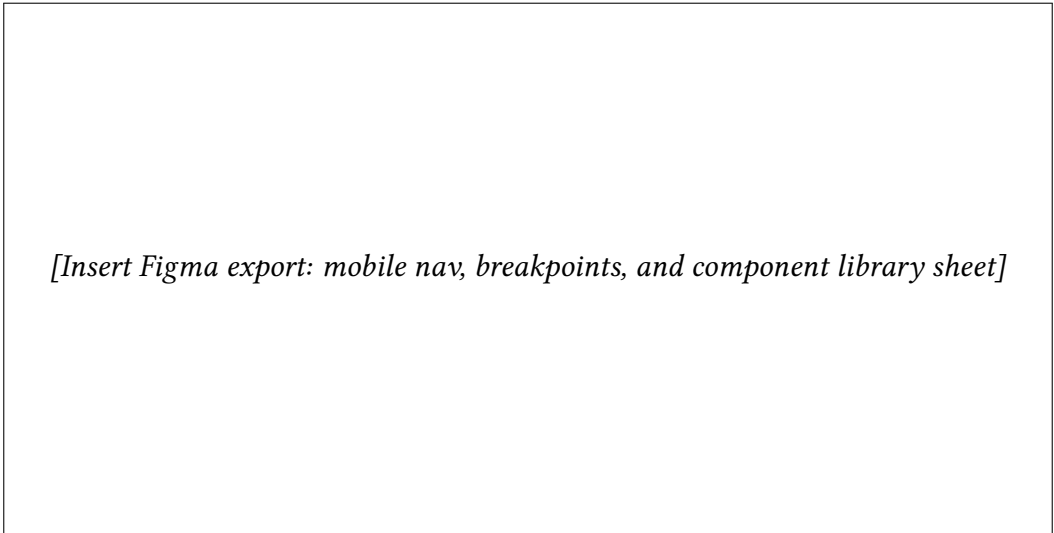
Group solidarity lending application.



[Insert Figma export: SAR/AML alert queue and officer review screen]

Figure C.7: Compliance and AML alert review prototype (Figma).

Compliance and AML alert review.



[Insert Figma export: mobile nav, breakpoints, and component library sheet]

Figure C.8: Mobile navigation and responsive layout prototype (Figma).

Mobile navigation and responsive layout.

References

- [1] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt. 2022. SoK: Decentralized Finance (DeFi). *arXiv preprint arXiv:2101.08778*. <https://arxiv.org/abs/2101.08778>
- [2] M. Bastankhah, V. Nadkarni, C. Jin, S. Kulkarni, and P. Viswanath. 2024. Thinking Fast and Slow: Data-Driven Adaptive DeFi Borrow-Lending Protocol. In *Proc. 6th Conf. Advances in Financial Technologies (AFT)* (LIPIcs, Vol. 316). Article 27, 23 pages. <https://doi.org/10.4230/LIPIcs.AFT.2024.27>
- [3] G. Palaiokrassas, S. Scherrers, I. Ofeidis, and L. Tassiulas. 2024. Leveraging Machine Learning for Multichain DeFi Fraud Detection. In *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*. <https://doi.org/10.1109/ICBC59979.2024.10634350>
- [4] D. Adom, E. O. Acheampong, and M. Boateng. 2022. LIME and SHAP: A Comparison of Model-Agnostic Approaches to Explainability in Loan Approval Systems. *International Journal of Advanced Computer Science and Applications* 13, 11. https://thesai.org/Downloads/Volume13No11/Paper_90-LIME_and_SHAP_A_Comparison.pdf
- [5] B. W. Tan. 2023. Central Bank Digital Currency and Financial Inclusion. *IMF Working Paper WP/23/69*. <https://www.imf.org/en/Publications/WP/Issues/2023/03/27/Central-Bank-Digital-Currency-and-Financial-Inclusion-531273>
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou. 2008. Isolation Forest. In *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pages 413–422. <https://doi.org/10.1109/ICDM.2008.17>
- [7] N. Atzei, M. Bartoletti, and T. Cimoli. 2017. A Survey of Attacks on Ethereum Smart Contracts (SoK). In *Proc. Int. Conf. Principles of Security and Trust (POST)*, pages 164–186. https://doi.org/10.1007/978-3-662-54455-6_8
- [8] DefiLlama. 2024. Lending Protocols — DeFi TVL and Protocol Rankings. <https://defillama.com/protocols/Lending>
- [9] World Bank. 2021. The Global Findex Database 2021: Financial Inclusion, Digital Payments, and Resilience. <https://www.worldbank.org/en/publication/globalfindex>
- [10] Aave. 2024. Aave Protocol Documentation. <https://docs.aave.com/>
- [11] Compound. 2024. Compound Protocol Documentation. <https://docs.compound.finance/>
- [12] World Bank. 2022. World Development Report 2022: Finance for an Equitable Recovery. <https://www.worldbank.org/en/publication/wdr2022>
- [13] International Finance Corporation (IFC). 2017. MSME Finance Gap: Assessment of the Shortfalls and Opportunities in Financing Micro, Small, and Medium Enterprises in Emerging Markets. World Bank. <https://openknowledge.worldbank.org/entities/publication/ff4c9839-21ac-5676-a23a-7cf6f745df0c>
- [14] Bank for International Settlements. 2024. DeFi Lending: Intermediation Without Information?. BIS Working Paper No. 1183. <https://www.bis.org/publ/work1183.pdf>

- [15] Committee on Payments and Market Infrastructures and Bank for International Settlements. 2016. Correspondent Banking — Technical Report. CPMI Papers No. 147. <https://www.bis.org/cpmi/publ/d147.pdf>
- [16] Ripple Labs. 2025. RLUSD Stablecoin Product Overview. Ripple Documentation. <https://ripple.com/solutions/stablecoin/>
- [17] World Bank. 2024. Remittance Prices Worldwide: Making Markets More Transparent. Migration and Development Brief 40. <https://remittanceprices.worldbank.org/>
- [18] DefiLlama. 2026. Aave V3 TVL, Fees, Revenue & Income Statement. March. <https://defillama.com/protocol/aave-v3>
- [19] DefiLlama. 2026. Compound V3 TVL, Fees, Revenue & Income Statement. March. <https://defillama.com/protocol/compound-v3>
- [20] Fensory. 2026. Sky Protocol Projects \$611M Revenue in 2026 as USDS Supply Targets \$20.6 Billion. February. <https://www.fensory.com/intelligence/rwa/sky-protocol-tokenization-regatta-solana-february-2026>
- [21] Fensory. 2026. Maple Finance — Institutional DeFi Lending Protocol. <https://www.fensory.com/insights/protocols/maple-finance>
- [22] Fensory. 2026. DeFi Credit Platforms Hit \$2.4B Milestone. March. <https://www.fensory.com/intelligence/rwa/defi-private-credit-platforms-2-4-billion-loans-ethereum>
- [23] Financial Stability Board. 2020. Enhancing Cross-border Payments: Stage 3 Roadmap. <https://www.fsb.org/2020/10/enhancing-cross-border-payments-stage-3-roadmap/>
- [24] GlobeNewswire. 2025. R3's Corda Leads Tokenized RWA Market with Over \$10 Billion in On-chain Assets. February. <https://www.globenewswire.com/news-release/2025/02/13/3025637/0/en/R3-s-Corda-leads-tokenized-RWA-market-with-over-10-billion-in-on-chain-assets-and-unrivalled-industry-adoption.html>
- [25] World Bank. 2025. World Bank Group Tracks Project Funds with New Blockchain Tool. Press Release, September. <https://www.worldbank.org/en/news/press-release/2025/09/26/world-bank-group-tracks-project-funds-with-new-blockchain-tool>
- [26] JPMorgan. 2025. Kinexys Digital Payments: Real-Time Multicurrency Payments. <https://www.jpmorgan.com/kinexys/index>
- [27] World Economic Forum. 2022. Why Decentralized Finance Is a Leapfrog Technology for the 1.1 Billion People Who Are Unbanked. September. <https://www.weforum.org/stories/2022/09/decentralized-finance-a-leapfrog-technology-for-the-unbanked/>
- [28] Opera Newsroom. 2026. 160M CELO Allocation Proposal to Grow Opera from Distribution Partner into Key Network Stakeholder. March. <https://press.opera.com/2026/03/19/opera-celo-partnership-2026/>
- [29] Morpho. 2026. Network Data. March. <https://data.morpho.org/>
- [30] Stellar. 2025. End of Year 2025 Report — Execution at Scale. <https://www.stellar.org/blog/foundation-news/2025-year-in-review>
- [31] Y. Chen, S. Bin, and H. He. 2024. Design and Implementation of a Multi-Chain Lending Model in Blockchain. In *Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT)*, pages 97–102.

- [32] S. Yang and W. Cui. 2023. An Evaluation System for DeFi Lending Protocols. *arXiv preprint arXiv:2303.01022*. <https://arxiv.org/abs/2303.01022>
- [33] J. Hartmann and O. Hasan. 2021. A Social-Capital Based Approach to Blockchain-Enabled Peer-to-Peer Lending. In *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*, pages 105–110. <https://hal.science/hal-03371872>
- [34] P. T. Hasan, H. K. T. Akhter, M. R. Tahsinur, A. B. Haque, A. K. M. N. Islam, and R. M. Rahman. 2022. Blockchain and Machine Learning for Fraud Detection: A Privacy-Preserving and Adaptive Incentive Based Approach. *IEEE Access*, vol. 10, pages 87115–87131. <https://ieeexplore.ieee.org/document/9857827>
- [35] Bank for International Settlements et al. 2020. Central Bank Digital Currencies: Foundational Principles and Core Features. BIS and Central Banks Joint Report. <https://www.bis.org/publ/othp33.htm>
- [36] M. Asaduzzaman, F. Hasib, and Z. B. Hafiz. 2020. Towards Using Blockchain Technology for Microcredit Industry in Bangladesh. In *Proc. 23rd Int. Conf. Computer and Information Technology (ICCIT)*, pages 1–6. <https://doi.org/10.1109/ICCIT51783.2020.9392730>
- [37] H. Kim and D. Kim. 2024. Optimal Gas Fee Minimization in DeFi: Enhancing Efficiency and Security on the Ethereum Blockchain. *IEEE Access*, vol. 12, pages 173810–173823. <https://ieeexplore.ieee.org/document/10750187>
- [38] X. Yuan. 2024. SHAP-based Interpretable Models for Credit Default Assessment Using Machine Learning. In *Proc. 14th Int. Conf. Software Technology and Engineering (ICSTE)*, pages 213–217. <https://doi.org/10.1109/ICSTE63875.2024.00044>
- [39] T. Dao, T. Trinh, and V. Pham. 2025. Optimizing Credit Scoring Models for Decentralized Financial Applications. In *Information and Communication Technology*, Springer. https://doi.org/10.1007/978-981-96-4282-3_36
- [40] A. Beniiche. 2020. A Study of Blockchain Oracles. *arXiv preprint arXiv:2004.07140*. <https://arxiv.org/pdf/2004.07140>
- [41] A. Pasdar, Y. C. Lee, and Z. Dong. 2023. Connect API with Blockchain: A Survey on Blockchain Oracle Implementation. *ACM Computing Surveys*, 55, 10. <https://doi.org/10.1145/3567582>
- [42] L. Gudgeon, S. M. Werner, D. Perez, and W. J. Knottenbelt. 2020. DeFi Protocols for Loanable Funds: Interest Rates, Liquidity and Market Efficiency. In *Proc. 2nd ACM Conf. Advances in Financial Technologies (AFT)*, pages 92–112. <https://doi.org/10.1145/3419614.3423254>
- [43] K. W. Wu. 2024. Strengthening DeFi Security: A Static Analysis Approach to Flash Loan Vulnerabilities. *arXiv preprint arXiv:2411.01230*.
- [44] Y. Li et al. 2025. A Comprehensive Study of Exploitable Patterns in Smart Contracts: From Vulnerability to Defense. *arXiv preprint arXiv:2504.21480*. <https://arxiv.org/abs/2504.21480>
- [45] F. Piper, K. Wolf, and J. Heiss. 2025. Privacy-Preserving On-chain Permissioning for KYC-Compliant Decentralized Applications. TU Berlin, *arXiv preprint arXiv:2510.05807*.
- [46] N. Decker. 2025. Zero-Knowledge Proofs: Cryptographic Model for Financial Compliance and Global Banking Security. *SSRN Working Paper No. 5170068*.

- [47] Grameen Bank. 2024. Grameen Bank at a Glance. Grameen Communications. <https://www.grameen-info.org/>
- [48] A. Carstens et al. 2021. Stablecoins: Risks, Potential and Regulation. *BIS Working Papers No. 905*, Bank for International Settlements. <https://www.bis.org/publ/work905.pdf>
- [49] J. A. Ocampo and K. Gallagher. 2024. The Role of Multilateral Development Banks and Development Assistance in the Provision of Global Public Goods. UNDP Background Paper. <https://hdr.undp.org/system/files/documents/background-paper-document/2024jaocampokdgonzaleztheroleofmultilateraldevlmtbanks.pdf>
- [50] F. A. Aponte-Novoa, A. L. S. Orozco, R. Villanueva-Polanco, and P. Wightman. 2021. The 51% Attack on Blockchains: A Mining Behavior Study. *IEEE Access*, vol. 9, pages 140549–140564. <https://doi.org/10.1109/ACCESS.2021.3119291>
- [51] Sygnum Bank. 2026. Institutional DeFi in 2025: The Disconnect Between Infrastructure and Allocation. Sygnum Blog, May 2025. <https://www.sygnum.com/blog/2025/05/30/institutional-defi-in-2025-the-disconnect-between-infrastructure-and-allocation/>
- [52] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, vol. 372, n71. <https://doi.org/10.1136/bmj.n71>
- [53] D. Cheng, X. Wang, Y. Zhang, and L. Zhang. 2022. Graph Neural Network for Fraud Detection via Spatial-Temporal Attention. *IEEE Trans. Knowledge and Data Engineering*, 34, 8, pages 3800–3813. <https://ieeexplore.ieee.org/document/9356317>
- [54] D. Wang, B. Wu, X. Yuan, L. Wu, Y. Zhou, and H. Cui. 2024. DeFiGuard: A Price Manipulation Detection Service in DeFi Using Graph Neural Networks. *arXiv preprint arXiv:2406.11157*. <https://arxiv.org/abs/2406.11157>
- [55] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*.
- [56] Z. Chen, J. Chen, M. Sra, et al. 2025. Standard Benchmarks Fail — Auditing LLM Agents in Finance Must Prioritize Risk. *arXiv preprint arXiv:2502.15865*. <https://arxiv.org/abs/2502.15865>
- [57] MicroSave Consulting. 2025. Shaping a Sustainable Microfinance Sector in Bangladesh. MSC Signature Project, September. https://www.microsave.net/signature_projects/shaping-a-sustainable-microfinance-sector-in-bangladesh-through-mscs-transformation-feasibility-study/
- [58] European Parliament and Council of the European Union. 2024. Regulation (EU) 2023/1114 of 31 May 2023 on Markets in Crypto-Assets (MiCA). *Official Journal of the European Union*, L 150/40, June 2023; fully applicable from 30 December. <https://eur-lex.europa.eu/eli/reg/2023/1114/oj>
- [59] European Banking Authority. 2024. Guidelines on the Minimum Content of the Governance Arrangements for Issuers of Asset-Referenced Tokens under MiCAR. EBA/GL//06, June. <https://www.eba.europa.eu/activities/single-rulebook/regulatory-activities/asset-referenced-and-e-money-tokens-micar/guidelines-internal-governance-arrangements-issuers-arts-under-micar>

- [60] United States Congress. 2025. Guiding and Establishing National Innovation for U.S. Stablecoins (GENIUS) Act. Pub. L. No. 119-27, July 18. <https://www.congress.gov/bill/119th-congress/senate-bill/919>
- [61] Bank for International Settlements Innovation Hub. 2024. Project mBridge: Connecting Economies through CBDC – Reaches Minimum Viable Product. BIS Innovation Hub Report, June. https://www.bis.org/about/bisih/topics/cbdc/mcbdc_bridge.htm
- [62] Bank for International Settlements and Central Banks of France, Japan, Korea, Mexico, Switzerland, the UK, and the US. 2024. Project Agora: Tokenization of Cross-Border Payments using Unified Ledgers. BIS Innovation Hub Working Paper, April. <https://www.bis.org/about/bisih/topics/fmis/agora.htm>
- [63] Bangladesh Bank. 2025. FE Circular No. 06: Electronic Communication for Letters of Credit and Trade Finance Instruments. Foreign Exchange Policy Department, January. <https://www.bb.org.bd/mediaroom/circulars/fepd/jan172025fepd06.pdf>
- [64] Bangladesh Bank. 2025. Financial Inclusion in Bangladesh: Progress and Gender-Disaggregated Outlook 2024–25. Special Publication SP-02, Financial Inclusion Department, June. <https://www.bb.org.bd/pub/special/SP2025-02.pdf>
- [65] BRAC. 2025. BRAC Microfinance Annual Performance Report 2025: Women-Centric Lending Outcomes. BRAC Research and Evaluation Division. <https://www.brac.net/program/microfinance/>
- [66] Ethereum Improvement Proposals. 2025. EIP-4626: Tokenized Vault Standard. Ethereum Foundation, April 2022. ; “EIP-7540: Asynchronous ERC-4626 Tokenized Vaults,” Ethereum Foundation, 2023. ; “EIP-3643: T-REX – Token for Regulated EXchanges,” Ethereum Foundation, 2021. See also: Spectral Finance, “On-Chain Credit Scoring for DeFi Lending,” 2023. ; RociFi Labs, “Undercollateralized DeFi Lending via On-Chain Credit Score,” 2023. ; Qwen Team (Alibaba Cloud), “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*. <https://eips.ethereum.org/EIPS/eip-4626>
- [67] Ethereum Improvement Proposals. 2022–2025. EIP-4626, EIP-7540, EIP-3643; Spectral Finance and RociFi documentation; Qwen Team. 2025. Qwen3 Technical Report (*arXiv:2505.09388*). <https://eips.ethereum.org/EIPS/eip-4626>
- [68] Elliptic. 2019. The Elliptic Data Set. Kaggle. <https://www.kaggle.com/datasets/ellipticco/elliptic-data-set>
- [69] M. Alizadeh, M. Gholampour, A. Imani, and J. Schulz. 2026. Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution. *arXiv preprint arXiv:2506.01055*, University of Zurich / IPM, June.
- [70] OWASP Foundation. 2026. OWASP Top 10 for Large Language Model Applications and Agents, 2026 Edition. OWASP Foundation. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [71] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [72] G. Aspris and J. Svec. 2025. Institutionalizing Risk Curation in Decentralized Credit. *arXiv preprint arXiv:2512.11976*. <https://arxiv.org/abs/2512.11976>
- [73] S. Jaffer. 1999. Microfinance and the Mechanics of Solidarity Lending. Harvard Law School, *SSRN Working Paper No. 162548*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=162548

- [74] R. Sherratt. 2016. From Solidarity to Coercion: The Dynamics of Group Liability. *Oxford Economic Papers*, 68, 4, pages 955–974. <https://doi.org/10.1093/oep/gpw018>
- [75] H. Fazliani, A. Pasdar, and Y. C. Lee. 2025. Ensemble-Based Semi-Supervised Learning for Illicit Account Detection in Ethereum DeFi Transactions. *arXiv preprint arXiv:2412.02408*, rev. October. <https://arxiv.org/abs/2412.02408>
- [76] H. Alhasawi, A. Almutairi, and S. Alharbi. 2025. FedFraud: A Federated Approach to Scalable and Trustworthy Financial Fraud Detection. *Security and Privacy*, Wiley. <https://doi.org/10.1002/spy2.70012>
- [77] European Parliament and Council of the European Union. 2026. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). *Official Journal of the European Union*, L 2024/1689, July 2024; high-risk credit-scoring provisions applicable from August. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [78] L. Dong, Y. Qiu, and F. Xu. 2023. Blockchain-Enabled Deep-Tier Supply Chain Finance. *Manufacturing & Service Operations Management*, 25, 6, pages 2021–2037. <https://doi.org/10.1287/msom.2022.1123>
- [79] U. Bindseil. 2020. Tiered CBDC and the Financial System. *ECB Working Paper Series*, no. 2351, January. <https://www.ecb.europa.eu/pub/pdf/scpwps/ecb.wp2351~c8c18bbd60.en.pdf>
- [80] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. Zhou. 2020. A Survey of Research on Retail Central Bank Digital Currency. *IMF Working Paper WP/20/104*, June. <https://www.imf.org/en/Publications/WP/Issues/2020/06/26/A-Survey-of-Research-on-Retail-Central-Bank-Digital-Currency-49069>
- [81] R. Auer and R. Böhme. 2020. The Technology of Retail Central Bank Digital Currency. *BIS Quarterly Review*, March; see also BIS Working Paper No. 948. https://www.bis.org/publ/qtrpdf/r_qt2003j.htm
- [82] L. Ante. 2025. From Adoption to Continuance: Stablecoins in Cross-Border Remittances and the Role of Digital and Financial Literacy. *Telematics and Informatics*, vol. 97, art. 102230. <https://doi.org/10.1016/j.tele.2024.102230>
- [83] Y. Li, Y. Xiang, Q. Wang, T. H. Yuen, A. Deppeler, and J. Yu. 2026. SoK: Stablecoins in Retail Payments. *arXiv preprint arXiv:2601.00196*. <https://arxiv.org/abs/2601.00196>
- [84] W. Du, C. Huang, and D. Scharfstein. 2026. Competing Rails for Cross-Border Payments: Banks, Fintechs, and Stablecoins. Harvard Business School Working Paper, February. <https://www.hbs.edu/faculty/Pages/item.aspx?num=66992>
- [85] M. Fröhlich, J. A. Vega Vermehren, F. Alt, and A. Schmidt. 2022. Implementation and Evaluation of a Point-Of-Sale Payment System Using Bitcoin Lightning. In *Proc. NordiCHI*. <http://www.medien.ifi.lmu.de/pubdb/publications/pub/froehlich2022nordichi/froehlich2022nordichi.pdf>
- [86] K. Zhang, T.-M. Choi, S.-H. Chung, Y. Dai, and X. Wen. 2024. Blockchain Adoption in Retail Operations: Stablecoins and Traceability. *European Journal of Operational Research*, 315, 1, pages 147–160. <https://doi.org/10.1016/j.ejor.2023.11.026>
- [87] J. Lin, M. Liu, S. Li, and X. Wang. 2025. SecurePay: Enabling Secure and Fast Payment Processing for Platform Economy. In *Proc. IEEE/ACM Int. Symp. Quality of Service (IWQoS)*; *arXiv:2505.17466*. <https://arxiv.org/abs/2505.17466>

- [88] Committee on Payments and Market Infrastructures. 2023. Considerations for the Use of Stablecoin Arrangements in Cross-Border Payments. Bank for International Settlements, July. <https://www.bis.org/cpmi/publ/d220.pdf>
- [89] A. Kosse, J. Lisack, and N. Boakye-Agyeman. 2023. The Impact of Stablecoins on the International Monetary and Financial System. *BIS Papers*, no. 170, February. <https://www.bis.org/publ/bppdf/bispap170.pdf>
- [90] E. Cerutti, M. Firat, and H. Pérez-Saiz. 2025. Estimating the Impact of Digital Money on Cross-Border Flows: Scenario Analysis Covering the Intensive Margin. *IMF Fintech Notes* 2025/002, February. <https://www.imf.org/-/media/files/publications/ftn063/2025/english/ftnea2025002.pdf>
- [91] S. Chaliasos, M. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, and B. Livshits. 2024. Smart Contract and DeFi Security: Insights from Tool Evaluations and Practitioner Surveys. In *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*, pages 160:1–160:13. <https://doi.org/10.1145/3597503.3623302>
- [92] K. Qin, L. Zhou, B. Livshits, and A. Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *Financial Cryptography and Data Security (FC)*, pages 3–32. https://doi.org/10.1007/978-3-662-63514-3_1
- [93] T. Mackinga, T. Nadahalli, and R. Wattenhofer. 2024. SoK: Attacks on DAOs. *arXiv preprint arXiv:2406.15071*. <https://arxiv.org/abs/2406.15071>
- [94] E. G. Weyl, P. Ohlhaver, and V. Buterin. 2022. Decentralized Society: Finding Web3’s Soul. *SSRN Working Paper No. 4105763*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4105763
- [95] SmartContractSecurity. 2020. SWC Registry: Smart Contract Weakness Classification and Test Cases. <https://swcregistry.io/>
- [96] G. Caldarelli. 2025. Can Artificial Intelligence Solve the Blockchain Oracle Problem? Unpacking the Challenges and Possibilities. *Frontiers in Blockchain*, vol. 8, art. 1682623. <https://doi.org/10.3389/fbloc.2025.1682623>
- [97] M. Gurupriya, R. Gayathri, C. Yadav, and U. Umamaheshwar. 2025. Blockchain-Powered Platform for Microloans and Lending Solutions. In *Proc. Int. Conf. on Intelligent Computing, Instrumentation and Control Technologies (ICICT)*, pages 1–6. <https://doi.org/10.1109/ICTEST64710.2025.11042350>
- [98] V. Buterin, Y. Weiss, K. Gazso, N. Patel, D. Tirosh, S. Nacson, and T. Hess. 2021. ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals. <https://eips.ethereum.org/EIPS/eip-4337>
- [99] Behaviour-Centric Cybersecurity Center (BCCC), York University. 2025. BCCC-DeFiFraudTrans-2025: A Labelled Ethereum DeFi Fraud Transaction Dataset. Accessed June 29, 2026. <https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/defi-fraud-transactions-bccc-defifraudtrans-2025/>
- [100] P. K. Ozili. 2024. Central Bank Digital Currency Adoption Challenges. *Journal of Central Banking Theory and Practice*, 13, 1, pages 5–24. <https://doi.org/10.2478/jcbtp-2024-0001>
- [101] McKinsey & Company. 2018. Global Payments 2018: A Dynamic Industry Continues to Defy Predictions. McKinsey Global Payments Map. <https://www.mckinsey.com/>

industries/financial-services/our-insights/global-payments-a-dynamic-industry-continues-to-defy-predictions

- [102] Y. Wang, Q. Zhang, K. Li, Y. Tang, J. Chen, X. Luo, and T. Chen. 2021. iBatch: Saving Ethereum Fees via Secure and Cost-Effective Batching of Smart-Contract Invocations. In *Proc. ACM Joint Eur. Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, pages 777–789. <https://doi.org/10.1145/3468264.3468574>
- [103] World Inequality Lab. 2026. Exorbitant Privilege. *World Inequality Report 2026*. <https://wir2026.wid.world/insight/exorbitant-privilege/>
- [104] G. Wood. 2023. Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper). Ethereum Foundation. <https://ethereum.github.io/yellowpaper/paper.pdf>
- [105] Y. Wang, J. Liu, and Z. Chen. 2020. ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts. *IEEE Trans. Network Science and Engineering*, vol. 8, no. 2, pages 1133–1144. <https://doi.org/10.1109/TNSE.2020.2968505>
- [106] J.-W. Liao, T.-T. Tsai, C.-K. He, and C.-W. Tien. 2019. SoliAudit: Smart Contract Vulnerability Assessment Based on Machine Learning and Fuzz Testing. In *Proc. 6th Int. Conf. Internet of Things: Systems, Management and Security (IOTSMS)*, pages 458–465. <https://doi.org/10.1109/iotsms48152.2019.8939256>
- [107] Federal Reserve Bank of New York and Bank for International Settlements Innovation Hub. 2025. Project Pine: Central Bank Open Market Operations with Smart Contracts. BIS Innovation Hub / NY Fed Joint Report, May. <https://www.bis.org/publ/othp95.htm>